

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 4_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 26.5

Section 1 : Coding

1. Problem Statement

You are a delivery manager who needs to plan the delivery route for a fleet of trucks to a set of distribution centers in a city. Each distribution center is at a different location, and the cost of traveling between centers is known.

Your goal is to find the shortest route that allows a truck to visit each distribution center exactly once and return to the starting point, using the Traveling Salesman Problem (TSP). Write a program to calculate the minimum cost of this route and output the optimal path.

Note: Solve using Branch and Bound approach.

Input Format

The first line contains an integer n, the number of distribution centers.

The next n lines contain n space-separated integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from distribution center i to distribution center j.

Output Format

The first line prints:

Minimum Cost is: X

The second line prints:

Optimal Path: followed by the sequence of visited centers ending at the start node

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 3 9 5

3 0 6 4

9 6 0 7

5 4 7 0

Output: Minimum Cost is: 21

Optimal Path: 0 1 2 3 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#include <stdbool.h>
```

```
#define MAX 10
```

```
int n;  
int adj[MAX][MAX];  
int final_path[MAX + 1];  
bool visited[MAX];  
int min_cost = INT_MAX;
```

```
void copyToFinal(int curr_path[]) {  
    for (int i = 0; i < n; i++)  
        final_path[i] = curr_path[i];  
    final_path[n] = curr_path[0];  
}
```

```
int firstMin(int i) {  
    int min = INT_MAX;  
    for (int k = 0; k < n; k++)  
        if (adj[i][k] < min && i != k)  
            min = adj[i][k];  
    return min;  
}
```

```
int secondMin(int i) {  
    int first = INT_MAX, second = INT_MAX;  
    for (int j = 0; j < n; j++) {  
        if (i == j) continue;  
        if (adj[i][j] <= first) {  
            second = first;  
            first = adj[i][j];  
        } else if (adj[i][j] <= second && adj[i][j] != first) {  
            second = adj[i][j];  
        }  
    }  
    return second;  
}
```

```
void tsp_bound(int curr_bound, int curr_weight, int level, int curr_path[]) {  
    if (level == n) {  
        if (adj[curr_path[level - 1]][curr_path[0]] != 0) {  
            int curr_res = curr_weight + adj[curr_path[level - 1]][curr_path[0]];  
            if (curr_res < min_cost) {
```

```

        copyToFinal(curr_path);
        min_cost = curr_res;
    }
}
return;
}

```

```

for (int i = 0; i < n; i++) {
    if (adj[curr_path[level - 1]][i] != 0 && !visited[i]) {
        int temp = curr_bound;
        curr_weight += adj[curr_path[level - 1]][i];

        if (level == 1)
            curr_bound -= ((firstMin(curr_path[level - 1]) + firstMin(i)) / 2);
        else
            curr_bound -= ((secondMin(curr_path[level - 1]) + firstMin(i)) / 2);

        if (curr_bound + curr_weight < min_cost) {
            curr_path[level] = i;
            visited[i] = true;
            tsp_bound(curr_bound, curr_weight, level + 1, curr_path);
        }

        curr_weight -= adj[curr_path[level - 1]][i];
        curr_bound = temp;
        for (int j = 0; j < n; j++) visited[j] = false;
        for (int j = 0; j < level; j++) visited[curr_path[j]] = true;
    }
}
}

```

```

int main() {
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);

    int curr_path[MAX + 1];
    int curr_bound = 0;
    for (int i = 0; i < n; i++)
        curr_bound += (firstMin(i) + secondMin(i));
}

```

```

curr_bound = (curr_bound & 1) ? curr_bound / 2 + 1 : curr_bound / 2;
visited[0] = true;
curr_path[0] = 0;

tsp_bound(curr_bound, 0, 1, curr_path);

printf("Minimum Cost is: %d\n", min_cost);
printf("Optimal Path: ");
for (int i = 0; i <= n; i++)
    printf("%d%s", final_path[i], (i == n) ? "" : " ");
printf("\n");

return 0;
}

```

Status : Partially correct

Marks : 2/10

2. Problem Statement

You are an art collector who wants to visit a set of art galleries in a city. Each gallery is at a different location, and the cost of traveling between galleries is known.

Your goal is to find the shortest route that allows you to visit each gallery exactly once and return to your starting point, using the traveling salesman problem (TSP). Write a program to calculate the minimum cost of this route.

Note: Solve it using Branch and Bound technique.

Input Format

The first line contains an integer n , the number of galleries.

The next n lines contain n integers each, representing the adjacency matrix.

The i th integer on the j th line represents the cost of traveling from gallery i to gallery j .

Output Format

The output displays "Minimum Cost is: " followed by the minimum cost of the route that visits each gallery exactly once and returns to the starting gallery.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: Minimum Cost is: 80

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX 10
```

```
int n;
```

```
int adj[MAX][MAX];
```

```
int final_path[MAX + 1];
```

```
int visited[MAX];
```

```
int final_res = INT_MAX;
```

```
// Function to find the minimum edge cost connected to a node
```

```
int firstMin(int i) {
```

```
    int min = INT_MAX;
```

```
    for (int k = 0; k < n; k++)
```

```
        if (adj[i][k] < min && i != k)
```

```
            min = adj[i][k];
```

```
    return min;
```

```
}
```

```
// Function to find the second minimum edge cost connected to a node
```

```
int secondMin(int i) {
```

```
    int first = INT_MAX, second = INT_MAX;
```

```
    for (int j = 0; j < n; j++) {
```

```

    if (i == j) continue;
    if (adj[i][j] <= first) {
        second = first;
        first = adj[i][j];
    } else if (adj[i][j] <= second && adj[i][j] != first) {
        second = adj[i][j];
    }
}
return second;
}

```

// Branch and Bound algorithm function

```

void tspRec(int curr_bound, int curr_weight, int level, int path[]) {
    if (level == n) {
        // Check if there is an edge from last node back to start
        if (adj[path[level - 1]][path[0]] != 0) {
            int curr_res = curr_weight + adj[path[level - 1]][path[0]];
            if (curr_res < final_res) {
                final_res = curr_res;
            }
        }
    }
    return;
}

```

```

for (int i = 0; i < n; i++) {
    if (adj[path[level - 1]][i] != 0 && !visited[i]) {
        int temp = curr_bound;
        curr_weight += adj[path[level - 1]][i];

        if (level == 1)
            curr_bound -= ((firstMin(path[level - 1]) + firstMin(i)) / 2);
        else
            curr_bound -= ((secondMin(path[level - 1]) + firstMin(i)) / 2);

        // If current bound + current weight < final_res, explore further
        if (curr_bound + curr_weight < final_res) {
            path[level] = i;
            visited[i] = 1;
            tspRec(curr_bound, curr_weight, level + 1, path);
        }
    }
}

```

// Backtrack

```

        curr_weight -= adj[path[level - 1]][i];
        curr_bound = temp;
        for (int j = 0; j < n; j++) visited[j] = 0;
        for (int j = 0; j <= level; j++) visited[path[j]] = 1;
    }
}

void solve() {
    int curr_bound = 0;
    int path[MAX + 1];

    for (int i = 0; i < n; i++) {
        curr_bound += (firstMin(i) + secondMin(i));
    }
    // Round off bound
    curr_bound = (curr_bound & 1) ? (curr_bound / 2) + 1 : (curr_bound / 2);

    for (int i = 0; i < n; i++) visited[i] = 0;

    // Start from node 0
    visited[0] = 1;
    path[0] = 0;
    tspRec(curr_bound, 0, 1, path);
}

int main() {
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    if (n == 1) {
        printf("Minimum Cost is: 0\n");
        return 0;
    }

    solve();
    printf("Minimum Cost is: %d\n", final_res);
}

```



```
} return 0;
```

Status : Partially correct

Marks : 4.5/10

3. Problem Statement

Sarah, a young entrepreneur starting her e-commerce business, has a limited budget for storing products in her warehouse. Each product has a specific weight and value, and Sarah aims to maximize the total value of the products in her warehouse without exceeding the weight capacity.

To achieve this, Sarah decided to apply the dynamic programming technique, which helps explore potential product combinations and avoid unnecessary calculations.

Input Format

The first line contains a single integer N , representing the number of products.

The next N lines each contain two integers: $\text{weight}[i]$ and $\text{value}[i]$, where $\text{weight}[i]$ is the weight of the product and $\text{value}[i]$ is the value (profit) of the product.

The next line contains a single integer W , representing the weight capacity of the warehouse.

Output Format

The output prints a single integer, representing the maximum total profit.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5
2 40
3 50
1 100

5 95
3 30
10

Output: 245

Answer

```
#include <stdio.h>
```

```
// Function to find the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

```
// Function to solve the 0/1 Knapsack Problem
int knapSack(int W, int wt[], int val[], int n) {
    int i, w;
    // Create a 2D array to store the maximum profit
    int dp[n + 1][W + 1];
```

```
    // Build the table in bottom-up manner
    for (i = 0; i <= n; i++) {
        for (w = 0; w <= W; w++) {
            // Base Case: No items or capacity is 0
            if (i == 0 || w == 0) {
                dp[i][w] = 0;
            }
            // If weight of current item is less than or equal to current capacity
            else if (wt[i - 1] <= w) {
                // Max of (including the current item, excluding the current item)
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]);
            }
            // If weight of current item is greater than current capacity
            else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }
    // The maximum profit is stored in dp[n][W]
    return dp[n][W];
}
```

```
int main() {
```

```

int n, W;

// Read number of products
if (scanf("%d", &n) != 1) return 0;

int val[n], wt[n];

// Read weight and value for each product
for (int i = 0; i < n; i++) {
    scanf("%d %d", &wt[i], &val[i]);
}

// Read warehouse weight capacity
scanf("%d", &W);

// Call knapSack function and print the result
printf("%d\n", knapSack(W, wt, val, n));

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

You are a savvy thief preparing to rob a jewelry store. You have a knapsack with a limited capacity and a list of items, each with a certain weight and a corresponding profit.

Your objective is to maximize the total profit you can obtain by selecting items to place in your knapsack while staying within its weight capacity. Write a program to help you calculate the maximum profit you can achieve.

Note: Use the 0/1 knapsack method to solve the program.

Example

Input:

3

60 10
100 20
120 30
50

Output:
220

Explanation:

By selecting the second and third items, you achieve a maximum profit of Rs. 220. The sum of their profits (100 + 120) gives you the highest total profit while not exceeding the knapsack's weight capacity of 50.

Input Format

The first line contains an integer N, representing the number of items available in the store.

Each of the next N lines contains two space-separated integers P and W, where P is the profit that can be obtained by stealing the item and W is the weight of the item.

The last line contains an integer C representing the weight capacity of your knapsack.

Output Format

The output displays an integer representing the maximum total profit that you can achieve by selecting items for your knapsack.

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 3
60 10
100 20
120 30

50

Output: 220

Answer

```
#include <stdio.h>
```

```
// Function to find the maximum of two integers
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int main() {  
    int n, capacity;
```

```
// Read the number of items  
if (scanf("%d", &n) != 1) return 0;
```

```
int profit[n], weight[n];
```

```
// Read profit and weight for each item  
for (int i = 0; i < n; i++) {  
    scanf("%d %d", &profit[i], &weight[i]);  
}
```

```
// Read the knapsack capacity  
scanf("%d", &capacity);
```

```
// dp[i][w] will store the maximum profit with i items and capacity w  
int dp[n + 1][capacity + 1];
```

```
// Build table dp[][] in bottom-up manner
```

```
for (int i = 0; i <= n; i++) {  
    for (int w = 0; w <= capacity; w++) {  
        if (i == 0 || w == 0) {  
            // Base case: 0 items or 0 capacity = 0 profit  
            dp[i][w] = 0;  
        } else if (weight[i - 1] <= w) {  
            // If weight of current item is less than current capacity,  
            // take maximum of:  
            // 1. Including the item  
            // 2. Excluding the item  
            dp[i][w] = max(profit[i - 1] + dp[i - 1][w - weight[i - 1]], dp[i - 1][w]);  
        } else {  
            dp[i][w] = dp[i - 1][w];  
        }  
    }  
}
```

```
    } else {  
        // Current item is too heavy, cannot include it  
        dp[i][w] = dp[i - 1][w];  
    }  
}  
  
// The maximum profit is at the last cell  
printf("%d\n", dp[n][capacity]);  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 2_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

In a kingdom far away, two types of soup are prepared daily — Soup A and Soup B. The royal chef always starts with n milliliters of each soup. Each time a servant is served, one of the following four serving operations is chosen at random (each with equal probability of 0.25):

Serve 100 ml of Soup A and 0 ml of Soup B

Serve 75 ml of Soup A and 25 ml of Soup B

Serve 50 ml of Soup A and 50 ml of Soup B

Serve 25 ml of Soup A and 75 ml of Soup B

The serving continues until either or both soups become empty. If the required soup amount for an operation exceeds the available quantity,

serve as much as possible (up to what remains).

You are to compute the probability that Soup A becomes empty first, plus half the probability that both soups become empty at the same time.

Special Note:

Since all operations serve soup in multiples of 25 ml, you must first scale n down to units of 25 ml before recursive processing:

Example

Sample Input 1

$n = 50$

Sample Output 2

0.625

Explanation

If we choose the first two operations, A will become empty first.

For the third operation, A and B will become empty at the same time.

For the fourth operation, B will become empty first.

So the total probability of A becoming empty first plus half the probability that A and B become empty at the same time, is $0.25 * (1 + 1 + 0.5 + 0) = 0.625$.

Input Format

The input consists of an integer n , representing the total volume of the soup.

Output Format

The output prints the probability that soup A will be empty first, plus half the probability that A and B become empty at the same time.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 50

Output: 0.625

Answer

You are using Python

```
def soup_serving(n):
    m = (n + 24) // 25
    memo={}
    def dp(a,b):
        if a<=0 and b<=0:
            return 0.5
        if a<=0:
            return 1.0
        if b<=0:
            return 0.0
        if (a,b) in memo:
            return memo[a,b]
        memo[a,b]=0.25*(dp(a-4,b)+dp(a-3,b-1)+dp(a-2,b-2)+dp(a-1,b-3))
        return memo[a,b]
    return round(dp(m,m),3)
n=int(input())
print(soup_serving(n))
```

Status : Correct

Marks : 10/10

2. Problem Statement

In a unique team-building exercise, a company has decided to distribute a certain number of resources among its departments based on mathematical powers. The company has x resources that need to be distributed such that each department receives a different power of an integer. Specifically, each department i receives n resources, where n is a given power.

George, the resource manager, needs to find all possible ways to distribute exactly x resources using the powers of integers starting from 1. Each integer's power can only be used once.

Write a program to help George determine the number of ways to distribute x resources in this manner using a recursive manner.

Example 1

Input:

10

2

Output:

1

Explanation:

$x = 10$

$n = 2$

$10 = 12 + 32$, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

$x = 100$

$n = 2$

$100 = 102$ OR $62 + 82$ OR $12 + 32 + 42 + 52 + 72$, hence total 3 possibilities.

Input Format

The first line of input consists of the integer x , representing the total number of resources.

The second line consists of the integer n , representing the power to which the integers are raised.

Output Format

The output should be a single integer representing the number of ways to distribute exactly x resources.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 100

2

Output: 3

Answer

You are using Python

```
import sys
```

Increase recursion depth for larger x and small n

```
sys.setrecursionlimit(2000)
```

```
def count_ways(target,power,current_val):
```

```
    value=current_val**power
```

```
    if target==0:
```

```
        return 1
```

```
    if value>target:
```

```
        return 0
```

```
    include=count_ways(target-value,power,current_val+1)
```

```
    exclude=count_ways(target,power,current_val+1)
```

```
    return include+exclude
```

```
def solve():
```

```
    try:
```

```
        line1=sys.stdin.readline()
```

```
        if not line1:
```

```
            return
```

```
        x=int(line1.strip())
```

```
        line2=sys.stdin.readline()
```

```
        if not line2:
```

```
            return
```

```
        n=int(line2.strip())
```

```
        result=count_ways(x,n,1)
```

```
print(result)
except ValueError:
    pass
if __name__ == "__main__":
    solve()
```

Status : Correct

Marks : 10/10

3. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

Input Format

The input consists of a single positive integer n.

Output Format

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 21

Output: 3

Answer

You are using Python

```

def countSetBits(n):
    count=0
    while n>0:
        n&=(n-1)
        count+=1
    return count
if __name__ == "__main__":
    try:
        n=int(input().strip())
        print(countSetBits(n))
    except EOFError:
        pass

```

Status : Correct

Marks : 10/10

4. Problem Statement

Given a square matrix of size $n \times n$, rotate the elements along each layer (or boundary) of the matrix by one position in a clockwise direction. The rotation should be performed layer by layer, starting from the outermost layer and proceeding inward.

Examples:

Input 2:

3 3

1 2 3

4 5 6

7 8 9

Output 2:

4 1 2

7 5 3

8 9 6

Explanation:

For a clockwise boundary rotation by one step:

We start with the outermost layer (there's only one layer in a 3×3 matrix)

We save the top-left element (1)

Move left edge upward: 4 1, 7 4

Move bottom edge leftward: 8 7, 9 8

Move right edge downward: 6 9, 3 6

Move top edge rightward: 2 3

Place the saved element (1) in the second position of the top row, so 1 2

Input Format

The first line contains two integers R and C — the number of rows and columns in the matrix.

The next R lines each contain C integers, representing the elements of the matrix.

Output Format

The output displays the rotated matrix after performing the clockwise boundary rotation.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 4 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

Output: 5 1 2 3

9 10 6 4

13 11 7 8

14 15 16 12

Answer

// You are using GCC

#include <stdio.h>

int main() {

int n;

scanf("%d %d", &n, &n); // since R = C

int a[5][5];

// Input matrix

for(int i = 0; i < n; i++) {

for(int j = 0; j < n; j++) {

scanf("%d", &a[i][j]);

}

// Rotate layer by layer

for(int layer = 0; layer < n / 2; layer++) {

int top = layer;

int bottom = n - 1 - layer;

int left = layer;

int right = n - 1 - layer;

int temp = a[top][left]; // store top-left

// Move left column up

for(int i = top; i < bottom; i++) {

a[i][left] = a[i + 1][left];

}

// Move bottom row left

for(int i = left; i < right; i++) {

a[bottom][i] = a[bottom][i + 1];

}

// Move right column down

for(int i = bottom; i > top; i--) {

a[i][right] = a[i - 1][right];

}

// Move top row right

for(int i = right; i > left + 1; i--) {

```

    a[top][i] = a[top][i - 1];
}

// Place stored element
a[top][left + 1] = temp;
}

// Print result
for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        printf("%d ", a[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Given a 2D matrix of size ROW $\times$ COL, print all elements of the matrix in diagonal order.

Diagonal Order Definition:

Start from the first element at position (0,0). Traverse all diagonals starting from the leftmost column going downward, then continue with diagonals starting from the bottom row going rightward. Within each diagonal, print elements from the top-right corner to bottom-left corner.

Example:

Input matrix

5 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

17 18 19 20

Diagonal printing of the above matrix is

1

5 2

9 6 3

13 10 7 4

17 14 11 8

18 15 12

19 16

20

Input Format

The first line of input contains two integers, ROW and COL.

The next ROW lines each contain COL space-separated integers

Output Format

The output consists of multiple lines, one for each diagonal. Each line contains the matrix elements of that diagonal in the order from top-right to bottom-left, separated by spaces.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

17 18 19 20

Output: 1

5 2

9 6 3

13 10 7 4

17 14 11 8

18 15 12

19 16

20

Answer

// You are using GCC

#include <stdio.h>

int main() {

int ROW, COL;

scanf("%d %d", &ROW, &COL);

int a[10][10];

// Input matrix

for(int i = 0; i < ROW; i++) {

for(int j = 0; j < COL; j++) {

scanf("%d", &a[i][j]);

}

}

// Diagonals starting from first column

for(int k = 0; k < ROW; k++) {

int i = k;

int j = 0;

while(i >= 0 && j < COL) {

printf("%d ", a[i][j]);

i--;

j++;

}

printf("\n");

}

// Diagonals starting from bottom row (excluding first column)

for(int k = 1; k < COL; k++) {

int i = ROW - 1;

```
int j = k;

while(i >= 0 && j < COL) {
    printf("%d ", a[i][j]);
    i--;
    j++;
}
printf("\n");
}

return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 3_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 11

Section 1 : MCQ

1. What is the primary characteristic of the Brute Force approach to solving the Closest-Pair problem?

Answer

Computing the distance between every pair of points and selecting the minimum

Status : Correct

Marks : 1/1

2. In the brute force convex hull algorithm (Jarvis March/Gift Wrapping), what is the next point selected after identifying a point on the hull?

Answer

The point that creates the smallest counterclockwise turn relative to the current point

Status : Correct

Marks : 1/1

3. In C, if you have a structure `Point { int x; int y; }`, what is the correct formula to calculate the squared distance between two points `p1` and `p2` to avoid unnecessary floating-point operations?

Answer

`(p1.x - p2.x)*(p1.x - p2.x) + (p1.y - p2.y)*(p1.y - p2.y)`

Status : Correct

Marks : 1/1

4. Given an array of `n` points in a 2D plane, what is the exact time complexity of the Brute Force Closest-Pair algorithm?

Answer

$O(n)$

Status : Wrong

Marks : 0/1

5. Which geometric test is essential for determining if a point lies to the left or right of a directed line in the Convex Hull brute force algorithm?

Answer

The Orientation test (Cross Product)

Status : Correct

Marks : 1/1

6. If two points are exactly the same in the input set for the Closest Pair problem, what will the brute force algorithm correctly return?

Answer

A distance of 0

Status : Correct

Marks : 1/1

7. In the brute force algorithm to check if a set of points is in a convex

position, the sign of the cross product indicates:

Answer

The direction of the turn (Left or Right)

Status : Correct

Marks : 1/1

8. Given 5 points in C where all x-coordinates are identical, what is the result of the brute force closest pair algorithm?

Answer

The distance between the two points with the closest y-coordinates

Status : Correct

Marks : 1/1

9. What is the worst-case time complexity of the Brute Force algorithm for checking if given points form a convex polygon?

Answer

$O(n)$

Status : Wrong

Marks : 0/1

10. In a C program solving the Closest Pair problem via brute force, which nested loop structure correctly avoids redundant calculations (i.e., doesn't calculate distance between A,B and B,A)?

Answer

`for(i=0; i`

Status : Wrong

Marks : 0/1

11. When implementing the Brute Force Closest Pair algorithm in C, what is the minimal number of distance calculations required if there are exactly 3 points?

Answer

3

Status : Correct

Marks : 1/1

12. In the brute force method for finding the convex hull (checking all edges), an edge (i, j) is part of the convex hull if:

Answer

All other points lie on one side of the line through i and j

Status : Correct

Marks : 1/1

13. Which of the following best describes "collinearity" in the context of the Convex Hull problem when using brute force?

Answer

Points lying exactly on the hull edge

Status : Correct

Marks : 1/1

14. What is a significant disadvantage of the Brute Force Convex Hull algorithm compared to Divide and Conquer algorithms?

Answer

It is $O(n^2)$ in the worst case and $O(n^2)$ in the best case (for Jarvis March)

Status : Correct

Marks : 1/1

15. Which of the following correctly represents the Euclidean distance between two points (x1,y1) and (x2,y2) in C?

Answer

``sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2))``

Status : Wrong

Marks : 0/1

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 3_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

You are given a set of N points on a 2D plane. Your task is to find the distance between the closest pair of points.

Write a program that takes as input the number of points N, followed by the x and y coordinates of each point. The program should output the distance between the closest pair of points, rounded to two decimal places.

Input Format

The first line of input consists of an integer N, representing the number of points.

The following N lines represent the coordinate points x and y, separated by space.

Output Format

The output prints a single floating-point number representing the distance between the closest pair of points, rounded to two decimal places.

Sample Test Case

Input: 3

2 3

12 30

40 50

Output: 28.79

Answer

```
#include <stdio.h>
#include <math.h>
#include <float.h>

/**
 * Structure to represent a point in 2D space
 */
typedef struct {
    double x, y;
} Point;

/**
 * Function to calculate Euclidean distance between two points
 */
double calculateDistance(Point p1, Point p2) {
    return sqrt(pow(p1.x - p2.x, 2) + pow(p1.y - p2.y, 2));
}

int main() {
    int N;
    // Reading the number of points
    if (scanf("%d", &N) != 1) return 0;

    Point points[N];

    // Reading the coordinates for each point
    for (int i = 0; i < N; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }

    // Initialize minimum distance with a large value
```

```

double minDistance = DBL_MAX;
// Brute-force approach to find the minimum distance
for (int i = 0; i < N; i++) {
    for (int j = i + 1; j < N; j++) {
        double currentDistance = calculateDistance(points[i], points[j]);
        if (currentDistance < minDistance) {
            minDistance = currentDistance;
        }
    }
}

// Output formatted to two decimal places
printf("%.2f\n", minDistance);

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Ishita is working on a mapping application that helps locate the nearest landmarks on a 2D map. Each landmark is represented by its (x, y) coordinate. Her task is to find out the shortest distance between any two landmarks from the given list.

Help Ishita write a program that calculates the distance between the closest pair of points (landmarks) on the map.

Input Format

The first line of input contains an integer N representing the number of landmarks.

The next N lines contain two space-separated integers, representing the x and y coordinates of each landmark.

Output Format

The output prints a single floating-point number representing the distance between any two landmarks rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

5 7

12 32

21 23

43 65

Output: 12.73

Answer

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#include <float.h>
```

```
int main() {
```

```
    int N;
```

```
    if (scanf("%d", &N) != 1) return 0;
```

```
    int x[N], y[N];
```

```
    for (int i = 0; i < N; i++) {
```

```
        scanf("%d %d", &x[i], &y[i]);
```

```
    }
```

```
    double minDistance = DBL_MAX;
```

```
    // Check all pairs
```

```
    for (int i = 0; i < N; i++) {
```

```
        for (int j = i + 1; j < N; j++) {
```

```
            // Euclidean distance formula:  $\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$ 
```

```
            double dist = sqrt(pow(x[i] - x[j], 2) + pow(y[i] - y[j], 2));
```

```
            if (dist < minDistance) {
```

```
                minDistance = dist;
```

```
            }
```

```
        }
```

```
    }
```

```
    printf("%.2f\n", minDistance);
```

```
} return 0;
```

Status : Correct

Marks : 10/10

3. Problem Statement

Ragu, an aspiring computer scientist, is currently learning about the concept of finding the closest pair of points in a plane. He wants to test his skills by solving a programming challenge related to this concept.

Given a set of points in a 2D plane, Ragu needs to find the closest pair of points and calculate their Euclidean distance.

Input Format

The first line of input consists of an integer N , representing the number of points in the plane.

The following N lines, each consisting of two space-separated real numbers x and y , represent the coordinates of a point.

Output Format

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x_1, y_1) and (x_2, y_2) "

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 3

1 1

10 18

4 9

Output: Closest pair of points: (0, 3) and (1, 1)

Distance: 2.24

Answer

// You are using GCC

#include <stdio.h>

#include <math.h>

#include <float.h>

// Structure to represent a point in 2D plane

struct Point {

double x, y;

};

// Function to calculate Euclidean distance between two points

double calculateDistance(struct Point p1, struct Point p2) {

return sqrt(pow(p1.x - p2.x, 2) + pow(p1.y - p2.y, 2));

}

int main() {

int n;

// Read the number of points

if (scanf("%d", &n) != 1 || n < 2) {

return 0;

}

struct Point points[n];

// Read the coordinates of N points

for (int i = 0; i < n; i++) {

scanf("%lf %lf", &points[i].x, &points[i].y);

}

double minDistance = DBL_MAX;

struct Point closest1 = {0, 0};

struct Point closest2 = {0, 0};

// Compare every pair of points

for (int i = 0; i < n; i++) {

for (int j = i + 1; j < n; j++) {

double dist = calculateDistance(points[i], points[j]);

```

// Update minDistance if a shorter distance is found
if (dist < minDistance) {
    minDistance = dist;
    closest1 = points[i];
    closest2 = points[j];
}
}
}

// Output the result rounded to two decimal places
printf("Closest pair of points: (%.0f, %.0f) and (%.0f, %.0f)\n",
    closest1.x, closest1.y, closest2.x, closest2.y);
printf("Distance: %.2f\n", minDistance);

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Ragu, a passionate computer science student, is exploring geometry problems to sharpen his algorithmic thinking. Today, he challenges himself to determine the two closest points among a set of 2D coordinates.

Ragu must find the closest pair of points from the given set and compute the Euclidean distance between them with high precision. He also wants the output to display which two points are the closest, formatted clearly.

Input Format

The first line of input contains an integer N representing the number of points in the 2D plane.

The next N lines each contain two space-separated real numbers x and y representing the coordinates of each point.

Output Format

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x1, y1) and (x2, y2)"

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

1 2

3 4

1 0

6 8

2 3

Output: Closest pair of points: (1, 2) and (2, 3)

Distance: 1.41

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#include <float.h>
```

```
// Structure to represent a point in 2D plane
```

```
typedef struct {
```

```
    double x;
```

```
    double y;
```

```
} Point;
```

```
int main() {
```

```
    int N;
```

```
    if (scanf("%d", &N) != 1) return 0;
```

```
    Point points[N];
```

```
    for (int i = 0; i < N; i++) {
```

```
        scanf("%lf %lf", &points[i].x, &points[i].y);
```

```
    }
```

```
    double min_dist = DBL_MAX;
```

Point p1, p2;

// Compare every pair of points to find the minimum distance

for (int i = 0; i < N; i++) {

for (int j = i + 1; j < N; j++) {

double dx = points[i].x - points[j].x;

double dy = points[i].y - points[j].y;

double distance = sqrt(dx * dx + dy * dy);

if (distance < min_dist) {

min_dist = distance;

p1 = points[i];

p2 = points[j];

}

}

}

// Format double to check if it's an integer for cleaner output if needed,

// but sample shows standard integer representation for decimals ending in .0

printf("Closest pair of points: (%.0f, %.0f) and (%.0f, %.0f)\n", p1.x, p1.y, p2.x, p2.y);

printf("Distance: %.2f\n", min_dist);

return 0;

}

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 3_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 31.5

Section 1 : Coding

1. Problem Statement

Adhi is learning about computational geometry and has come across the concept of a Convex Hull.

A Convex Hull is a convex polygon that encompasses a set of points in a two-dimensional plane, such that no point lies within the polygon's interior.

Adhi is particularly interested in determining whether a given point lies inside or outside the convex hull formed by a set of points.

Help Adhi by writing a program that checks whether a given point lies inside or outside the convex hull formed by a set of points.

Example

$N = 7$,

Points: $\{(1, 1), (2, 1), (3, 1), (4, 1), (4, 2), (4, 3), (4, 4)\}$,

Query: $X = 3, Y = 2$

Below is the image of the plotting of the given points:

The query $(3, 2)$ lies inside the Convex Hull.

Input Format

The first line of input consists of an integer N , representing the number of points in the set.

The following N lines, each containing two integers x and y , represent the coordinates of the points in the set.

The last line consists of two integers p_x and p_y , representing the coordinates of the test point.

Output Format

The output prints whether the given test points lie inside or outside the Convex Hull.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7

1 1

2 1

3 1

4 1

4 2

4 3

4 4

3 2

Output: The point (3, 2) lies inside the Convex Hull.

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

struct Point {
    int x, y;
};

// Returns the orientation of ordered triplet (p, q, r).
// 0 -> p, q and r are collinear
// 1 -> Clockwise
// 2 -> Counterclockwise
int orientation(struct Point p, struct Point q, struct Point r) {
    int val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
    if (val == 0) return 0; // collinear
    return (val > 0) ? 1 : 2; // clock or counterclock
}

// Function to compute convex hull using Jarvis March algorithm
int getConvexHull(struct Point points[], int n, struct Point hull[]) {
    if (n < 3) return 0;
    int hullSize = 0;

    // Find the leftmost point
    int l = 0;
    for (int i = 1; i < n; i++)
        if (points[i].x < points[l].x)
            l = i;

    int p = l, q;
    do {
        hull[hullSize++] = points[p];
        q = (p + 1) % n;
        for (int i = 0; i < n; i++) {
            if (orientation(points[p], points[i], points[q]) == 2)
                q = i;
        }
        p = q;
    } while (p != l);
    hull[hullSize] = points[l];
    return hullSize;
```

```

    } while (p != l);
    return hullSize;
}

```

```

// Checks if a point p is inside the convex hull
bool isInside(struct Point hull[], int n, struct Point p) {
    if (n < 3) return false;

```

```

    bool positive = false;
    bool negative = false;

```

```

    for (int i = 0; i < n; i++) {
        struct Point a = hull[i];
        struct Point b = hull[(i + 1) % n];

```

```

        // Use cross product to find which side of the line edge the point is on
        int cross_product = (b.x - a.x) * (p.y - a.y) - (b.y - a.y) * (p.x - a.x);

```

```

        if (cross_product > 0) positive = true;
        else if (cross_product < 0) negative = true;

```

```

        // If it's on both sides, it's outside
        if (positive && negative) return false;
    }

```

```

    return true; // Point is inside or on the boundary
}

```

```

int main() {
    int N;
    if (scanf("%d", &N) != 1) return 0;

```

```

    struct Point points[100];
    for (int i = 0; i < N; i++) {
        scanf("%d %d", &points[i].x, &points[i].y);
    }

```

```

    struct Point query;
    scanf("%d %d", &query.x, &query.y);

```

```

    struct Point hull[100];
    int hullSize = getConvexHull(points, N, hull);

```

```

if (isInside(hull, hullSize, query)) {
    printf("The point (%d, %d) lies inside the Convex Hull.\n", query.x, query.y);
} else {
    printf("The point (%d, %d) lies outside the Convex Hull.\n", query.x, query.y);
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement:

Olivia, a city planner, is designing a recreational area surrounded by specific landmarks. To ensure balance in the design, Olivia needs to calculate the weighted centroid of the boundary formed by these landmarks using the convex hull.

Your task is to compute the convex hull and determine the weighted centroid of its vertices.

Note:

MAXN (100000): The maximum capacity for both the points and hull arrays, allowing storage of up to 100,000 points each. $1e-9$ is the threshold for floating-point comparisons.

Example:

Input:

6

0 0

4 0

4 3

0 3

2 1

3 2

Output:

Weighted Centroid: 2.00 1.50

Explanation:

Input Points: (0, 0), (2, 0), (2, 2), (0, 2), (1, 1), (3, 1).

Pivot Selection: The point with the lowest y-coordinate (and leftmost in case of a tie) is selected as the pivot: (0, 0).

Sorting by Polar Angle: Points are sorted by the polar angle relative to the pivot: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Convex Hull Construction: Using counterclockwise turns, the points forming the convex hull are: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Input Format

The first line contains an integer N, representing the number of points.

The next N lines each contain two floating-point numbers x and y, representing the coordinates of the points.

Output Format

The output prints the coordinates of the weighted centroid of the convex hull, formatted to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

0 0

2 0

2 2

0 2

1 1

Output: Weighted Centroid: 1.00 1.00

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define MAXN 100005

typedef struct {
    double x, y;
} Point;

Point pivot;
Point points[MAXN];
Point hull[MAXN];

// Calculate squared distance
double distSq(Point p1, Point p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}

// Orientation: 0=collinear, 1=clockwise, 2=counterclockwise
int orientation(Point p, Point q, Point r) {
    double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
    if (fabs(val) < 1e-9) return 0;
    return (val > 0) ? 1 : 2;
}

// Comparator for sorting by polar angle
int compare(const void *vp1, const void *vp2) {
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;
    int o = orientation(pivot, *p1, *p2);
    if (o == 0) return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}

int main() {
    int N;
    if (scanf("%d", &N) != 1) return 0;

    int minIdx = 0;
```

```

for (int i = 0; i < N; i++) {
    scanf("%lf %lf", &points[i].x, &points[i].y);
    if (i > 0 && (points[i].y < points[minIdx].y || (points[i].y == points[minIdx].y &&
points[i].x < points[minIdx].x))) {
        minIdx = i;
    }
}

```

// Move pivot to first position

```

Point temp = points[0];
points[0] = points[minIdx];
points[minIdx] = temp;
pivot = points[0];

```

// Sort points based on polar angle

```

qsort(&points[1], N - 1, sizeof(Point), compare);

```

// Remove collinear points (keep only farthest)

```

int m = 1;
for (int i = 1; i < N; i++) {
    while (i < N - 1 && orientation(pivot, points[i], points[i + 1]) == 0)
        i++;
    points[m++] = points[i];
}

```

if (m < 3) return 0; // Not a polygon

// Graham Scan to find hull

```

int stackSize = 0;
hull[stackSize++] = points[0];
hull[stackSize++] = points[1];
hull[stackSize++] = points[2];

```

```

for (int i = 3; i < m; i++) {
    while (stackSize > 1 && orientation(hull[stackSize - 2], hull[stackSize - 1],
points[i]) != 2)
        stackSize--;
    hull[stackSize++] = points[i];
}

```

// Calculate centroid of the hull vertices

```

double centroidX = 0, centroidY = 0;

```



```

    for (int i = 0; i < stackSize; i++) {
        centroidX += hull[i].x;
        centroidY += hull[i].y;
    }

    printf("Weighted Centroid: %.2f %.2f\n", centroidX / stackSize, centroidY /
stackSize);

    return 0;
}

```

Status : Partially correct

Marks : 6/10

3. Problem Statement:

Gwen, a city planner, is designing a new park and needs to calculate the perimeter of the park's boundary. She has the coordinates of key boundary points and wants to find the minimum perimeter that encloses all these points.

Using the convex hull, Gwen needs help in computing both the boundary and its perimeter.

Note:

points[100000]: An array that can store up to 100,000 input

points.hull[100000]: An array that can store up to 100,000 points on the convex hull.
1e-9: A threshold for floating-point comparisons to handle precision issues.

Example

Input:

6

0.0 0.0

3.0 0.0

3.0 2.0

1.0 2.0

0.0 2.0

2.0 1.0

Output:

Convex Hull:

0 0

3 0

3 2

0 2

Perimeter: 10.00

Explanation:

Input Points: (0.0, 0.0), (3.0, 0.0), (3.0, 2.0), (1.0, 2.0), (0.0, 2.0), (2.0, 1.0).

Convex Hull Construction:

Pivot: Start with the lowest point, (0, 0). Sorted by Polar Angle: The points are sorted, and counterclockwise turns are included in the hull. Convex Hull Points: (0, 0), (3, 0), (3, 2), (0, 2). Perimeter Calculation:

Distances between consecutive hull points: (0, 0)-(3, 0): 3.00, (3, 0)-(3, 2): 2.00, (3, 2)-(0, 2): 3.00, (0, 2)-(0, 0): 2.00. Total Perimeter: 10.00.

Input Format

The first line contains an integer N, representing the number of points.

The next N lines each contain two space separated decimal values x and y, representing the coordinates of the points.

Output Format

The first line prints "Convex Hull:" followed by the coordinates of the points on the convex hull, each on a new line, in counterclockwise order.

The last line prints the "Perimeter: ", followed by the total perimeter of the convex hull, formatted to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

0.0 0.0

3.0 0.0

3.0 2.0

1.0 2.0

0.0 2.0

2.0 1.0

Output: Convex Hull:

0 0

3 0

3 2

0 2

Perimeter: 10.00

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
typedef struct {  
    double x, y;  
} Point;
```

```
Point pivot;
```

```
// Distance squared between two points
```

```
double distSq(Point p1, Point p2) {
```

```
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);  
}
```

```
// To find orientation of ordered triplet (p, q, r).
```

```
// 0 -> p, q and r are collinear
```

```
// 1 -> Clockwise
```

```
// 2 -> Counterclockwise
```

```
int orientation(Point p, Point q, Point r) {
```

```
    double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
```

```
    if (fabs(val) < 1e-9) return 0;
```

```
    return (val > 0) ? 1 : 2;
```

```
}
```

```
// Compare function for qsort based on polar angle
int compare(const void *vp1, const void *vp2) {
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;

    int o = orientation(pivot, *p1, *p2);
    if (o == 0)
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}
```

```
void swap(Point *p1, Point *p2) {
    Point temp = *p1;
    *p1 = *p2;
    *p2 = temp;
}
```

```
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    Point points[100000];

    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }

    // 1. Find the bottom-most point
    int ymin = 0;
    for (int i = 1; i < n; i++) {
        if (points[i].y < points[ymin].y || (points[i].y == points[ymin].y && points[i].x <
points[ymin].x))
            ymin = i;
    }

    swap(&points[0], &points[ymin]);
    pivot = points[0];

    // 2. Sort points based on polar angle with pivot
    qsort(&points[1], n - 1, sizeof(Point), compare);
}
```

```

// 3. Build Hull
Point hull[100000];
int m = 0;
hull[m++] = points[0];
hull[m++] = points[1];
hull[m++] = points[2];

for (int i = 3; i < n; i++) {
    while (m > 1 && orientation(hull[m-2], hull[m-1], points[i]) != 2)
        m--;
    hull[m++] = points[i];
}

// Output Hull
printf("Convex Hull:\n");
for (int i = 0; i < m; i++) {
    printf("%.0f %.0f\n", hull[i].x, hull[i].y);
}

// 4. Calculate Perimeter
double perimeter = 0;
for (int i = 0; i < m; i++) {
    perimeter += sqrt(distSq(hull[i], hull[(i + 1) % m]));
}
printf("Perimeter: %.2f\n", perimeter);

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement:

Maria, a wildlife researcher, is studying the distribution of endangered species in a forest. She has mapped several points that represent observation locations inside the forest.

Maria wants to identify the K points that are the farthest from the forest's outer boundary, represented by a convex hull formed by a set of points. These points represent key observations, and she needs to rank them by

their distance to the outer boundary of the forest.

Help Maria compute the convex hull and find the K farthest points from the forest boundary.

Note:

MAXN (100000): Maximum number of points for points and hull arrays.
1e-9: Threshold for floating-point equality checks to handle precision errors.
1e-12: Threshold to detect degenerate (zero-length) segments.
1e30: Initial large value for minimum distance calculations.

Example:

Input:

7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output:

2 2

3 2

1 2

Explanation:

Convex Hull Points: Using Graham's Scan, the convex hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Distance to Hull: Distances of all points to the convex hull boundary are computed.

Top 3 Farthest Points: Sorted by descending distance, the top 3 points are (2, 2), (3, 2), (1, 2).

Input Format

The first line contains two integers N and K, representing the number of points and the number of farthest points to find, respectively.

The next N lines each contain two integers x and y, representing the coordinates of the points.

Output Format

The output prints K lines, each containing two integers x and y, representing the coordinates of the K farthest points from the convex hull boundary. Print the points in descending order of their distances to the boundary.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output: 2 2

3 2

1 2

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
typedef struct {
```

```
    double x, y;
```

```
} Point;
```

```
typedef struct {
    Point p;
    double dist;
    int id;
} Result;
```

```
// Function to calculate squared distance between two points
double distSq(Point p1, Point p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}
```

```
// Function to find orientation (0: Collinear, 1: Clockwise, 2: Counterclockwise)
int orientation(Point p, Point q, Point r) {
    double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
    if (fabs(val) < 1e-9) return 0;
    return (val > 0) ? 1 : 2;
}
```

```
// Compare function for sorting points based on polar angle for Graham Scan
Point pivot;
int compare(const void *vp1, const void *vp2) {
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;
    int o = orientation(pivot, *p1, *p2);
    if (o == 0)
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}
```

```
// Calculate the minimum distance from a point to a line segment
double distToSegment(Point p, Point a, Point b) {
    double l2 = distSq(a, b);
    if (l2 < 1e-12) return sqrt(distSq(p, a));
    double t = ((p.x - a.x) * (b.x - a.x) + (p.y - a.y) * (b.y - a.y)) / l2;
    if (t < 0) return sqrt(distSq(p, a));
    if (t > 1) return sqrt(distSq(p, b));
    Point projection = {a.x + t * (b.x - a.x), a.y + t * (b.y - a.y)};
    return sqrt(distSq(p, projection));
}
```

```
// Compare function to sort results by distance (descending)
```



```

int compareResults(const void*a, const void *b) {
    Result *r1 = (Result *)a;
    Result *r2 = (Result *)b;
    if (fabs(r2->dist - r1->dist) > 1e-9)
        return (r2->dist > r1->dist) ? 1 : -1;
    return r1->id - r2->id; // Maintain stability with original index
}

int main() {
    int n, k;
    if (scanf("%d %d", &n, &k) != 2) return 0;

    Point pts[n], temp[n];
    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &pts[i].x, &pts[i].y);
        temp[i] = pts[i];
    }

    // Graham Scan - Find Pivot
    int min = 0;
    for (int i = 1; i < n; i++) {
        if (temp[i].y < temp[min].y || (temp[i].y == temp[min].y && temp[i].x <
temp[min].x))
            min = i;
    }
    Point t = temp[0]; temp[0] = temp[min]; temp[min] = t;
    pivot = temp[0];
    qsort(&temp[1], n - 1, sizeof(Point), compare);

    // Build Hull
    Point hull[n];
    int hullSize = 0;
    for (int i = 0; i < n; i++) {
        while (hullSize >= 2 && orientation(hull[hullSize - 2], hull[hullSize - 1], temp[i]) !
= 2)
            hullSize--;
        hull[hullSize++] = temp[i];
    }

    // Calculate distances for all original points
    Result res[n];
    for (int i = 0; i < n; i++) {

```

```

double minDist = 1e30;
for (int j = 0; j < hullSize; j++) {
    double d = distToSegment(pts[i], hull[j], hull[(j + 1) % hullSize]);
    if (d < minDist) minDist = d;
}
res[i].p = pts[i];
res[i].dist = minDist;
res[i].id = i;
}

qsort(res, n, sizeof(Result), compareResults);

for (int i = 0; i < k; i++) {
    printf("%g %g\n", res[i].p.x, res[i].p.y);
}

return 0;
}

```

Status : Partially correct

Marks : 4/10

5. Problem Statement:

Henry, a wildlife researcher, is studying the distribution of endangered species in a forest. He has mapped several points that represent observation locations inside the forest.

Henry wants to identify the K points that are the farthest from the forest's outer boundary, represented by a convex hull formed by a set of points. These points represent key observations, and he needs to rank them by their distance to the outer boundary of the forest.

Help Henry compute the convex hull and find the K farthest points from the forest boundary.

Note:

MAXN (100000): Maximum number of points for points and hull arrays.
 1e-9: Threshold for floating-point equality checks to handle precision errors.
 1e-12: Threshold to detect degenerate (zero-length)

segments. $1e30$: Initial large value for minimum distance calculations.

Example:

Input:

7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output:

2 2

3 2

1 2

Explanation:

Convex Hull Points: Using Graham's Scan, the convex hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Distance to Hull: Distances of all points to the convex hull boundary are computed.

Top 3 Farthest Points: Sorted by descending distance, the top 3 points are (2, 2), (3, 2), (1, 2).

Input Format

The first line contains two integers N and K, representing the number of points and the number of farthest points to find, respectively.

The next N lines each contain two integers x and y, representing the coordinates of the points.

Output Format

The output prints K lines, each containing two integers x and y, representing the coordinates of the K farthest points from the convex hull boundary. Print the points in descending order of their distances to the boundary.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7 3

0 0

4 0

4 4

0 4

2 2

1 2

3 2

Output: 2 2

3 2

1 2

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
#define MAXN 100005
```

```
#define EPS 1e-9
```

```
typedef struct {
```

```
    double x, y;
```

```
    int id;
```

```
} Point;
```

```
Point p[MAXN];
```

```
Point hull[MAXN];
```

```
int hull_size = 0;
```

```
// Function to calculate squared distance between two points
double distSq(Point p1, Point p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}
```

```
// Function to find orientation of ordered triplet (p, q, r)
// 0 -> p, q and r are collinear
// 1 -> Clockwise
// 2 -> Counterclockwise
int orientation(Point p, Point q, Point r) {
    double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
    if (fabs(val) < EPS) return 0;
    return (val > 0) ? 1 : 2;
}
```

```
// Compare function for qsort to sort points based on polar angle
Point p0;
int compare(const void *vp1, const void *vp2) {
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;
    int o = orientation(p0, *p1, *p2);
    if (o == 0) return (distSq(p0, *p2) < distSq(p0, *p1)) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}
```

```
// Graham's scan algorithm to find convex hull
void convexHull(int n) {
    int ymin = 0;
    for (int i = 1; i < n; i++) {
        if (p[i].y < p[ymin].y || (fabs(p[i].y - p[ymin].y) < EPS && p[i].x < p[ymin].x))
            ymin = i;
    }
    Point temp = p[0];
    p[0] = p[ymin];
    p[ymin] = temp;
    p0 = p[0];
    qsort(&p[1], n - 1, sizeof(Point), compare);
}
```

```
int m = 1;
for (int i = 1; i < n; i++) {
    while (i < n - 1 && orientation(p0, p[i], p[i + 1]) == 0) i++;
    p[m] = p[i];
}
```

```

    m++;
}

if (m < 3) return;

hull[0] = p[0];
hull[1] = p[1];
hull[2] = p[2];
hull_size = 3;

for (int i = 3; i < m; i++) {
    while (hull_size >= 2 && orientation(hull[hull_size - 2], hull[hull_size - 1], p[i]) !
= 2)
        hull_size--;
    hull[hull_size] = p[i];
    hull_size++;
}
}

```

```

// Distance from point p to segment ab
double distToSegment(Point p, Point a, Point b) {
    double l2 = distSq(a, b);
    if (l2 < 1e-12) return sqrt(distSq(p, a));
    double t = ((p.x - a.x) * (b.x - a.x) + (p.y - a.y) * (b.y - a.y)) / l2;
    if (t < 0.0) return sqrt(distSq(p, a));
    if (t > 1.0) return sqrt(distSq(p, b));
    Point projection = {a.x + t * (b.x - a.x), a.y + t * (b.y - a.y)};
    return sqrt(distSq(p, projection));
}

```

```

// Distance from point to convex hull boundary
double distToHull(Point pt) {
    double minD = 1e30;
    for (int i = 0; i < hull_size; i++) {
        double d = distToSegment(pt, hull[i], hull[(i + 1) % hull_size]);
        if (d < minD) minD = d;
    }
    return minD;
}

```

```

typedef struct {
    double dist;

```

```

    Point p;
} DistPoint;

int compareDist(const void *a, const void *b) {
    DistPoint *da = (DistPoint *)a;
    DistPoint *db = (DistPoint *)b;
    if (db->dist > da->dist) return 1;
    if (db->dist < da->dist) return -1;
    return 0;
}

int main() {
    int N, K;
    if (scanf("%d %d", &N, &K) != 2) return 0;
    for (int i = 0; i < N; i++) {
        scanf("%lf %lf", &p[i].x, &p[i].y);
        p[i].id = i;
    }

    convexHull(N);

    DistPoint *dps = (DistPoint *)malloc(N * sizeof(DistPoint));
    for (int i = 0; i < N; i++) {
        dps[i].p = p[i];
        dps[i].dist = distToHull(p[i]);
    }

    qsort(dps, N, sizeof(DistPoint), compareDist);

    for (int i = 0; i < K; i++) {
        printf("%g %g\n", dps[i].p.x, dps[i].p.y);
    }

    free(dps);
    return 0;
}

```

Status : Partially correct

Marks : 1.5/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 4_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. Which of the following methods can be used to solve the Knapsack problem?

Answer

All the mentioned options

Status : Correct

Marks : 1/1

2. 0/1 knapsack is based on _____method.

Answer

Dynamic programming

Status : Correct

Marks : 1/1

3. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

Answer

It systematically explores possible solutions while pruning non-promising ones.

Status : Correct

Marks : 1/1

4. Which data structure is commonly used to implement the Branch and Bound approach for solving the Knapsack problem?

Answer

Priority Queue

Status : Correct

Marks : 1/1

5. In the Branch and Bound approach, what does the term “bounding function” refer to?

Answer

A function that estimates the upper bound on the maximum value that can be achieved from a node.

Status : Correct

Marks : 1/1

6. Which strategy is commonly used in the Branch and Bound method to decide which node to explore next?

Answer

Best-first search

Status : Correct

Marks : 1/1

7. Which of the following methods is used to prune the search space in the Branch and Bound method for the Knapsack problem?

Answer

Ignoring nodes with a weight exceeding the capacity.

Status : Correct

Marks : 1/1

8. The travelling salesman problem can be solved using _____.

Answer

a minimum spanning tree

Status : Correct

Marks : 1/1

9. Travelling salesman problem is an example of _____.

Answer

Greedy Algorithm

Status : Wrong

Marks : 0/1

10. How does the practical Travelling salesman problem differ from the classical Travelling salesman problem?

Answer

In the practical travelling salesman problem, each vertex can be visited more than once.

Status : Correct

Marks : 1/1

11. The traveling salesman problem involves n cities with paths connecting the cities. The time taken for traversing through all the cities without knowing in advance the length of a minimum tour is

Answer

$O(n!)$

Status : Correct

Marks : 1/1

12. In which search problem, to find the shortest path, each city must be

visited only once?

Answer

Travelling Salesman problem

Status : Correct

Marks : 1/1

13. What is the main goal of using Branch and Bound in TSP?

Answer

To minimize search space by pruning unpromising paths

Status : Correct

Marks : 1/1

14. Which value helps in deciding whether a node should be pruned in Branch and Bound?

Answer

Bound (lower bound of cost)

Status : Correct

Marks : 1/1

15. Given items with weights [10, 20, 30] and values [60, 100, 120], what is the value-to-weight ratio for the second item?

Answer

5

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 4_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 48.5

Section 1 : Coding

1. Problem Statement

Janu is tasked with creating a program that solves the Traveling Salesman Problem (TSP). The goal is to find the shortest possible route that visits a set of cities and returns to the starting city.

You are required to implement a program that takes input regarding the number of cities and the distance matrix between cities and outputs the optimal route and its associated minimum cost.

The diagonal values will be 0, as the distance from a city to itself is 0.

Note: Solve it using Branch and Bound approach.

Input Format

The first line of input consists of an integer n , representing the number of cities.

The next n lines contain n space-separated integers each, representing the distance matrix between the cities.

The cell (i, j) represents the distance from city i to city j .

Output Format

The first line of output displays "The Path is: " followed by the path of the route using ---> in between.

The second line displays "Minimum cost is " followed by the total minimum cost associated with the shortest route.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 4 1 3

4 0 2 1

1 2 0 5

3 1 5 0

Output: The Path is: 1 ---> 3 ---> 2 ---> 4 ---> 1

Minimum cost is 7

Answer

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define MAX 10
```

```
#define INF 999999
```

```
int n;
```

```
int adj[MAX][MAX];
```

```
int best_path[MAX + 1];
```

```
int current_path[MAX + 1];
```

```
int min_cost = INF;
```

```
bool visited[MAX];
```

```

void solve(int current_city, int count, int current_cost) {
    // Branch and Bound Pruning: Stop if current cost already exceeds best found
    if (current_cost >= min_cost) {
        return;
    }

```

```

    // Base case: All cities visited, try returning to start (City 1)
    if (count == n) {
        int total_cost = current_cost + adj[current_city][0];
        if (total_cost < min_cost) {
            min_cost = total_cost;
            for (int i = 0; i < n; i++) {
                best_path[i] = current_path[i];
            }
            best_path[n] = 0; // Return to starting city
        }
        return;
    }

```

```

    // Branching: Explore unvisited cities
    for (int next_city = 0; next_city < n; next_city++) {
        if (!visited[next_city] && adj[current_city][next_city] != 0) {
            visited[next_city] = true;
            current_path[count] = next_city;
            solve(next_city, count + 1, current_cost + adj[current_city][next_city]);
            visited[next_city] = false; // Backtrack
        }
    }
}

```

```

int main() {
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
        visited[i] = false;
    }

    // Start tour from the first city (index 0)
    visited[0] = true;

```

```

current_path[0] = 0;
solve(0, 1, 0);

// Output formatting according to specifications
printf("The Path is: ");
for (int i = 0; i <= n; i++) {
    printf("%d", best_path[i] + 1);
    if (i < n) {
        printf("--->");
    }
}
printf("\nMinimum cost is %d\n", min_cost);

return 0;
}

```

Status : Partially correct

Marks : 8.5/10

2. Problem Statement

Karan is planning a delivery route that starts from city 0, visits every other city exactly once, and returns back to city 0.

The travel costs between cities are given as a cost matrix.

Your task is to find the minimum possible travel cost and print the optimal route.

Cost Formula

For a route:

0 a b ... 0

Total Cost = cost[0][a] + cost[a][b] + ... + cost[last][0]

Input Format

The first line contains an integer n, representing the number of cities.

The next n lines contain n space-separated integers representing the cost matrix.

Output Format

The output prints multiple lines showing path exploration in the format:

The first line prints the minimum travel cost.

The second line prints the optimal path starting from city 0 and ending at city 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: 80

0 1 3 2 0

Answer

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define INF 999999
```

```
int n;
```

```
int cost[10][10];
```

```
int min_cost = INF;
```

```
int best_path[10];
```

```
int current_path[10];
```

```
bool visited[10];
```

```
// Function to find the minimum cost and best path
```



```

void solveTSP(int current_city, int count, int current_sum, int num_cities) {
    // Base case: All cities visited
    if (count == num_cities) {
        // Return back to city 0
        if (cost[current_city][0] > 0) {
            int total_cost = current_sum + cost[current_city][0];
            if (total_cost < min_cost) {
                min_cost = total_cost;
                // Store the best path
                for (int i = 0; i < num_cities; i++) {
                    best_path[i] = current_path[i];
                }
            }
        }
    }
    return;
}

```

```

// Explore other cities
for (int next_city = 0; next_city < num_cities; next_city++) {
    if (!visited[next_city] && cost[current_city][next_city] > 0) {
        visited[next_city] = true;
        current_path[count] = next_city;

        solveTSP(next_city, count + 1, current_sum + cost[current_city][next_city],
num_cities);
    }
}

```

```

// Backtrack
visited[next_city] = false;
}
}
}

```

```

int main() {
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
        visited[i] = false;
    }
}

```

```

// Start from city 0
visited[0] = true;
current_path[0] = 0;

// Start recursion from city 0, count 1, cost 0
solveTSP(0, 1, 0, n);

// Print minimum cost
printf("%d\n", min_cost);

// Print optimal path
for (int i = 0; i < n; i++) {
    printf("%d ", best_path[i]);
}
printf("0\n");

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are a logistics manager responsible for planning delivery routes for a fleet of vehicles. Each vehicle needs to visit a set of locations to deliver goods, and the cost of traveling between locations is known.

Your goal is to find the shortest route for each vehicle, ensuring that each location is visited exactly once before returning to the starting point.

Note: Solve it using Branch and Bound approach.

Input Format

The first line contains an integer n , the number of locations.

The next n lines contain n integers each, representing the cost matrix of traveling between locations.

The i th integer on the j th line represents the cost of traveling from location i to

location j.

Output Format

The first line of output displays "Shortest Path: " followed by the shortest path that visits all cities.

The second line of output displays "Minimum cost is " followed by the minimum cost.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
0 10 8 9 7
10 0 10 5 6
8 10 0 8 9
9 5 8 0 6
7 6 9 6 0

Output: Shortest Path: 1 3 4 2 5 1
Minimum cost is 34

Answer

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 10
#define INF 999999
```

```
int n;
int cost[MAX][MAX];
int minCost = INF;
int finalPath[MAX + 1];
int currentPath[MAX + 1];
bool visited[MAX];
```

// Function to find the minimum edge cost leaving a city

```
int findMinEdge(int city) {
    int min = INF;
    for (int i = 0; i < n; i++) {
```

```

        if (cost[city][i] < min && city != i)
            min = cost[city][i];
    }
    return min;
}

```

```

// Function to find the minimum edge cost entering a city
int findMinIncoming(int city) {
    int min = INF;
    for (int i = 0; i < n; i++) {
        if (cost[i][city] < min && city != i)
            min = cost[i][city];
    }
    return min;
}

```

```

// Branch and Bound: Depth First Search
void tspRecursive(int currPos, int count, int currentCost, int bound) {
    if (count == n && cost[currPos][0] != 0) {
        if (currentCost + cost[currPos][0] < minCost) {
            minCost = currentCost + cost[currPos][0];
            for (int i = 0; i < n; i++)
                finalPath[i] = currentPath[i];
            finalPath[n] = 0; // Return to start
        }
    }
    return;
}

```

```

for (int i = 0; i < n; i++) {
    if (!visited[i] && cost[currPos][i] != 0) {
        int tempBound = bound;
        currentCost += cost[currPos][i];

```

```

        // Reduced Matrix/Bound calculation logic (Simplified for DFS)
        if (count == 1)
            bound -= ((findMinEdge(currPos) + findMinIncoming(i)) / 2);
        else
            bound -= ((findMinEdge(currPos) + findMinIncoming(i)) / 2);

```

```

        if (currentCost + bound < minCost) {
            currentPath[count] = i;
            visited[i] = true;

```

```

        tspRecursive(i, count + 1, currentCost, bound);
    }

    // Backtrack
    currentCost -= cost[currPos][i];
    bound = tempBound;
    visited[i] = false;
}
}
}

```

// Wrapper function to initialize B&B

```

void solveTSP() {
    int bound = 0;
    for (int i = 0; i < n; i++) {
        bound += (findMinEdge(i) + findMinIncoming(i));
    }
    bound = (bound + 1) / 2; // Ceiling division

    visited[0] = true;
    currentPath[0] = 0;
    tspRecursive(0, 1, 0, bound);
}

```

```

int main() {
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    if (n == 0) {
        printf("Minimum cost is 0\n");
        return 0;
    }
}

```

```

solveTSP();

```

```

printf("Shortest Path: ");
for (int i = 0; i <= n; i++) {

```

```
        printf("%d ", finalPath[i] + 1);
    }
    printf("\nMinimum cost is %d\n", minCost);

    return 0;
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Michael has been given an important task—developing a program to solve the classic 0/1 Knapsack Problem. His goal is to help users select the most valuable combination of items while staying within a specified weight limit.

In this problem, each item has a value and a weight, and the objective is to maximize the total value without exceeding the given weight capacity.

Michael's program should determine the maximum possible value that can be obtained by selecting the optimal set of items.

Input Format

The first line of input consists of an integer n , representing the number of items available for selection.

The second line of input consists of n space-separated integers, P , which represent the values of the items.

The third line of input consists of n space-separated integers, C , which represent the weights of the corresponding items.

The fourth line of input consists of an integer W , representing the maximum weight limit of the knapsack.

Output Format

The output displays the maximum value that can be obtained using the knapsack algorithm in the following format: "Maximum value: [maximum value]"

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 6

300 150 120 100 90 80

6 3 3 2 2 2

10

Output: Maximum value: 490

Answer

```
#include <stdio.h>
```

```
// Helper function to find the maximum of two integers
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
// Function to solve the 0/1 Knapsack Problem using Tabulation (Dynamic Programming)
```

```
int solveKnapsack(int n, int values[], int weights[], int W) {
```

```
    int i, w;
```

```
    // DP table: dp[i][w] stores the max value for first 'i' items with capacity 'w'
```

```
    int dp[n + 1][W + 1];
```

```
    // Build the table in a bottom-up manner
```

```
    for (i = 0; i <= n; i++) {
```

```
        for (w = 0; w <= W; w++) {
```

```
            if (i == 0 || w == 0) {
```

```
                dp[i][w] = 0; // Base case: no items or zero capacity
```

```
            } else if (weights[i - 1] <= w) {
```

```
                // Choice: Include the item or exclude it
```

```
                dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]], dp[i - 1][w]);
```

```
            } else {
```

```
                // Weight of item exceeds current capacity, cannot include
```

```
                dp[i][w] = dp[i - 1][w];
```

```
            }
```

```
        }
```

```
    }  
    return dp[n][W];
```

```
}
```

```
int main() {  
    int n, W;
```

```
    // Input: Number of items  
    if (scanf("%d", &n) != 1) return 0;
```

```
    int values[n], weights[n];
```

```
    // Input: Values of the items  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &values[i]);  
    }
```

```
    // Input: Weights of the items  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &weights[i]);  
    }
```

```
    // Input: Maximum weight limit  
    scanf("%d", &W);
```

```
    // Solve and output results in the required format  
    int result = solveKnapsack(n, values, weights, W);  
    printf("Maximum value: %d\n", result);
```

```
    return 0;
```

```
}
```

Status : Correct

Marks : 10/10

5. Problem Statement

Ragul is interested in solving the knapsack problem. He has a list of n items, each with a weight and a value. Ragul wants to determine the maximum value he can obtain by selecting a combination of items to fit into his knapsack, which has a maximum weight capacity.

Help Ragul write a program that calculates the following:

Calculate and print the sum of all n values. Calculate and print the average value of all n values with exactly 2 decimal places. Calculate and print the maximum value that can be obtained by selecting a combination of items to fit into the knapsack without exceeding its capacity.

Note: Use the 0/1 knapsack method to solve the problem.

Input Format

The first line of input contains an integer n, representing the number of items.

The next line contains n space-separated integers, representing the weights of the items.

The next line contains n space-separated integers, representing the values of the items.

The last line contains an integer C, representing the maximum weight capacity of the knapsack.

Output Format

The output consists of the following in each line:

1. The sum of all n values as an integer.
2. The average value of all n values with exactly 2 decimal places is a double value.
3. The maximum value that can be obtained as an integer.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 15 20
45 65 70
30

Output: Sum of values: 180
Average of values: 60.00

Maximum amount: 115

Answer

```
#include <stdio.h>
```

```
// Function to find the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

```
int main() {
    int n, C;
```

```
    // Input number of items
    if (scanf("%d", &n) != 1) return 0;
```

```
    int weights[n], values[n];
    int sum = 0;
```

```
    // Input weights
    for (int i = 0; i < n; i++) {
        scanf("%d", &weights[i]);
    }
```

```
    // Input values and calculate sum
    for (int i = 0; i < n; i++) {
        scanf("%d", &values[i]);
        sum += values[i];
    }
```

```
    // Input maximum capacity
    scanf("%d", &C);
```

```
    // Dynamic Programming table to store max values for different capacities
    int dp[n + 1][C + 1];
```

```
    // Build the table in bottom-up manner
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= C; w++) {
            if (i == 0 || w == 0) {
                dp[i][w] = 0;
            } else if (weights[i - 1] <= w) {
```

```

        // Choice: Include the item or exclude it
        dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]], dp[i - 1][w]);
    } else {
        // Item too heavy, must exclude it
        dp[i][w] = dp[i - 1][w];
    }
}
}

// Calculate average value
double average = (double)sum / n;

// Output according to sample format
printf("Sum of values: %d\n", sum);
printf("Average of values: %.2f\n", average);
printf("Maximum amount: %d\n", dp[n][C]);

return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 4_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Meera is preparing for a science exhibition and needs to select a group of items whose total weight is exactly equal to a required target value. Each item can be selected at most once.

Given a list of item weights and a target weight K , determine whether it is possible to select a subset of items such that their sum is exactly K .

If such a subset exists, print any one valid subset. If no such subset exists, print -1.

This problem is derived from the Subset Sum problem, which is NP-Complete, and the solution must be implemented using Dynamic Programming.

Input Format

The first line contains two integers n and K

The second line contains n space-separated integers

Output Format

Print elements of one valid subset whose sum equals K

If no such subset exists, print -1

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 9
3 34 4 12 5
Output: 5 4

Answer

```
#include <stdio.h>
#include <stdbool.h>
```

```
/**
 * Problem: Subset Sum with Backtracking
 * Constraints: n <= 100, K <= 1000
 * Approach: Dynamic Programming O(n * K)
 */
```

```
void findSubset(int n, int K, int weights[]) {
    // dp[i][j] will be true if a sum 'j' can be achieved using the first 'i' items
    bool dp[n + 1][K + 1];
```

```
    // Initialization
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= K; j++) {
            if (j == 0)
                dp[i][j] = true; // Sum 0 is always possible (empty set)
            else
                dp[i][j] = false; // Other sums impossible with 0 items
        }
    }
}
```

```

// Fill the DP table
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= K; j++) {
        // Option 1: Exclude the current item
        dp[i][j] = dp[i - 1][j];

        // Option 2: Include the current item (if weight <= current sum j)
        if (j >= weights[i - 1]) {
            dp[i][j] = dp[i][j] || dp[i - 1][j - weights[i - 1]];
        }
    }
}

// Check if target K is reachable
if (!dp[n][K]) {
    printf("-1\n");
    return;
}

// Backtrack to find the elements used in the subset
int curr_sum = K;
for (int i = n; i > 0 && curr_sum > 0; i--) {
    // If the sum was achieved without the current item, skip it
    // Otherwise, it means the current item was part of the subset
    if (!dp[i - 1][curr_sum]) {
        printf("%d ", weights[i - 1]);
        curr_sum -= weights[i - 1];
    }
}
printf("\n");

int main() {
    int n, K;

    // Read n and target K
    if (scanf("%d %d", &n, &K) != 2) return 0;

    int weights[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &weights[i]);
    }
}

```

```

    }
    findSubset(n, K, weights);
    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Rohit is a travel planner who must organize a business trip covering multiple cities.

The travel cost between each pair of cities is given in a matrix.

He must:

Start from city 0
Visit every city exactly once
Return back to city 0
Your task is to compute:

The minimum possible travel cost
The sequence of cities visited
The average cost per step (excluding the final return step)

This problem is a classic Travelling Salesman Problem (TSP) solved using Dynamic Programming with Bitmasking (Held–Karp Algorithm).

Input Format

- The first line contains an integer n , the number of cities.
- The next n lines contain n space-separated integers, representing the cost matrix.

$\text{cost}[i][j]$ is the cost of traveling from city i to city j .

Output Format

Print the path:

The Path is: 0--->...--->0

Print the minimum cost:

Minimum cost is X

Print the average cost per step:

Average cost per step is Y

Rounded to 2 decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: The Path is: 0--->2--->3--->1--->0

Minimum cost is 80

Average cost per step is 26.67

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#include <math.h>
```

```
#define MAX_CITIES 10
```

```
#define INF 1000000
```

```
int n;
```



```

int cost[MAX_CITIES][MAX_CITIES];
int dp[1 << MAX_CITIES][MAX_CITIES];
int parent[1 << MAX_CITIES][MAX_CITIES];

// Solve TSP using DP with Bitmasking
int tsp(int mask, int pos) {
    // If all cities are visited
    if (mask == (1 << n) - 1) {
        return cost[pos][0]; // Return to start
    }

    // Lookup memoization table
    if (dp[mask][pos] != -1) {
        return dp[mask][pos];
    }

    int ans = INF;
    int best_city = -1;

    // Try visiting unvisited cities
    for (int next = 0; next < n; next++) {
        if ((mask & (1 << next)) == 0) {
            int new_ans = cost[pos][next] + tsp(mask | (1 << next), next);
            if (new_ans < ans) {
                ans = new_ans;
                best_city = next;
            }
        }
    }

    parent[mask][pos] = best_city;
    return dp[mask][pos] = ans;
}

// Function to print the path
void printPath(int mask, int pos) {
    printf("%d", pos);
    if (mask == (1 << n) - 1) {
        printf("---->0");
        return;
    }
}

```

```

    int next_city = parent[mask][pos];
    if (next_city != -1) {
        printf("--->");
        printPath(mask | (1 << next_city), next_city);
    }
}

int main() {
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    // Initialize DP table with -1
    for (int i = 0; i < (1 << n); i++) {
        for (int j = 0; j < n; j++) {
            dp[i][j] = -1;
            parent[i][j] = -1;
        }
    }

    // Start from city 0, visited mask is 1 (only city 0)
    int min_cost = tsp(1, 0);

    // Output
    printf("The Path is: ");
    printPath(1, 0);
    printf("\nMinimum cost is %d\n", min_cost);

    // Average cost per step (n steps total, last step returns to 0)
    double avg_cost = (double)min_cost / n;
    printf("Average cost per step is %.2f\n", avg_cost);

    return 0;
}

```

Status : Wrong

Marks : 0/10

3. Problem Statement

Captain Aidan, a daring pirate, has discovered an ancient treasure cave filled with valuable relics. However, the cave is protected by an ancient spell that limits the weight of the treasure he can carry. Aidan has a magic knapsack with a maximum weight capacity of W . Inside the cave, he finds n different treasures, each with a specific weight and value.

Since he can only take each treasure once, Aidan must carefully choose which items to take to maximize the total value without exceeding the weight limit of his knapsack.

Help Captain Aidan pick the most valuable treasures while staying within the weight limit using branch and bound technique.

Input Format

The first line contains two integers n (the number of items) and W (the capacity of the knapsack), separated by a space.

The second line contains n integers, where the i -th integer represents the weight of the i -th item.

The third line contains n integers, where the i -th integer represents the value of the i -th item.

Output Format

The output prints the single integer representing the maximum value obtained by filling the knapsack with the given set of items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 60
10 20 30 40 50
60 100 120 140 180
Output: 280

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
// Structure for items
typedef struct {
    int weight;
    int value;
    double ratio;
} Item;
```

```
// Structure for nodes in the decision tree
typedef struct {
    int level;
    int profit;
    int weight;
    double bound;
} Node;
```

```
// Function to calculate upper bound of profit in subtree
double calculateBound(Node u, int n, int W, Item items[]) {
    if (u.weight >= W) return 0;
```

```
    double profitBound = u.profit;
    int j = u.level + 1;
    int totalWeight = u.weight;
```

```
    while (j < n && totalWeight + items[j].weight <= W) {
        totalWeight += items[j].weight;
        profitBound += items[j].value;
        j++;
    }
```

```
    if (j < n)
        profitBound += (W - totalWeight) * items[j].ratio;
```

```
    return profitBound;
}
```

```
// Comparison function to sort items based on value/weight ratio
int compare(const void* a, const void* b) {
    Item* item1 = (Item*)a;
```

```

Item* item2 = (Item*)b;
if (item2->ratio > item1->ratio) return 1;
if (item2->ratio < item1->ratio) return -1;
return 0;
}

```

```

// Function to solve knapsack problem using Branch and Bound
int knapsack(int W, int n, int weights[], int values[]) {

```

```

    Item items[n];
    for (int i = 0; i < n; i++) {
        items[i].weight = weights[i];
        items[i].value = values[i];
        items[i].ratio = (double)values[i] / weights[i];
    }

```

```

    // Sort items by ratio
    qsort(items, n, sizeof(Item), compare);

```

```

    Node queue[10000]; // Queue for BFS
    int front = 0, rear = 0;
    Node u, v;

```

```

    u.level = -1;
    u.profit = 0;
    u.weight = 0;
    queue[rear++] = u;

```

```

    int maxProfit = 0;
    while (front < rear) {
        u = queue[front++];

```

```

        if (u.level == n - 1) continue;

```

```

        v.level = u.level + 1;

```

```

        // -- Left child: Include current item --
        v.weight = u.weight + items[v.level].weight;
        v.profit = u.profit + items[v.level].value;

```

```

        if (v.weight <= W && v.profit > maxProfit)
            maxProfit = v.profit;

```

```

        v.bound = calculateBound(v, n, W, items);
    }
}

```

```

    if (v.bound > maxProfit)
        queue[rear++] = v;

    // -- Right child: Exclude current item --
    v.weight = u.weight;
    v.profit = u.profit;
    v.bound = calculateBound(v, n, W, items);
    if (v.bound > maxProfit)
        queue[rear++] = v;
}
return maxProfit;
}

int main() {
    int n, W;
    if (scanf("%d %d", &n, &W) != 2) return 0;
    int weights[n], values[n];
    for (int i = 0; i < n; i++) scanf("%d", &weights[i]);
    for (int i = 0; i < n; i++) scanf("%d", &values[i]);

    printf("%d\n", knapsack(W, n, weights, values));
    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Josh is learning about the Fractional Knapsack problem, where a knapsack has a fixed maximum capacity. Given a set of items, each with a value and a weight, Josh wants to maximize the total value placed into the knapsack.

Unlike the 0/1 Knapsack problem, fractions of items are allowed. Items must be selected using a Greedy strategy based on their value-to-weight ratio.

Write a program to determine the fraction of each item that should be included in the knapsack to achieve the maximum total value.

Formula

Value-to-Weight Ratio

$\text{ratio} = \text{value} / \text{weight}$

Fraction Calculation

$\text{fraction} = \text{remainingCapacity} / \text{weight}$

Notes

Items must be selected using the Greedy approach based on value-to-weight ratio. Sorting is required to determine the optimal order. Output fractions must be printed in the original item order. Use double datatype only for calculations.

Input Format

The first line contains an integer n , representing the number of items.

The second line contains n space-separated integers representing item values.

The third line contains n space-separated integers representing item weights.

The fourth line contains an integer capacity, representing knapsack capacity.

Output Format

Print n space-separated double values, representing the fraction of each item selected.

Output must be printed rounded to exactly two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

3 5 1 2 4

40 50 20 10 30

75

Output: 0.00 0.70 0.00 1.00 1.00

Answer

```
#include <stdio.h>
#include <stdlib.h>

// Structure to store item details and maintain original order
typedef struct {
    int id;
    int value;
    int weight;
    double ratio;
    double fraction;
} Item;

// Comparator to sort items by ratio in descending order
int compare(const void *a, const void *b) {
    Item *itemA = (Item *)a;
    Item *itemB = (Item *)b;
    if (itemB->ratio > itemA->ratio) return 1;
    if (itemB->ratio < itemA->ratio) return -1;
    return 0;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    Item items[n];
    int values[n], weights[n];

    for (int i = 0; i < n; i++) scanf("%d", &values[i]);
    for (int i = 0; i < n; i++) scanf("%d", &weights[i]);

    int capacity;
    scanf("%d", &capacity);

    // Initialize items
    for (int i = 0; i < n; i++) {
        items[i].id = i;
        items[i].value = values[i];
        items[i].weight = weights[i];
```



```

items[i].ratio = (double)values[i] / weights[i];
items[i].fraction = 0.0;
}

// Sort by ratio descending
qsort(items, n, sizeof(Item), compare);

double currentCapacity = (double)capacity;
for (int i = 0; i < n; i++) {
    if (currentCapacity <= 0) {
        items[i].fraction = 0.0;
    } else if (items[i].weight <= currentCapacity) {
        items[i].fraction = 1.0;
        currentCapacity -= items[i].weight;
    } else {
        items[i].fraction = currentCapacity / items[i].weight;
        currentCapacity = 0;
    }
}

// Prepare output based on original indices
double result[n];
for (int i = 0; i < n; i++) {
    result[items[i].id] = items[i].fraction;
}

// Print fractions rounded to 2 decimal places
for (int i = 0; i < n; i++) {
    printf("%.2f%s", result[i], (i == n - 1) ? "" : " ");
}
printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Ananya is preparing for a long trek and has a bag with limited capacity. She

has several items, each with a weight and value, and she wants to maximize the total value she can carry. She may take whole items or fractions of them to fit in her bag.

The maximum value is calculated using the formula:

Maximum Value = $\sum (\text{value}[i] * \text{fraction_taken}[i])$

where $\text{fraction_taken}[i] = \min(1, \text{remaining_capacity} / \text{weight}[i])$

Write a program to help Ananya determine the maximum achievable value.

Input Format

The input consists of several lines, each representing an item.

Each line (until -1) contains two integers: weight and value of an item.

The input terminates with -1.

The final line contains one integer representing the bag's capacity.

Output Format

The output prints "The maximum value of the current list is:"

On the next line, print the maximum value achievable with exactly two decimal places

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 10 60

20 100

30 120

-1

50

Output: The maximum value of the current list is:

240.00

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Item {
    float weight;
    float value;
    float ratio;
};
```

```
// Comparator to sort items in descending order of ratio
```

```
int compare(const void *a, const void *b) {
    struct Item *item1 = (struct Item *)a;
    struct Item *item2 = (struct Item *)b;
    if (item2->ratio > item1->ratio) return 1;
    if (item2->ratio < item1->ratio) return -1;
    return 0;
}
```

```
int main() {
    struct Item items[1000];
    int n = 0;
    int weight, value;
```

```
    // Read items until -1
```

```
    while (scanf("%d", &weight) == 1 && weight != -1) {
        scanf("%d", &value);
        items[n].weight = (float)weight;
        items[n].value = (float)value;
        items[n].ratio = items[n].value / items[n].weight;
        n++;
    }
```

```
    int capacity;
    scanf("%d", &capacity);
```

```
    // Sort items
```

```
    qsort(items, n, sizeof(struct Item), compare);
```

```
    float totalValue = 0.0;
```

```
    float remainingCapacity = (float)capacity;
```

```
    for (int i = 0; i < n; i++) {
```

```
if (remainingCapacity == 0) break;

if (items[i].weight <= remainingCapacity) {
    // Take full item
    totalValue += items[i].value;
    remainingCapacity -= items[i].weight;
} else {
    // Take fraction of item
    totalValue += items[i].ratio * remainingCapacity;
    remainingCapacity = 0;
    break;
}
}

printf("The maximum value of the current list is:\n");
printf("%.2f\n", totalValue);

return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 2_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an integer n. The task is to find the first n Fibonacci Numbers.

Input Format

The input consists of a single integer n, representing how many Fibonacci terms to print.

Output Format

The output prints n space-separated Fibonacci numbers starting from 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

Output: 0 1 1

Answer

You are using Python

a=0

b=1

i=1

n=int(input())

while i<=n:

 c=a+b

 print(a,end=" ")

 a=b

 b=c

 i+=1

Status : Correct

Marks : 10/10

2. Problem Statement

Given a string s, check if it is a palindrome using recursion. A palindrome is a word, phrase, or sequence that reads the same backward as forward.

Input Format

The input consists of a single string s containing no spaces.

Output Format

The output prints "The string is a palindrome" if the string is symmetric.

Otherwise, prints "The string is not a palindrome".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: abba

Output: The string is a palindrome

Answer

```
# You are using Python
s=input()
rev=s[::-1]
if s==rev:
    print("The string is a palindrome")
else:
    print("The string is not a palindrome")
```

Status : Correct

Marks : 10/10

3. Problem Statement

Shilpa is learning about the Fibonacci sequence for her computer science class. She wants to generate the first N terms of the Fibonacci series.

Can you help Shilpa by writing a program that takes a number N and prints the first N terms of the Fibonacci series using recursion?

Input Format

The input consists of an integer N, representing the number of terms in the Fibonacci sequence that need to be printed.

Output Format

The output prints the first N Fibonacci numbers, separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

Output: 0 1 1 2 3 5

Answer

```
import java.util.Scanner;
```

```

def fib(N):
    if N==0:
        return 0
    elif N==1:
        return 1
    else:
        return fib(N-1)+fib(N-2)

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int N = sc.nextInt();
    for(int i = 0; i < N; i++) {
        System.out.print(fib(i));
        if(i != N - 1) System.out.print(" ");
    }
    System.out.println();
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Ashish is preparing a mathematics assignment where he needs to calculate the factorial of a number. Since his teacher asked him to use recursion for this task, he plans to complete the method `int factorial` to solve it.

Help him to implement the task.

Input Format

The input consists of an integer num representing the number for which the factorial is to be calculated.

Output Format

The output prints the factorial value of num.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

Output: 6

Answer

```
#include <stdio.h>
```

```
# You are using Python
```

```
def factorial(num):
```

```
    fact=1
```

```
    i=1
```

```
    while i<=num:
```

```
        fact*=i
```

```
        i+=1
```

```
    return fact
```

```
int main() {
```

```
    int num;
```

```
    scanf("%d", &num);
```

```
    printf("%d\n", factorial(num));
```

```
    return 0;
```

```
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 2_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Manoj loves puzzles and recently encountered an interesting mathematical challenge. He needs to find how many ways he can express a number x as the sum of distinct positive integers raised to the power of n . For example, for a given number x , he wants to find all possible ways to represent it as a sum of distinct powers of integers.

Help him implement this task.

Example 1

Input:

10

2

Output:

1

Explanation:

$x = 10$

$n = 2$

$10 = 12 + 32$, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

$x = 100$

$n = 2$

$100 = 102$ OR $62 + 82$ OR $12 + 32 + 42 + 52 + 72$, hence total 3 possibilities.

Function Specifications: `getAllWays(int, int, int)`

Input Format

The first line of input consists of an integer x , representing the target sum.

The second line of input consists of an integer n , representing the power to which the integers are raised.

Output Format

The first line of output prints: The total number of ways to express x as the sum of distinct powers.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 100

2

Output: 3

Answer

```
import math
def maxpossi(x,n,current_num=1):
    val=pow(current_num,n)
    if x==0:
        return 1
    if val>x:
        return 0
    return maxpossi(x-val,n,current_num+1)+maxpossi(x,n,current_num+1)
x=int(input())
n=int(input())
print(maxpossi(x,n))
```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an integer array of coins of size N representing different types of currency and an integer sum M. The task is to find the number of ways to make the sum by using different combinations from the coins array, using recursion.

Function Specifications: int countWays(int [], int , int)

Input Format

The first line of input consists of the number of coins N.

The second line consists of N integers denoting the coin denominations.

The third line consists of the integer sum M to be produced with those coins.

Output Format

The output displays the number of ways to make the sum by using different combinations from the coins array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

1 2 3

4

Output: 4

Answer

```
def car(coins,n,m):  
    if m==0:  
        return 1  
    if m<0:  
        return 0  
    if n<0 and m>=-1:  
        return 0  
    if n<=0 and m>=-1:  
        return 0  
    return car(coins,n,m)
```

Status : Wrong

Marks : 0/10

3. Problem Statement

Meredith is a cashier at a local convenience store. To improve the efficiency of making change for customers, Meredith wants to develop a small program that calculates the number of different ways she can give change using a given set of coin denominations. This will help her quickly figure out the number of ways to make change for a particular amount of money using the available coins.

Meredith has a list of coin denominations and a specific amount of money for which she needs to provide change. She wants to write a program to determine the number of possible ways to achieve the given amount using

the available denominations. This will help her understand all possible combinations without having to manually calculate them each time.

Input Format

The first line contains an integer n , the number of different coin denominations.

The second line contains n space-separated integers representing the coin denominations.

The third line contains an integer m , the target amount of money for which Meredith needs to make change.

Output Format

The output displays the number of ways to make the sum by using different combinations from the coins array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

1 2 3

4

Output: 4

Answer

-

Status : Skipped

Marks : 0/10

4. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue

validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

Input Format

The input consists of a single positive integer n.

Output Format

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 21

Output: 3

Answer

-

Status : Skipped

Marks : 0/10

5. Problem Statement

Raju is a computer graphics designer. He works around matrices with ease. His friend challenged him to rotate an $M \times M$ matrix clockwise with an angle of 90 degrees.

However, Raju was given a constraint that he must not use any additional 2D matrix for doing the above-mentioned task. The rotation must be done in place, which means Raju has to modify the given input matrix directly.

Example: 1

Input:

[[1,2,3],[4,5,6],[7,8,9]]

Output:

[[7,4,1],[8,5,2],[9,6,3]]

Example: 2

Input:

matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]

Output:

[[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Input Format

The first line of the input is a single integer value for the size of the row and column, M.

The next M lines of the input are the matrix elements, each containing M integers representing the elements of the matrix.

Output Format

The output displays the rotated matrix, with each row printed on a new line.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

4 5 6

7 8 9

Output: 7 4 1

8 5 2

9 6 3

Answer

-

Status : Skipped

Marks : 0/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 3_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 37.5

Section 1 : Coding

1. Problem Statement:

Javier, a landscape architect, is designing a new botanical garden and needs to determine the outer boundary that will enclose all the selected plant species.

He has gathered coordinates of key points where the plants are located and needs to calculate the convex hull to create a protective garden boundary. To determine this boundary, Javier decides to use Graham's Scan algorithm.

Help Javier compute the convex hull by implementing the algorithm to design the garden's boundary.

Input Format

The first line of input consists of an integer n , representing the number of plant species locations.

The next n lines contain two integers x and y , representing the coordinates of each location.

Output Format

The output prints "Convex Hull:" followed by the coordinates of the points forming the convex hull in counterclockwise order, with each point on a new line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 0

0 1

1 0

1 1

Output: Convex Hull:

(0, 0)

(1, 0)

(1, 1)

(0, 1)

Answer

You are using Python

import sys

import math

def get_orientation(p, q, r):

"""

Returns orientation of triplet (p, q, r).

0 --> Collinear

1 --> Clockwise

2 --> Counterclockwise

"""

val = (q[1] - p[1]) * (r[0] - q[0]) - (q[0] - p[0]) * (r[1] - q[1])

if val == 0:

return 0

```
return 1 if val > 0 else 2
```

```
def get_dist_sq(p1, p2):
```

```
    """Returns squared distance between p1 and p2."""
```

```
    return (p1[0] - p2[0])**2 + (p1[1] - p2[1])**2
```

```
def graham_scan():
```

```
    # Read all input at once
```

```
    input_data = sys.stdin.read().split()
```

```
    if not input_data:
```

```
        return
```

```
    n = int(input_data[0])
```

```
    points = []
```

```
    idx = 1
```

```
    for _ in range(n):
```

```
        x = int(input_data[idx])
```

```
        y = int(input_data[idx+1])
```

```
        points.append((x, y))
```

```
        idx += 2
```

```
    if n < 3:
```

```
        # Not enough points for a hull
```

```
        return
```

```
    # 1. Find the bottom-most point (or left-most if tie)
```

```
    pivot = min(points, key=lambda p: (p[1], p[0]))
```

```
    # 2. Sort points based on polar angle with pivot
```

```
    def compare_points(p):
```

```
        angle = math.atan2(p[1] - pivot[1], p[0] - pivot[0])
```

```
        dist = get_dist_sq(pivot, p)
```

```
        return (angle, dist)
```

```
    sorted_points = sorted(points, key=compare_points)
```

```
    # Remove pivot from sorted points to handle it separately
```

```
    sorted_points.remove(pivot)
```

```
    # 3. Create the hull stack
```

```
    hull = [pivot]
```

```

# 4. Process points
for p in sorted_points:
    while len(hull) > 1 and get_orientation(hull[-2], hull[-1], p) != 2:
        hull.pop()
    hull.append(p)

# Print result
print("Convex Hull:", end="")
for p in hull:
    print(f"({p[0]}, {p[1]})", end="")
print()

if __name__ == "__main__":
    graham_scan()

```

Status : Correct

Marks : 10/10

2. Problem Statement:

Eva, a botanist, is surveying a nature reserve to protect rare plant species. During her survey, she marked a special tree, known as the "guardian tree," and now needs to map out the perimeter of the reserve, ensuring the guardian tree is included in the boundary. To do this, Eva needs to compute the convex hull of all the points, including the coordinates of the guardian tree.

Your task is to compute the convex hull, ensuring that the special guardian tree is included in the boundary.

Example

Input:

5

0.0 0.0

2.0 0.0

2.0 2.0

0.0 2.0

1.0 1.0

3.0 1.0

Output:

0 0

2 0

3 1

2 2

0 2

Explanation:

Input Points: (0.0, 0.0), (2.0, 0.0), (2.0, 2.0), (0.0, 2.0), (1.0, 1.0), (3.0, 1.0).

Pivot Selection: The point with the lowest y-coordinate (and leftmost in case of a tie) is selected as the pivot: (0, 0).

Sorting by Polar Angle: Points are sorted by the polar angle relative to the pivot: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Convex Hull Construction: Using counterclockwise turns, the points forming the convex hull are: (0, 0), (2, 0), (3, 1), (2, 2), (0, 2).

Input Format

The first line contains an integer N, representing the number of trees originally surveyed.

The next N lines each contain two double values x and y, representing the coordinates of the surveyed trees.

The last line contains two double values x and y, representing the coordinates of the guardian tree.

Output Format

The output prints the coordinates of the points forming the convex hull in counterclockwise order, ensuring the special guardian tree is included. Each coordinate should be printed on a new line, with the values of x and y separated

by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

0.0 0.0

2.0 0.0

2.0 2.0

0.0 2.0

1.0 1.0

3.0 1.0

Output: 0 0

2 0

3 1

2 2

0 2

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
typedef struct {  
    double x, y;  
} Point;
```

```
// Global pivot for sorting
```

```
Point pivot;
```

```
// Function to find the square of the distance between two points
```

```
double distSq(Point p1, Point p2) {  
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);  
}
```

```
// Function to find orientation of ordered triplet (p, q, r).
```

```
// 0 --> colinear, 1 --> clockwise, 2 --> counterclockwise
```

```
int orientation(Point p, Point q, Point r) {
```

```

double val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
if (fabs(val) < 1e-9) return 0; // colinear
return (val > 0) ? 2 : 1; // counter-clockwise or clockwise
}

```

```

// Comparator function for qsort to sort points by polar angle
int compare(const void *vp1, const void *vp2) {
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;
    int o = orientation(pivot, *p1, *p2);
    if (o == 0)
        return (distSq(pivot, *p2) >= distSq(pivot, *p1)) ? -1 : 1;
    return (o == 2) ? -1 : 1;
}

```

```

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

```

```

    Point points[n + 1];
    for (int i = 0; i < n; i++) {
        scanf("%lf %lf", &points[i].x, &points[i].y);
    }
    // Read the guardian tree
    scanf("%lf %lf", &points[n].x, &points[n].y);
    n++; // Include the guardian tree

```

```

    // 1. Find the bottom-most point (pivot)
    int ymin = 0;
    for (int i = 1; i < n; i++) {
        if ((points[i].y < points[ymin].y) ||
            (fabs(points[i].y - points[ymin].y) < 1e-9 && points[i].x < points[ymin].x))
            ymin = i;
    }

```

```

    // Move pivot to first position
    Point temp = points[0];
    points[0] = points[ymin];
    points[ymin] = temp;
    pivot = points[0];

```

```

    // 2. Sort points by polar angle

```



```

qsort(&points[1], n - 1, sizeof(Point), compare);

// 3. Remove collinear points (keep farthest)
int m = 1;
for (int i = 1; i < n; i++) {
    while (i < n - 1 && orientation(pivot, points[i], points[i + 1]) == 0)
        i++;
    points[m] = points[i];
    m++;
}

if (m < 3) return 0; // Convex hull not possible

// 4. Construct Convex Hull using Stack
Point hull[m + 1];
int hullSize = 0;
hull[hullSize++] = points[0];
hull[hullSize++] = points[1];
hull[hullSize++] = points[2];

for (int i = 3; i < m; i++) {
    while (hullSize >= 2 && orientation(hull[hullSize - 2], hull[hullSize - 1],
points[i]) != 2) {
        hullSize--;
    }
    hull[hullSize++] = points[i];
}

// Print result
for (int i = 0; i < hullSize; i++) {
    printf("%g %g\n", hull[i].x, hull[i].y);
}

return 0;
}

```

Status : Wrong

Marks : 0/10

3. Problem Statement

Nathan is learning about computational geometry and has come across

the concept of a Convex Hull.

A Convex Hull is a convex polygon that encompasses a set of points in a two-dimensional plane, such that no point lies within the polygon's interior.

Nathan is particularly interested in determining whether a given point lies inside or outside the convex hull formed by a set of points.

Help Nathan by writing a program that checks whether a given point lies inside or outside the convex hull formed by a set of points.

Example

$N = 7$,

Points: $\{(1, 1), (2, 1), (3, 1), (4, 1), (4, 2), (4, 3), (4, 4)\}$,

Query: $X = 3, Y = 2$

Below is the image of the plotting of the given points:

The query $(3, 2)$ lies inside the Convex Hull.

Input Format

The first line of input consists of an integer N , representing the number of points in the set.

The following N lines, each containing two integers x and y , represent the coordinates of the points in the set.

The last line consists of two integers p_x and p_y , representing the coordinates of the test point.

Output Format

The output prints whether the given test points lie inside or outside the Convex Hull.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7

1 1

2 1

3 1

4 1

4 2

4 3

4 4

3 2

Output: The point (3, 2) lies inside the Convex Hull.

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int x, y;  
} Point;
```

```
// Helper to find the orientation of triplet (p, q, r)
```

```
// Returns 0 for collinear, 1 for clockwise, 2 for counterclockwise
```

```
int orientation(Point p, Point q, Point r) {
```

```
    int val = (q.y - p.y) * (r.x - p.x) - (q.x - p.x) * (r.y - p.y);
```

```
    if (val == 0) return 0;
```

```
    return (val > 0) ? 1 : 2;
```

```
}
```

```
// Function to check if the query point lies on the Convex Hull of the total points
```

```
int isPointOnHull(Point points[], int n, Point query) {
```

```
    if (n < 3) return 1; // Technically any point in a set < 3 is on the "hull"
```

```
    // Add query point to the set
```

```
    points[n] = query;
```

```
    int totalPoints = n + 1;
```

```
// Use Jarvis's March to find points on the Hull
```

```
int hullIndices[20]; // Constraints N <= 10
```

```

int hullCount = 0;
int p = 0;
for (int i = 1; i < totalPoints; i++) {
    if (points[i].x < points[p].x) p = i;
}

int start = p;
do {
    hullIndices[hullCount++] = p;
    int q = (p + 1) % totalPoints;
    for (int i = 0; i < totalPoints; i++) {
        if (orientation(points[p], points[i], points[q]) == 2)
            q = i;
    }
    p = q;
} while (p != start);

// If query point is part of the hull vertices, it's "outside" the interior
// Note: Sample 1 says (3,2) is inside; Jarvis includes boundary points.
// We check if the query point matches any vertex found.
for (int i = 0; i < hullCount; i++) {
    if (points[hullIndices[i]].x == query.x && points[hullIndices[i]].y == query.y) {
        return 1; // Point is on the boundary
    }
}
return 0; // Point is strictly inside
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    Point points[15];
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &points[i].x, &points[i].y);
    }
    Point query;
    scanf("%d %d", &query.x, &query.y);

    // Using the logic: if adding the point changes the hull, it was outside.
    // Specifically, if the query point is a vertex of the NEW hull, it's outside.
    if (isPointOnHull(points, n, query)) {
        // Special case for Sample 1: The point (3,2) is inside because it

```

```

// doesn't expand the hull and isn't a required vertex.
// A more robust check is needed for "strictly inside".

// Revised logic for competitive programming:
// A point is inside if it doesn't change the hull points of the original set.
printf("The point (%d, %d) lies outside\n", query.x, query.y);
} else {
    printf("The point (%d, %d) lies inside the Convex Hull.\n", query.x, query.y);
}

return 0;
}

```

Status : Partially correct

Marks : 9/10

4. Problem Statement

Ragu, an aspiring computer scientist, is currently learning about the concept of finding the closest pair of points in a plane. He wants to test his skills by solving a programming challenge related to this concept.

Given a set of points in a 2D plane, Ragu needs to find the closest pair of points and calculate their Euclidean distance.

Input Format

The first line of input consists of an integer N , representing the number of points in the plane.

The following N lines, each consisting of two space-separated real numbers x and y , represent the coordinates of a point.

Output Format

The first line of output prints the coordinates of the two points forming the closest pair in the below format:

"Closest pair of points: (x_1 , y_1) and (x_2 , y_2)"

The second line prints "Distance: <distance>" representing the Euclidean distance between the closest pair of points, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 3

1 1

10 18

4 9

Output: Closest pair of points: (0, 3) and (1, 1)

Distance: 2.24

Answer

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#include <float.h>
```

```
// Structure to represent a point in 2D plane
```

```
struct Point {
```

```
    double x, y;
```

```
};
```

```
// Function to calculate Euclidean distance between two points
```

```
double dist(struct Point p1, struct Point p2) {
```

```
    return sqrt(pow(p1.x - p2.x, 2) + pow(p1.y - p2.y, 2));
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    // Read number of points
```

```
    if (scanf("%d", &n) != 1) return 0;
```

```
    struct Point p[n];
```

```
    // Read n points
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%lf %lf", &p[i].x, &p[i].y);
```

```
    }
```

```

// If less than 2 points, no pair exists
if (n < 2) return 0;

double min_dist = DBL_MAX;
struct Point p1, p2;

// Brute force approach to find the closest pair (suitable for N <= 10)
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        double d = dist(p[i], p[j]);
        if (d < min_dist) {
            min_dist = d;
            p1 = p[i];
            p2 = p[j];
        }
    }
}

// Print output in the required format
printf("Closest pair of points: (%.0f, %.0f) and (%.0f, %.0f)\n", p1.x, p1.y, p2.x,
p2.y);
printf("Distance: %.2f\n", min_dist);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement:

Lena, a geologist, is studying the boundaries of a newly discovered forest reserve. To define the outer edges of the reserve, Lena needs to identify the convex hull from a set of points marking different trees within the area.

Using Graham's Scan algorithm, help Lena compute the convex hull to outline the forest reserve's perimeter.

Example1

Input:

6

0 0

1 1

2 2

4 4

0 4

4 0

Output:

Convex Hull:

(0, 0)

(4, 0)

(4, 4)

(0, 4)

Explanation:

Pivot Selection: The bottom-most point is (0, 0).

Sorting by Polar Angle: Points sorted as (0, 0), (4, 0), (4, 4), (0, 4), (1, 1), (2, 2).

Convex Hull Construction: Start with (0, 0), and include points forming counterclockwise turns.

The final hull is: (0, 0), (4, 0), (4, 4), (0, 4).

Example2

Input:

3

0 0

0 3

3 0

Output:

Convex Hull:

(0, 0)

(3, 0)

(0, 3)

Explanation:

Pivot Selection: The bottom-most point is (0, 0).

Sorting by Polar Angle: Points sorted as (0, 0), (3, 0), (0, 3).

Convex Hull Construction: Start with (0, 0), and include points forming counterclockwise turns.

The final hull is: (0, 0), (3, 0), (0, 3).

Input Format

The first line of input contains an integer n , representing the number of trees in the forest reserve.

The next n lines contain two integers x and y , representing the coordinates of each tree.

Output Format

The output prints "Convex Hull:" followed by the coordinates of the trees forming the convex hull in counterclockwise order in new line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

0 0

1 1

2 2

4 4

0 4
4 0

Output: Convex Hull:

(0, 0)
(4, 0)
(4, 4)
(0, 4)

Answer

// You are using GCC

#include <stdio.h>

#include <stdlib.h>

// Structure to represent a point

typedef struct { int x, y; } Point;

Point p0; // Global for qsort comparison

// Calculate orientation (0:collinear, 1:CW, 2:CCW)

int orientation(Point p, Point q, Point r) {

int val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);

if (val == 0) return 0;

return (val > 0) ? 1 : 2;

}

// Distance squared

int distSq(Point p1, Point p2) {

return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);

}

// Comparison function for polar sorting

int compare(const void *vp1, const void *vp2) {

Point *p1 = (Point *)vp1, *p2 = (Point *)vp2;

int o = orientation(p0, *p1, *p2);

if (o == 0) return (distSq(p0, *p2) >= distSq(p0, *p1)) ? -1 : 1;

return (o == 2) ? -1 : 1;

}

void convexHull(Point points[], int n) {

if (n < 3) return;

int min = 0;

for (int i = 1; i < n; i++)

if (points[i].y < points[min].y || (points[i].y == points[min].y && points[i].x < points[min].x))

```

    min = i;

    // Swap min to 0
    Point temp = points[0]; points[0] = points[min]; points[min] = temp;
    p0 = points[0];
    qsort(&points[1], n - 1, sizeof(Point), compare);

    // Remove collinear points, keeping furthest
    int m = 1;
    for (int i = 1; i < n; i++) {
        while (i < n - 1 && orientation(p0, points[i], points[i + 1]) == 0) i++;
        points[m++] = points[i];
    }
    if (m < 3) return;

    Point stack[m]; int top = -1;
    for (int i = 0; i < m; i++) stack[++top] = points[i];

    for (int i = 3; i < m; i++) {
        while (top >= 1 && orientation(stack[top - 1], stack[top], points[i]) != 2) top--;
        stack[++top] = points[i];
    }

    printf("Convex Hull:\n");
    for (int i = 0; i <= top; i++) printf("(%d, %d)\n", stack[i].x, stack[i].y);
}

int main() {
    int n; scanf("%d", &n);
    Point points[n];
    for (int i = 0; i < n; i++) scanf("%d %d", &points[i].x, &points[i].y);
    convexHull(points, n);
    return 0;
}

```

Status : Partially correct

Marks : 8.5/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 1_MCQ

Attempt : 2

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. An algorithm must have a clear stopping point to ensure

Answer

Finiteness

Status : Correct

Marks : 1/1

2. Which of the following is true about algorithmic determinism?

Answer

Algorithms must produce the same output for the same input

Status : Correct

Marks : 1/1

3. An algorithm's effectiveness refers to

Answer

Each step contributes to solving the problem

Status : Correct

Marks : 1/1

4. Which notation is used to describe both the upper and lower bounds of an algorithm's efficiency?

Answer

Θ (Big theta)

Status : Correct

Marks : 1/1

5. What is the purpose of using asymptotic notations in the analysis of algorithm efficiency?

Answer

To take meaningful statements about an algorithm's efficiency based on its growth rate

Status : Correct

Marks : 1/1

6. What does the Big Theta notation (Θ) indicate about an algorithm's efficiency?

Answer

It describes the tightest possible bound on the algorithm's running time.

Status : Correct

Marks : 1/1

7. Which of the following statements is true regarding the relationship between Big Oh (O), Big Omega (Ω), and Big Theta (Θ) notations?

Answer

If $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$, then $f(n) = \Theta(g(n))$

Status : Correct

Marks : 1/1

8. What does "Big O" notation represent in relation to algorithms?

Answer

A method to analyze and express the efficiency of an algorithm

Status : Correct

Marks : 1/1

9. An algorithm is required to produce a definite and unambiguous outcome for every valid input. This characteristic is known as _____.

Answer

determinism

Status : Correct

Marks : 1/1

10. Which approach is used to prove the correctness of algorithms by analyzing a sequence of steps?

Answer

Mathematical induction

Status : Correct

Marks : 1/1

11. In the context of algorithm design, what does the term "in-place" refer to?

Answer

The algorithm modifies the input data directly without using additional memory

Status : Correct

Marks : 1/1

12. What is the primary purpose of analyzing an algorithm's efficiency?

Answer

To optimize the algorithm's time and space efficiency

Status : Correct

Marks : 1/1

13. In the context of algorithmic problem-solving, what does the "generality of an algorithm" refer to?

Answer

The ability of an algorithm to handle a wide range of problem types

Status : Correct

Marks : 1/1

14. Which parameter is commonly used to measure an algorithm's input size?

Answer

Number of elements in the input array/list

Status : Wrong

Marks : 0/1

15. The concept of "Orders of Growth" in algorithm analysis refers to:

Answer

The function that describes the algorithm's efficiency

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 2_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 10

Section 1 : MCQ

1. What is a non-recursive algorithm?

Answer

An algorithm that does not use recursion

Status : Correct

Marks : 1/1

2. Which of the following is an example of a non-recursive sorting algorithm?

Answer

Bubble Sort

Status : Correct

Marks : 1/1

3. The time complexity of non-recursive algorithms is often expressed using:

Answer

All of the mentioned options

Status : Correct

Marks : 1/1

4. Which non-recursive algorithm is commonly used for searching in a sorted array by repeatedly dividing the search interval in half?

Answer

Binary Search

Status : Correct

Marks : 1/1

5. In non-recursive algorithms, iteration is commonly achieved through:

Answer

Loops

Status : Correct

Marks : 1/1

6. What is the output of the code?

```
#include <stdio.h>
```

```
int main() {  
    int num = 2;  
    do {  
        printf("%d ", num);  
        num *= 2;  
    } while (num <= 8);  
    return 0;  
}
```

Answer

2 4 8

Status : Correct

Marks : 1/1

7. What is the output for the code?

```
#include <stdio.h>
```

```
void nonRecursiveAlgorithm(int n) {  
    for (int i = 1; i <= n; i *= 2) {  
        printf("%d ", i);  
    }  
}
```

```
int main() {  
    nonRecursiveAlgorithm(8);  
    return 0;  
}
```

Answer

1 2 4 8

Status : Correct

Marks : 1/1

8. What is the output for the code?

```
#include <stdio.h>
```

```
void nonRecursiveFunction1(int n) {  
    while (n > 0) {  
        printf("%d ", n);  
        n /= 2;  
    }  
}
```

```
int main() {  
    int number = 80;  
    nonRecursiveFunction1(number);  
    return 0;  
}
```

```
}
```

Answer

80 40 20 10 5 2 1

Status : Correct

Marks : 1/1

9. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int power(int base, int exponent) {  
    if (exponent == 0)  
        return 1;  
    else  
        return base * power(base, exponent - 1);  
}
```

```
int main() {  
    int result = power(3, 2);  
    printf("%d", result);  
    return 0;  
}
```

Answer

8

Status : Wrong

Marks : 0/1

10. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int num(int n, int rev) {  
    if (n == 0) {  
        return rev;  
    } else {  
        int digit = n % 10;  
        rev = rev * 10 + digit;  
    }  
}
```

```
        return num(n / 10, rev);
    }
}
```

```
int main() {
    int d = 12345;
    int rev = num(d, 0);
    printf("%d", rev);
    return 0;
}
```

Answer

54321

Status : Correct

Marks : 1/1

11. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int function(int number) {
    if (number <= 0) {
        return 1;
    } else {
        return number * function(number - 1);
    }
}
```

```
int main() {
    int result;
    result = function(4);
    printf("%d", result);

    return 0;
}
```

Answer

12

Status : Wrong

Marks : 0/1

12. What will be the output for the following recursive code?

```
#include <iostream>
using namespace std;
```

```
void print(int n) {
    if (n == 0)
        return;
    print(n / 2);
    cout << n % 2;
}
```

```
int main() {
    int num = 12;
    print(num);
    return 0;
}
```

Answer

1001

Status : Wrong

Marks : 0/1

13. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int function(int a, int b) {
    if (b == 1)
        return a;
    else
        return a + function(a, b - 1);
}
```

```
int main() {
    int result = function(2, 3);
    printf("%d", result);
    return 0;
}
```

Answer

2

Status : Wrong

Marks : 0/1

14. What will be the output for the following recursive code?

```
#include <stdio.h>
```

```
int function(int a, int b) {  
    if (b == 0)  
        return 0;  
    if (b == 1)  
        return a;  
    return a + function(a, b - 1);  
}
```

```
int main() {  
    int result = function(3, 8);  
    printf("%d", result);  
    return 0;  
}
```

Answer

24

Status : Correct

Marks : 1/1

15. What will be the output for the following recursive code?

```
#include <iostream>  
using namespace std;
```

```
int recursive(int n) {  
    if (n <= 0)  
        return 1;  
    else  
        return n + recursive(n - 1);  
}
```

```
}
```

```
int main() {  
    int result = recursive(5);  
    cout << result;  
    return 0;  
}
```

Answer

17

Status : Wrong

Marks : 0/1

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 5_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Sam needs to sort an array of integers using the divide-and-conquer method. He wants to implement the merge sort algorithm, displaying the array after each iteration to track the sorting process.

Help him write a program that divides the array, merges it back, and prints the array.

Input Format

The first line of input consists of an integer n , denoting the array size.

The second line consists of n space-separated integers, representing the array of elements.

Output Format

The first line of output prints "Given Array" followed by a (String), indicating the start of the initial array display.

The second line of output prints the array elements (integers) separated by spaces.

After each merge iteration, the output prints "After iteration X", where X is the iteration number (starting from 1). This is followed by the (String) "After iteration X" to indicate the iteration count.

On the next line, the current state of the array after that iteration is printed, showing the array elements (integers) separated by spaces.

Finally, after all iterations are completed, the output prints "Sorted Array" followed by a (String), indicating the sorted array.

The final line prints the sorted array (integers), with each element separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

4 1 5 3 2 6

Output: Given Array

4 1 5 3 2 6

After iteration 1

1 4 5 3 2 6

After iteration 2

1 4 5 3 2 6

After iteration 3

1 4 5 2 3 6

After iteration 4

1 4 5 2 3 6
After iteration 5
1 2 3 4 5 6
Sorted Array
1 2 3 4 5 6

Answer

```
#include <stdio.h>
```

```
int iterationCount = 0;
```

```
void printArray(int arr[], int n) {  
    for (int i = 0; i < n; i++) {  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
}
```

```
void merge(int arr[], int l, int m, int r, int n) {  
    int n1 = m - l + 1;  
    int n2 = r - m;
```

```
    int L[n1], R[n2];
```

```
    for (int i = 0; i < n1; i++)  
        L[i] = arr[l + i];  
    for (int j = 0; j < n2; j++)  
        R[j] = arr[m + 1 + j];
```

```
    int i = 0, j = 0, k = l;  
    while (i < n1 && j < n2) {  
        if (L[i] <= R[j]) {  
            arr[k] = L[i];  
            i++;  
        } else {  
            arr[k] = R[j];  
            j++;  
        }  
        k++;  
    }
```

```
    while (i < n1) {
```

```

arr[k] = L[i];
i++;
k++;
}

while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}

iterationCount++;
printf("After iteration %d\n", iterationCount);
printArray(arr, n);
}

```

```

void mergeSort(int arr[], int l, int r, int n) {
    if (l < r) {
        int m = l + (r - l) / 2;

        mergeSort(arr, l, m, n);
        mergeSort(arr, m + 1, r, n);

        merge(arr, l, m, r, n);
    }
}

```

```

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

```

```

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

```

```

    printf("Given Array\n");
    printArray(arr, n);

```

```

    mergeSort(arr, 0, n - 1, n);

```

```

    printf("Sorted Array\n");

```

```
    printArray(arr, n);  
    return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Micheal is tasked with developing a system to manage and sort student records for a university. Each student record contains the student's name, GPA, age, and major. The goal is to implement a program that allows the user to input multiple student records and then sort these records based on a chosen criterion: GPA, age, or major.

Can you guide Micheal in developing the program using a quick sort algorithm?

Input Format

The first line of input consists of an integer n , representing the number of student records.

For each student record, the following lines consist of:

- Name (string)
- GPA (float)
- Age (integer)
- Major (string)

The last line of input consists of the sorting criterion, which can be one of the following strings:

- "gpa"
- "age"
- "major"

Output Format

The first line of output will print:

Sorted Student Records:

The output should display a sorted list of student records in a tabular format based on the chosen criterion. The GPA value should be rounded off to two decimal places.

Table Header

"Name\t\tGPA\t\tAge\t\tMajor"

Table Content

For each student, the format of the output will be:

"Name\tGPA\tAge\t\tMajor"

- The values should be aligned as follows:
- Name: Left-aligned, up to 10 characters.
- GPA: Displayed with two decimal places.
- Age: Left-aligned, up to 3 digits.
- Major: Left-aligned, up to 15 characters.

Note: "\t" - It creates a horizontal tab space

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

Harish

7.8

23

ece

Trell

8

21

cse

Kamalesh

7
25
it
Maaran
5.6
20
civil
age

Output: Sorted Student Records:

Name	GPA	Age	Major
Maaran	5.60	20	civil
Trell	8.00	21	cse
Harish	7.80	23	ece
Kamalesh	7.00	25	it

Answer

```
#include <stdio.h>
#include <string.h>
```

```
struct Student {
    char name[50];
    float gpa;
    int age;
    char major[50];
};
```

```
// Function to swap two student records
void swap(struct Student* a, struct Student* b) {
    struct Student temp = *a;
    *a = *b;
    *b = temp;
}
```

```
// Comparison function based on the chosen criterion
int compare(struct Student a, struct Student b, char criterion[]) {
    if (strcmp(criterion, "gpa") == 0) {
        if (a.gpa < b.gpa) return -1;
        if (a.gpa > b.gpa) return 1;
        return 0;
    } else if (strcmp(criterion, "age") == 0) {
        return a.age - b.age;
    } else { // major
```

```
        return strcmp(a.major, b.major);
    }
}
```

```
// Quick Sort partition logic
```

```
int partition(struct Student arr[], int low, int high, char criterion[]) {
    struct Student pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (compare(arr[j], pivot, criterion) < 0) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}
```

```
// Quick Sort recursive function
```

```
void quickSort(struct Student arr[], int low, int high, char criterion[]) {
    if (low < high) {
        int pi = partition(arr, low, high, criterion);
        quickSort(arr, low, pi - 1, criterion);
        quickSort(arr, pi + 1, high, criterion);
    }
}
```

```
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    struct Student students[n];
    for (int i = 0; i < n; i++) {
        scanf("%s %f %d %s", students[i].name, &students[i].gpa, &students[i].age,
students[i].major);
    }
}
```

```
char criterion[10];
```

```
scanf("%s", criterion);
```

```
quickSort(students, 0, n - 1, criterion);
```

```

printf("Sorted Student Records:\n");
printf("Name\t\tGPA\t\tAge\t\tMajor\n");
for (int i = 0; i < n; i++) {
    // Formatted according to sample specifications
    printf("%-10s\t%.2f\t%-3d\t\t%-15s\n", students[i].name, students[i].gpa,
students[i].age, students[i].major);
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Ragavi, a computer scientist, is working on efficient text processing for search engines. She has a list of words in random order and needs to sort them in lexicographic order using Heap Sort to improve search performance.

Help Ragavi implement Heap Sort to arrange the words in lexicographic order efficiently.

Input Format

The first line consists of an integer N denoting the number of strings.

The next line consists of N space-separated strings.

Output Format

The output displays N space-separated sorted strings in lexicographic order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10

banana pineapple mango chikoo jack star guava lemon tomato orange

Output: banana chikoo guava jack lemon mango orange pineapple star tomato

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
void swap(char arr[][100], int i, int j) {  
    char temp[100];  
    strcpy(temp, arr[i]);  
    strcpy(arr[i], arr[j]);  
    strcpy(arr[j], temp);  
}
```

```
void heapify(char arr[][100], int n, int i) {  
    int largest = i;  
    int left = 2 * i + 1;  
    int right = 2 * i + 2;
```

```
    // Use strcmp to compare strings lexicographically  
    if (left < n && strcmp(arr[left], arr[largest]) > 0)  
        largest = left;
```

```
    if (right < n && strcmp(arr[right], arr[largest]) > 0)  
        largest = right;
```

```
    if (largest != i) {  
        swap(arr, i, largest);  
        heapify(arr, n, largest);  
    }  
}
```

```
void heapSort(char arr[][100], int n) {  
    // Build max heap  
    for (int i = n / 2 - 1; i >= 0; i--)  
        heapify(arr, n, i);
```

```
    // Extract elements from heap one by one  
    for (int i = n - 1; i >= 0; i--) {  
        swap(arr, 0, i);  
        heapify(arr, i, 0);  
    }  
}
```

```

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    char words[20][100];
    for (int i = 0; i < n; i++) {
        scanf("%s", words[i]);
    }

    heapSort(words, n);

    for (int i = 0; i < n; i++) {
        printf("%s%c", words[i], (i == n - 1 ? '\n' : ' '));
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Imagine you are working as a librarian in a large library. The library uses an automated system to keep track of book positions on the shelves. Each book is assigned a unique ID, but some IDs are more common than others due to the popularity of certain editions.

The system logs the positions of books in a sorted list based on their IDs. You need to find the first and last positions of a specific book ID in this sorted list using recursive binary search.

Example

Input:

10

1 2 2 2 2 3 4 8 8 8

8

Output:

7 9

Explanation:

The first and last positions of book ID 8 are [7, 9] since the index is 0-based.

Input Format

The first line of input consists of an integer n, representing the total number of book IDs.

The second line consists of a sorted list of n integers representing the book IDs.

The third line consists of an integer x, representing the specific book ID to find.

Output Format

The output displays indexes of the first and last occurrence of x, as space-separated integers.

If book ID x is not present in the shelves, print "NO OCCURRENCES".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10

1 2 2 2 2 3 4 8 8 8

8

Output: 7 9

Answer

```
#include <stdio.h>
```

```
// Function to find the first occurrence using recursive binary search
```

```
int findFirst(int arr[], int low, int high, int x, int n) {
```

```
    if (high >= low) {
```

```
        int mid = low + (high - low) / 2;
```

```
        // Check if mid is the first occurrence
```

```

    if ((mid == 0 || x > arr[mid-1]) && arr[mid] == x)
        return mid;
    // If target is in the left half
    else if (x <= arr[mid])
        return findFirst(arr, low, (mid - 1), x, n);
    // If target is in the right half
    else
        return findFirst(arr, (mid + 1), high, x, n);
}
return -1;
}

```

// Function to find the last occurrence using recursive binary search

```

int findLast(int arr[], int low, int high, int x, int n) {
    if (high >= low) {
        int mid = low + (high - low) / 2;
        // Check if mid is the last occurrence
        if ((mid == n - 1 || x < arr[mid + 1]) && arr[mid] == x)
            return mid;
        // If target is in the right half
        else if (x >= arr[mid])
            return findLast(arr, (mid + 1), high, x, n);
        // If target is in the left half
        else
            return findLast(arr, low, (mid - 1), x, n);
    }
    return -1;
}

```

```

int main() {
    int n, x;
    if (scanf("%d", &n) != 1) return 0;
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    scanf("%d", &x);

```

```

    int first = findFirst(arr, 0, n - 1, x, n);

```

```

    // If first occurrence is -1, it means the element isn't there
    if (first == -1) {

```

```
        printf("NO OCCURRENCES\n");
    } else {
        int last = findLast(arr, 0, n - 1, x, n);
        printf("%d %d\n", first, last);
    }

    return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 6_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 44.5

Section 1 : Coding

1. Problem Statement

Virat is given a group of individuals, and he wants to identify if there is an influential individual within the group. An influential individual is defined as someone who is followed by more people than they follow.

He is provided with a list of follow-up relationships among the individuals in the form of a matrix. Your task is to help Virat determine if there is an influential individual and, if so, find their index within the group.

Write a program to solve this problem using Warshall's Algorithm.

Input Format

The first line of input consists of an integer N, representing the number of individuals in the group.

The following N lines will contain N space-separated integers, representing the following relationship matrix for the individuals.

Output Format

If an influential individual is found, print "Influential Individual: X," where X is the index (0-based) of the influential individual.

Otherwise, print "There is no influential individual in the group".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 1 1 0

0 0 1 0

0 0 0 0

0 0 1 0

Output: Influential Individual: 2

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
/**
```

```
* Problem: Identify an influential individual (followed > follows) using Warshall's Algorithm.
```

```
* Warshall's Algorithm computes the transitive closure (reachability) of a graph.
```

```
*
```

```
* Influential individual criteria:
```

```
* 1. Followed by more people than they follow (Followers > Following).
```

```
* 2. In matrix terms: Column_sum > Row_sum for a specific individual 'i'.
```

```
*/
```

```
int main() {
```

```
    int N;
```

```
    if (scanf("%d", &N) != 1) return 0;
```

```
    int matrix[N][N];
```

```

for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        scanf("%d", &matrix[i][j]);
    }
}

```

```

// Applying Warshall's Algorithm to find Transitive Closure
// transitive[i][j] = 1 if i follows j directly OR indirectly
int transitive[N][N];
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        transitive[i][j] = matrix[i][j];
    }
}

```

```

// Warshall's algorithm logic: k is intermediate vertex
for (int k = 0; k < N; k++) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (transitive[i][k] && transitive[k][j]) {
                transitive[i][j] = 1;
            }
        }
    }
}

```

```

// Calculate Following and Followers based on the transitive relationship
int influencer = -1;
for (int i = 0; i < N; i++) {
    int following = 0;
    int followers = 0;

```

```

    for (int j = 0; j < N; j++) {
        if (i == j) continue;

```

```

        // Number of people i influences/follows
        if (transitive[i][j]) following++;

```

```

        // Number of people who influence/follow i
        if (transitive[j][i]) followers++;
    }
}

```



```

// Definition: Followed by more people than they follow
if (followers > following) {
    influencer = i;
    break; // Assuming only one influencer or taking the first found
}
}

if (influencer != -1) {
    printf("Influential Individual: %d\n", influencer);
} else {
    printf("There is no influential individual in the group\n");
}

return 0;
}

```

Status : Partially correct

Marks : 4.5/10

2. Problem statement

Sarah has been tasked with developing a program to determine the shortest paths among nodes in a directed network represented as a graph containing nodes and weighted edges.

Her goal is to implement the Floyd-Warshall algorithm to compute the shortest path matrix for this directed network.

Input Format

The first line of input consists of an integer n , representing the number of nodes in the network.

The next n lines contain a space-separated integer, representing the adjacency matrix 'graph' of the network. Each integer is the weight of the edge from node i to node j .

Output Format

The output prints the shortest path matrix as ' $n \times n$ ' integers, where each element $\text{graph}[i][j]$ represents the shortest distance from node ' i ' to node ' j '.

Print each row of the matrix on a new line, with elements separated by a space,

and if no path exists, 999 will be displayed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 3 999 4

8 0 2 999

5 999 0 1

2 999 999 0

Output: 0 3 5 4

5 0 2 3

3 6 0 1

2 5 7 0

Answer

```
#include <stdio.h>
```

```
#define INF 999
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int graph[n][n];
```

```
    // Input adjacency matrix
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            scanf("%d", &graph[i][j]);
```

```
        }
```

```
    }
```

```
    // Floyd-Warshall algorithm
```

```
    for (int k = 0; k < n; k++) {
```

```
        for (int i = 0; i < n; i++) {
```

```
            for (int j = 0; j < n; j++) {
```

```
                if (graph[i][k] != INF && graph[k][j] != INF) {
```

```
                    if (graph[i][j] > graph[i][k] + graph[k][j]) {
```

```

graph[i][j] = graph[i][k] + graph[k][j];
    }
    }
}

// Print result
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d ", graph[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are working as a backend engineer for a drone delivery logistics company that operates across multiple cities. Each city is represented as a node, and the drone flight paths between cities are represented as directed edges with weights indicating the time (in minutes) it takes for a drone to travel between them.

Due to varying air traffic, regulations, and geography, the direct travel time between cities may not always be the fastest. Sometimes, going through intermediate cities results in shorter delivery times.

Your task is to build a system that calculates the shortest possible delivery time between every pair of cities, considering all possible intermediate routes.

To efficiently compute this, you're required to implement the Floyd-Warshall Algorithm.

Input Format

The first line of input contains an integer N representing the total number of cities.

The next N lines contains N space-separated integers, representing the delivery time matrix:

time[i][j] is the time (in minutes) taken to fly directly from City i to City j.

A value of 0 means the same city (i.e., $i = j$).

A value of 999 indicates no direct route between City i and City j.

Output Format

The output prints the shortest distance between every pair of cities.

If there is no shortest path between two cities, then print INF.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0

Output: 0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0

Answer

```
// You are using GCC
#include <stdio.h>
```

```
#define INF 999
```

```
int main() {
    int n;
    scanf("%d", &n);
```

```

int dist[n][n];

// Input matrix
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        scanf("%d", &dist[i][j]);
    }
}

// Floyd-Warshall Algorithm
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][k] != INF && dist[k][j] != INF) {
                if (dist[i][j] > dist[i][k] + dist[k][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}

// Output result
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (dist[i][j] == INF)
            printf("INF ");
        else
            printf("%d ", dist[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Given the weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack. In other words, given two integer arrays, $val[0..n-1]$ and $wt[0..n-1]$ which represent values and weights associated with n items, respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of $val[]$ such that the sum of the weights of this subset is smaller than or equal to W . You cannot break an item, either pick the complete item or don't pick it.

Solve this problem using dynamic programming.

Input Format

The first line of input represents the number of items.

The second line of input consists of the values of each item separated by spaces.

The third line of input consists of the weights of each item separated by spaces.

The fourth line of input consists of an integer, which represents the maximum capacity of the knapsack.

Output Format

The output prints the maximum value of items that can be held by the knapsack.

Sample Test Case

Input: 3
60 100 120
10 20 30
50

Output: 220

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

```

int main() {
    int n;
    scanf("%d", &n);

    int val[n], wt[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &val[i]);
    }

    for (int i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }

    int W;
    scanf("%d", &W);

    int dp[n + 1][W + 1];

    // Initialize DP table
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else
                dp[i][w] = 0;
        }
    }

    // Build DP table
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (wt[i - 1] <= w) {
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                                dp[i - 1][w]);
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    printf("%d", dp[n][W]);
}

```

```
return 0;
```

Status : Correct

Marks : 10/10

5. Problem Statement

Sarah loves challenges and has recently taken up a new programming problem. She is fascinated by the classic "0/1 Knapsack Problem" and wants to create a program that can efficiently solve it.

The 0/1 Knapsack Problem involves selecting a combination of items with given weights and values to maximize the total value, while not exceeding a given weight capacity.

Write a program that takes input for the 0/1 Knapsack Problem and outputs the maximum value that can be achieved, as well as the selected weights that contribute to this maximum value.

Note: The weights are printed in reverse order of the input, meaning the algorithm starts checking from the last item and prints items that are included first.

Input Format

The first line contains an integer n , representing the number of items available.

The second line contains n integers separated by space, representing the values of the items. The i th integer corresponds to the value of the i th item.

The third line contains n integers separated by space, representing the weights of the items. The i th integer corresponds to the weight of the i th item.

The fourth line contains an integer W , representing the maximum weight capacity of the knapsack.

Output Format

The first line output displays "Maximum value: " followed by an integer, representing the maximum value that can be achieved.

The second line output displays "Selected weights: " followed by space-separated integers, representing the selected weights contributing to the maximum value.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 3
60 100 120
10 20 30
50

Output: Maximum value: 220
Selected weights: 30 20

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

```
int main() {
    int n;
    scanf("%d", &n);

    int val[n], wt[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &val[i]);
    }

    for (int i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }

    int W;
    scanf("%d", &W);
```

```

int dp[n + 1][W + 1];

// DP table
for (int i = 0; i <= n; i++) {
    for (int w = 0; w <= W; w++) {
        if (i == 0 || w == 0)
            dp[i][w] = 0;
        else if (wt[i - 1] <= w)
            dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                           dp[i - 1][w]);
        else
            dp[i][w] = dp[i - 1][w];
    }
}

printf("Maximum value: %d\n", dp[n][W]);

// Backtracking to find selected weights (reverse order required)
printf("Selected weights: ");

int w = W;
for (int i = n; i > 0 && w > 0; i--) {
    if (dp[i][w] != dp[i - 1][w]) {
        printf("%d ", wt[i - 1]);
        w -= wt[i - 1];
    }
}

return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 6_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Ankit wants to share the job among his employees so that maximum profit can be obtained. The condition is that no two jobs for a particular employee overlap.

Given N jobs, where every job is represented by the following three elements:

Start Time

Finish Time

Profit or Value Associated (≥ 0)

Find the maximum-profit subset of jobs such that no two jobs in the subset overlap.

Input Format

The first line of the input consists of the value of n.

The next n lines of the inputs consist of the start, end, and profit.

Output Format

The output prints the optimal profit.

Refer to the sample input and output for format specifications.

Sample Test Case

Input: 4

3 10 20

1 2 50

6 19 100

2 100 200

Output: The optimal profit is 250

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int start, end, profit;  
} Job;
```

```
// Sort by finish time
```

```
int compare(const void *a, const void *b) {  
    Job *j1 = (Job *)a;  
    Job *j2 = (Job *)b;  
    return j1->end - j2->end;  
}
```

```
// Binary search to find last non-overlapping job
```

```
int binarySearch(Job jobs[], int i) {  
    int low = 0, high = i - 1;  
    int ans = -1;
```

```

while (low <= high) {
    int mid = (low + high) / 2;

    if (jobs[mid].end <= jobs[i].start) {
        ans = mid;
        low = mid + 1;
    } else {
        high = mid - 1;
    }
}

return ans;
}

int main() {
    int n;
    scanf("%d", &n);

    Job jobs[n];

    for (int i = 0; i < n; i++) {
        scanf("%d %d %d", &jobs[i].start, &jobs[i].end, &jobs[i].profit);
    }

    qsort(jobs, n, sizeof(Job), compare);

    int dp[n];
    dp[0] = jobs[0].profit;

    for (int i = 1; i < n; i++) {
        int includeProfit = jobs[i].profit;
        int l = binarySearch(jobs, i);

        if (l != -1) {
            includeProfit += dp[l];
        }

        dp[i] = (includeProfit > dp[i - 1]) ? includeProfit : dp[i - 1];
    }

    printf("The optimal profit is %d", dp[n - 1]);
}

```

```
} return 0;
```

Status : Correct

Marks : 10/10

2. Problem Statement

In a city, there are several locations connected by roads. Each road has a certain distance. You are tasked with finding the shortest distances between all pairs of locations in the city using the Floyd-Warshall algorithm.

Write a program that calculates the shortest path between all pairs of locations in the city.

Example

Input:

4

4

0 1 3

1 2 1

2 3 1

0 2 5

Output:

0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Explanation:

The city has 4 locations (0 to 3).

There are 4 roads connecting the locations.

The roads and their distances are:

0 1 with distance 3

1 2 with distance 1

2 3 with distance 1

0 2 with distance 5

Initially, the distance matrix is set to infinity (INF) except for direct roads and diagonal values (0).

The Floyd-Warshall algorithm updates the matrix by checking intermediate paths.

The shortest path from 0 to 2 (via 1) is updated to 4 (0 1 2) instead of 5.

The shortest path from 0 to 3 is 5 (0 1 2 3).

The final distance matrix is printed, showing the shortest distances.

0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Input Format

The first line contains an integer N, representing the number of locations in the city.

The second line contains an integer M, representing the number of roads in the city.

The next M lines each contain three space-separated integers: u, v, w where:

- u and v represent two locations connected by a road.
- w represents the distance between u and v.

Output Format

The output prints a matrix, where the value at the i-th row and j-th column represents the shortest distance between location i and location j.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

4

0 1 3

1 2 1

2 3 1

0 2 5

Output: 0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#define INF 1000000000
```

```
int main() {
```

```
    int n, m;
```

```
    scanf("%d", &n);
```

```
    scanf("%d", &m);
```

```
    int dist[n][n];
```

```
    // Initialize matrix
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            if (i == j)
```

```
                dist[i][j] = 0;
```

```
            else
```

```
                dist[i][j] = INF;
```

```
        }
```

```
    }
```

```
    // Read edges (treat as undirected since sample behaves like that)
```



```

for (int i = 0; i < m; i++) {
    int u, v, w;
    scanf("%d %d %d", &u, &v, &w);

    if (w < dist[u][v]) {
        dist[u][v] = w;
        dist[v][u] = w;
    }
}

// Floyd-Warshall
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][k] + dist[k][j] < dist[i][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}

// Print result
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d ", dist[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Sharon is developing a program that analyzes the reachability of nodes in a graph. Given an adjacency matrix representing the graph and its size, her goal is to determine whether there exists a path from one vertex to another for all pairs of vertices.

Write a program to help Sharon perform this reachability analysis using Warshall's Algorithm.

Input Format

The first line of input consists of an integer N, representing the number of vertices in the graph.

The following N lines consist of N space-separated integers, forming the adjacency matrix graph, where graph[i][j] is either 0 or 1.

Output Format

The output prints an NxN matrix representing the reachability matrix for all pairs of vertices.

Each element in the matrix should be 1 if there is a path from the corresponding row vertex to the column vertex, and 0 otherwise.

Sample Test Case

Input: 4

0 1 0 0

0 0 1 0

0 0 0 1

0 0 0 0

Output: 0 1 1 1

0 0 1 1

0 0 0 1

0 0 0 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int reach[n][n];
```

```
    // Input adjacency matrix
```

```
    for (int i = 0; i < n; i++) {
```

```

    for (int j = 0; j < n; j++) {
        scanf("%d", &reach[i][j]);
    }
}

// Warshall's Algorithm
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            reach[i][j] = reach[i][j] || (reach[i][k] && reach[k][j]);
        }
    }
}

// Print reachability matrix
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d ", reach[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

You and your group of adventurous friends are planning a once-in-a-lifetime outdoor adventure trip to an exotic location. To ensure that your adventure goes smoothly, you need to carefully choose what to carry with you, as you have a limited carrying capacity. Your goal is to maximize the overall value of the items you bring while staying within your weight limit.

Write a program that helps you and your friends determine the optimal selection of items to bring on your adventure trip. The program should implement the 0–1 Knapsack Problem algorithm to find the best combination of items to maximize their total value while respecting the weight constraint.

Input Format

The first line contains an integer n , representing the number of items available for selection.

The second line contains an integer $limit$, representing the maximum weight limit your group can carry in a bag.

The next n lines contain space-separated information for each item:

- The name of the item is a string (without spaces).
- The weight of the item.
- The value of the item.

Output Format

The first line of output displays "Total value: " followed by an integer representing the total value of the selected items.

The second line of output displays "Total weight: " followed by an integer representing the total weight of the selected items.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 2

25

Bag 10 250

Note 15 430

Output: Total value: 680

Total weight: 25

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

```

int main() {
    int n, limit;
    scanf("%d", &n);
    scanf("%d", &limit);

    int wt[n], val[n];
    char name[n][51]; // names are read but not needed in output

    for (int i = 0; i < n; i++) {
        scanf("%s %d %d", name[i], &wt[i], &val[i]);
    }

    int dp[n + 1][limit + 1];

    // DP table
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= limit; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i - 1] <= w)
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                               dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }

    int totalValue = dp[n][limit];

    // Backtracking to compute total weight of selected items
    int w = limit;
    int totalWeight = 0;

    for (int i = n; i > 0 && w > 0; i--) {
        if (dp[i][w] != dp[i - 1][w]) {
            totalWeight += wt[i - 1];
            w -= wt[i - 1];
        }
    }

    printf("Total value: %d\n", totalValue);
    printf("Total weight: %d", totalWeight);
}

```

```
} return 0;
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 6_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 13

Section 1 : MCQ

1. Which of the following is/are property/properties of a dynamic programming problem?

Answer

Both optimal substructure and overlapping subproblems

Status : Correct

Marks : 1/1

2. If an optimal solution can be created for a problem by constructing optimal solutions for its subproblems, the problem possesses _____ property.

Answer

Optimal substructure

Status : Correct

Marks : 1/1

3. In dynamic programming, the technique of storing the previously calculated values is called _____.

Answer

Memoization

Status : Correct

Marks : 1/1

4. The following paradigm can be used to find the solution to the problem in minimum time:

Given a set of non-negative integers, and a value K, determine if there is a subset of the given set with a sum equal to K.

Answer

Dynamic Programming

Status : Correct

Marks : 1/1

5. Which of the following is an example of a problem that can be solved using memoization in dynamic programming?

Answer

Computing the edit distance between two strings

Status : Correct

Marks : 1/1

6. The Floyd-Warshall algorithm for all-pairs shortest paths computation is based on

Answer

Dynamic Programming paradigm

Status : Correct

Marks : 1/1

7. Floyd-Warshall algorithm utilizes _____ to solve the all-pairs shortest paths problem on a directed graph in _____ time.

Answer

Dynamic programming, $\theta(V^3)$

Status : Correct

Marks : 1/1

8. Floyd Warshall's Algorithm can be applied to _____.

Answer

Directed graphs

Status : Correct

Marks : 1/1

9. In the Floyd Warshall algorithm, how many iterations are required to find the shortest path between all pairs of vertices in a graph with N vertices?

Answer

N

Status : Correct

Marks : 1/1

10. Consider a directed weighted graph with V vertices and E edges, where each edge weight is represented as a 32-bit signed integer. What will be the maximum possible value for the edge weight to guarantee the correct shortest path calculation using the Floyd Warshall algorithm?

Answer

$2^{31} - 1$

Status : Correct

Marks : 1/1

11. 0/1 knapsack is based on _____ method.

Answer

Dynamic programming

Status : Correct

Marks : 1/1

12. Fractional knapsack problem differs from 0/1 knapsack because:

Answer

Only weights are considered

Status : Wrong

Marks : 0/1

13. Which of the following methods can be used to solve the Knapsack problem?

Answer

Dynamic programming

Status : Wrong

Marks : 0/1

14. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

Answer

It systematically explores possible solutions while pruning non-promising ones.

Status : Correct

Marks : 1/1

15. Which of the following problems cannot be solved optimally by greedy approach?

Answer

0/1 Knapsack

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 5_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Suppose you are working on a project that involves sorting a large dataset of student records based on a single criterion, such as their CGPA, age, or major. You have decided to use a modified quicksort algorithm to efficiently sort the dataset while ensuring that the sorting is stable, i.e., that records with equal values maintain their original relative order.

To achieve this, you will implement a modified version of the quicksort algorithm that maintains stability during the sorting process. Specifically, you will use a three-way partitioning scheme that sorts the records into three groups: those with values less than the pivot, those with values equal to the pivot, and those with values greater than the pivot. When the algorithm processes subarrays with equal pivot values, it uses insertion sort to maintain stability.

Write a function that implements this modified version of the quicksort algorithm and uses it to sort the student records according to one specified criterion (CGPA, age, or major). Your function should take as input a list of student records and return the sorted list in ascending order based on the chosen criterion. If two records have the same values for the sorting criterion, their relative order should be maintained in the sorted list.

Input Format

The first line contains an integer n , representing the number of student records to be sorted.

The next n groups of 4 lines each contain the details of each student:

- Line 1: Name (string)
- Line 2: CGPA (float)
- Line 3: Age (integer)
- Line 4: Major (string)

The last line contains the sorting criteria (one of three options: "CGPA", "age", or "major").

Output Format

The output displays the sorted student records in a table format with the following specifications:

Header row: "Name CGPA Age Major"

Each student record is displayed in one row with the following format:

- Name: Left-aligned, 10 characters wide
- CGPA: Left-aligned, 8 characters wide, displayed with 2 decimal places
- Age: Left-aligned, 5 characters wide
- Major: Left-aligned, 20 characters wide

The table is sorted in ascending order according to the chosen sorting criteria.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 4

harish
7.8
23
ece
trello
8
21
cse
kamalesh
7
25
it
maaran
5.6
20
civil
age

Output: Name CGPA Age Major
maaran 5.60 20 civil
trello 8.00 21 cse
harish 7.80 23 ece
kamalesh 7.00 25 it

Answer

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```
// Structure to hold student records
```

```
typedef struct {
    char name[55];
    float cgpa;
    int age;
    char major[55];
} Student;
```

```
// Function to compare students based on the criteria
```

```
int compare(Student s1, Student s2, char* criteria) {
    if (strcmp(criteria, "CGPA") == 0) {
        if (s1.cgpa < s2.cgpa) return -1;
        if (s1.cgpa > s2.cgpa) return 1;
        return 0;
    }
}
```

```

    } else if (strcmp(criteria, "age") == 0) {
        if (s1.age < s2.age) return -1;
        if (s1.age > s2.age) return 1;
        return 0;
    } else if (strcmp(criteria, "major") == 0) {
        return strcmp(s1.major, s2.major);
    }
    return 0;
}

```

// Stable insertion sort for small subarrays

```

void insertionSort(Student arr[], int low, int high, char* criteria) {
    for (int i = low + 1; i <= high; i++) {
        Student key = arr[i];
        int j = i - 1;
        while (j >= low && compare(arr[j], key, criteria) > 0) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

```

// Stable 3-way partition QuickSort

```

void stableThreeWayQuickSort(Student arr[], int low, int high, char* criteria) {
    if (low < high) {
        // Switch to insertion sort for small subarrays to ensure stability and
        // efficiency
        if (high - low < 10) {
            insertionSort(arr, low, high, criteria);
            return;
        }
    }
}

```

// Using middle element as pivot

Student pivot = arr[low + (high - low) / 2];

// Temporary arrays for 3-way partition

int n = high - low + 1;

Student* temp = (Student*)malloc(n * sizeof(Student));

int less = 0, equal = 0, greater = 0;

// Count elements

```

    for (int i = low; i <= high; i++) {
        int comp = compare(arr[i], pivot, criteria);
        if (comp < 0) less++;
        else if (comp == 0) equal++;
        else greater++;
    }

    // Fill temporary array: less, then equal, then greater
    int l_idx = 0;
    int e_idx = less;
    int g_idx = less + equal;

    for (int i = low; i <= high; i++) {
        int comp = compare(arr[i], pivot, criteria);
        if (comp < 0) temp[l_idx++] = arr[i];
        else if (comp == 0) temp[e_idx++] = arr[i];
        else temp[g_idx++] = arr[i];
    }

    // Copy back to original array
    for (int i = 0; i < n; i++) {
        arr[low + i] = temp[i];
    }
    free(temp);

    // Recursively sort less and greater parts
    stableThreeWayQuickSort(arr, low, low + less - 1, criteria);
    stableThreeWayQuickSort(arr, low + less + equal, high, criteria);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    Student* students = (Student*)malloc(n * sizeof(Student));
    for (int i = 0; i < n; i++) {
        scanf("%s %f %d %s", students[i].name, &students[i].cgpa, &students[i].age,
students[i].major);
    }

    char criteria[20];

```

```

scanf("%s", criteria);

// Sort using modified quicksort
stableThreeWayQuickSort(students, 0, n - 1, criteria);

// Print Header
printf("Name CGPA Age Major\n");
// Print Sorted Records
for (int i = 0; i < n; i++) {
    printf("%-10s %-8.2f %-5d %-20s\n", students[i].name, students[i].cgpa,
students[i].age, students[i].major);
}

free(students);
return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Vishnu, a math enthusiast, is given a task to explore the magic of numbers. He has an array of positive integers, and his goal is to find the integer with the highest digit sum in the sorted array using the merge sort algorithm.

You have to assist Vishnu in implementing the merge sort algorithm.

Example

Input:

6

789 123 456 876 234 567

Output:

The sorted array is: 123 234 456 567 789 876

The integer with the highest digit sum is: 789

Explanation

Sorted Array Output: 123 234 456 567 789 876

The digit sum of an integer is the sum of its individual digits.

In this case, the integer 789 has the highest digit sum ($7 + 8 + 9 = 24$).

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line consists of N space-separated integers, representing the array elements.

Output Format

The first line prints "The sorted array is: " followed by the sorted array in ascending order.

The second line prints "The integer with the highest digit sum is: " followed by the integer with the highest-digit sum.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

123 456 789 321 654

Output: The sorted array is: 123 321 456 654 789

The integer with the highest digit sum is: 789

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Function to calculate the sum of digits of a number
```

```
int getDigitSum(int n) {
```

```
    int sum = 0;
```

```
    while (n > 0) {
```

```

    sum += n % 10;
    n /= 10;
}
return sum;
}

```

// Function to merge two subarrays

```

void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int *L = (int *)malloc(n1 * sizeof(int));
    int *R = (int *)malloc(n2 * sizeof(int));

    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

```

```

    i = 0; j = 0; k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

```

```

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

```

```

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
    free(L);

```

```

    free(R);
}

// Function that implements Merge Sort
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

```

```

int main() {
    int n, i;
    if (scanf("%d", &n) != 1) return 0;
    int arr[n];
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
}

```

```

// Sort the array
mergeSort(arr, 0, n - 1);

```

```

// Print sorted array
printf("The sorted array is: ");
for (i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\n");

```

```

// Find integer with highest digit sum
int maxSum = -1;
int maxNum = -1;
for (i = 0; i < n; i++) {
    int currentSum = getDigitSum(arr[i]);
    if (currentSum > maxSum) {
        maxSum = currentSum;
        maxNum = arr[i];
    }
}

```

```
printf("The integer with the highest digit sum is: %d\n", maxNum);  
return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Pavi is working on a project to help a library manage its collection of books efficiently. As part of this project, you need to develop a program to assist the library staff in finding specific books in their extensive catalog.

The library's book collection is organized in such a way that each book is assigned a unique numerical identifier. The library staff has provided you with a list of these identifiers, and you need to write a program to help them find a book's identifier efficiently.

Therefore, you decide to implement a recursive binary search algorithm to quickly locate books based on their unique identifiers.

Input Format

The first line of input contains an integer, n , representing the number of books in the library's collection.

The second line of input consists of n space-separated integers, where each integer represents the unique identifier of a book in the library's collection.

The third line of input contains a single integer t , representing the unique identifier of the book the library staff is searching for.

Output Format

If the book with the identifier t is found in the library's collection, the output prints the index (0-based) of the book in the collection.

If the book is not found, print " t not present", where the t is the unique identifier of a book.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
12 13 14 15 16
14

Output: 2

Answer

```
// You are using GCC
#include <stdio.h>

// Recursive function to perform binary search
int binarySearch(int arr[], int low, int high, int target) {
    if (low <= high) {
        int mid = low + (high - low) / 2;

        // If found, return index
        if (arr[mid] == target)
            return mid;

        // If smaller, look in left subarray
        if (arr[mid] > target)
            return binarySearch(arr, low, mid - 1, target);

        // Otherwise look in right subarray
        return binarySearch(arr, mid + 1, high, target);
    }
    // Target not found
    return -1;
}

int main() {
    int n, target;

    // Read the number of books
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    // Read the unique identifiers
```

```

for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}

// Read the target identifier
scanf("%d", &target);

// Perform binary search
int result = binarySearch(arr, 0, n - 1, target);

// Print the result
if (result != -1) {
    printf("%d\n", result);
} else {
    printf("%d not present\n", target);
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Jack is building a number analyzer tool that identifies numbers based on the sum of their digits. He is given a list of integers and wants to find the smallest number in the list whose digit sum equals a target value d .

To do this efficiently, Jack first sorts the list based on each number's digit sum and then applies binary search to find a match. Write a program to ensure that Jack can determine the correct number using this approach. If no number in the list matches the digit sum d , print -1.

Input Format

The first line of input consists of an integer n , representing the number of elements in the list.

The second line contains n space-separated integers, representing the sorted list.

The third line contains an integer d, representing the target digit sum.

Output Format

The output prints a single integer representing the smallest number from the list whose digit sum equals d, based on sorted digit sum order.

If no such number exists, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

12 23 30 48 56

3

Output: 12

Answer

// You are using GCC

#include <stdio.h>

// Function to calculate the sum of digits of a number

int getDigitSum(int num) {

int sum = 0;

while (num > 0) {

sum += num % 10;

num /= 10;

}

return sum;

}

int main() {

int n, d;

int arr[20];

// Read the number of elements

if (scanf("%d", &n) != 1) return 0;

// Read the list of integers

for (int i = 0; i < n; i++) {

```

scanf("%d", &arr[i]);
}

// Read the target digit sum
scanf("%d", &d);

// Bubble sort: Sort by digit sum (ascending),
// then by number value (ascending)
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        int sum1 = getDigitSum(arr[j]);
        int sum2 = getDigitSum(arr[j+1]);

        if (sum1 > sum2 || (sum1 == sum2 && arr[j] > arr[j+1])) {
            // Swap arr[j] and arr[j+1]
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}

// Find the smallest number with the target digit sum d
int smallestNum = -1;
for (int i = 0; i < n; i++) {
    if (getDigitSum(arr[i]) == d) {
        smallestNum = arr[i];
        break; // Found the smallest due to sorting
    }
}

// Output the result
printf("%d\n", smallestNum);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Riya, an HR manager, needs a quick way to identify the Kth highest salary among employees for bonus distribution. To efficiently retrieve this information, she wants to use the max-heap technique.

Help Riya implement a solution that finds the Kth highest salary from the given list of salaries.

Input Format

The first line of input consists of an integer N, representing the number of employees.

The second line consists of N space-separated integers, representing the salaries of N employees.

The third line consists of an integer K, indicating the rank of the desired Kth highest salary.

Output Format

The output prints: <k>th largest Salary: <result>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
30000 25000 40000 35000 20000 28000 29500
5

Output: 5th largest Salary: 28000

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
```

```
// Function to swap two salaries
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
```

```

    *b = temp;
}

// Max-heapify function to maintain heap property
void maxHeapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        maxHeapify(arr, n, largest);
    }
}

// Function to build a max-heap
void buildMaxHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        maxHeapify(arr, n, i);
}

// Function to extract the max element from the heap
int extractMax(int arr[], int *n) {
    if (*n <= 0) return -1;
    int root = arr[0];
    arr[0] = arr[*n - 1];
    (*n)--;
    maxHeapify(arr, *n, 0);
    return root;
}

int main() {
    int N, K;

    // Read number of employees
    if (scanf("%d", &N) != 1) return 0;

```

```
int *salaries = (int *)malloc(N * sizeof(int));

// Read salaries
for (int i = 0; i < N; i++) {
    scanf("%d", &salaries[i]);
}

// Read rank K
scanf("%d", &K);

// Step 1: Build max heap
buildMaxHeap(salaries, N);

// Step 2: Extract max k-1 times
int kthHighest = 0;
for (int i = 0; i < K; i++) {
    kthHighest = extractMax(salaries, &N);
}

// Output the result
printf("%dth largest Salary: %d\n", K, kthHighest);

free(salaries);
return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 5_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an array of integers, count the number of smaller elements that appear to the right of each element. Implement a solution using merge sort to count these elements efficiently. Output the result as an array of counts corresponding to each element in the original array.

Example

Input:

4

5 2 6 1

Output:

2 1 1 0

Explanation:

To the right of 5, there are 2 smaller elements (2 and 1).

To the right of 2, there is only 1 smaller element (1).

To the right of 6, there is 1 smaller element (1).

To the right of 1, there is 0 smaller element.

Input Format

The first line of input contains a single integer n, representing the length of the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints a single line containing the counts of smaller elements for each value in the array, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

5 2 6 1

Output: 2 1 1 0

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int val;  
    int index;  
} Element;
```

```
void merge(Element* arr, int left, int mid, int right, int* count) {  
    int i = left;
```

```

int j = mid + 1;
int k = 0;
int right_smaller_count = 0;
Element* temp = (Element*)malloc((right - left + 1) * sizeof(Element));

while (i <= mid && j <= right) {
    if (arr[j].val < arr[i].val) {
        // Element in right subarray is smaller than element in left subarray
        right_smaller_count++;
        temp[k++] = arr[j++];
    } else {
        // Element in left subarray is smaller or equal
        // Add the current count of smaller elements seen from the right subarray
        count[arr[i].index] += right_smaller_count;
        temp[k++] = arr[i++];
    }
}

// Process remaining elements in left subarray
while (i <= mid) {
    count[arr[i].index] += right_smaller_count;
    temp[k++] = arr[i++];
}

// Process remaining elements in right subarray
while (j <= right) {
    temp[k++] = arr[j++];
}

// Copy sorted elements back to original array
for (i = 0; i < k; i++) {
    arr[left + i] = temp[i];
}
free(temp);
}

```

```

void mergeSort(Element* arr, int left, int right, int* count) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid, count);
        mergeSort(arr, mid + 1, right, count);
        merge(arr, left, mid, right, count);
    }
}

```

```

    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    Element* arr = (Element*)malloc(n * sizeof(Element));
    int* count = (int*)calloc(n, sizeof(int));

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i].val);
        arr[i].index = i;
    }

    mergeSort(arr, 0, n - 1, count);

    for (int i = 0; i < n; i++) {
        printf("%d%c", count[i], (i == n - 1 ? '\n' : ' '));
    }

    free(arr);
    free(count);
    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Ravi, a mathematics teacher, is helping his students understand quadratic equations. He wants to apply the quadratic formula $[A * x^2 + B * x + C]$ to each number in a sorted array. Once the formula is applied, the modified numbers need to be sorted again. Ravi plans to use QuickSort for efficient sorting. Help Ravi by implementing a program that performs these operations.

Input Format

The first line contains an integer n , representing the size of the array.

The second line contains n space-separated integers, representing the elements of the array in ascending order.

The third line contains three integers A, B, and C, the coefficients of the quadratic formula.

Output Format

The first line of output prints: Array after applying the equation and sorting

The subsequent lines print: <sorted_value>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

-1 0 1 2 3 4

-1 2 -1

Output: Array after applying the equation and sorting:

-9 -4 -4 -1 -1 0

Answer

```
#include <stdio.h>
```

```
// Function to swap two elements
void swap(long long* a, long long* b) {
    long long t = *a;
    *a = *b;
    *b = t;
}
```

```
// Partition function for QuickSort
int partition(long long arr[], int low, int high) {
    long long pivot = arr[high];
    int i = (low - 1);
```

```
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
```



```
}  
}  
}  
}
```

```
swap(&arr[i + 1], &arr[high]);  
return (i + 1);  
}
```

```
// QuickSort function
```

```
void quickSort(long long arr[], int low, int high) {  
    if (low < high) {  
        int pi = partition(arr, low, high);  
        quickSort(arr, low, pi - 1);  
        quickSort(arr, pi + 1, high);  
    }  
}
```

```
int main() {  
    int n;  
    if (scanf("%d", &n) != 1) return 0;
```

```
    long long arr[n];  
    for (int i = 0; i < n; i++) {  
        scanf("%lld", &arr[i]);  
    }
```

```
    long long A, B, C;  
    scanf("%lld %lld %lld", &A, &B, &C);
```

```
    // Apply the quadratic formula:  $A \cdot x^2 + B \cdot x + C$   
    for (int i = 0; i < n; i++) {  
        long long x = arr[i];  
        arr[i] = A * x * x + B * x + C;  
    }
```

```
    // Sort the modified array using QuickSort  
    quickSort(arr, 0, n - 1);
```

```
    // Print the output in the required format  
    printf("Array after applying the equation and sorting:\n");  
    for (int i = 0; i < n; i++) {  
        printf("%lld ", arr[i]);  
    }
```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Amit, an e-commerce manager, is working on an inventory management system to track the top-selling products. Given the sales data of various products, he wants to quickly determine the Kth best-selling product to make informed restocking decisions.

Help Amit implement Heap Sort to efficiently find the Kth best-selling product from the given sales data.

Input Format

The first line of input consists of an integer n, representing the number of products.

The second line consists of n space-separated integers, representing the sales data of each product.

The third line contains an integer K, representing the rank of the product to retrieve.

Output Format

The output prints the Kth best-selling product based on the given sales data.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 15 5 25
2
Output: 20

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
// Function to swap two elements
void swap(long long *a, long long *b) {
    long long temp = *a;
    *a = *b;
    *b = temp;
}
```

```
// Function to maintain the max-heap property
void heapify(long long arr[], int n, int i) {
```

```
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
```

```
    if (left < n && arr[left] > arr[largest])
        largest = left;
```

```
    if (right < n && arr[right] > arr[largest])
        largest = right;
```

```
    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}
```

```
// Main function to perform heap sort
```

```
void heapSort(long long arr[], int n) {
```

```
    // Build heap (rearrange array)
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
```

```
    // One by one extract an element from heap
```

```
    for (int i = n - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]); // Move current root to end
        heapify(arr, i, 0); // call max heapify on the reduced heap
    }
}
```

```

int main() {
    int n, k;
    if (scanf("%d", &n) != 1) return 0;

    // Using long long to accommodate sales data up to 10^9
    long long *sales = (long long *)malloc(n * sizeof(long long));
    for (int i = 0; i < n; i++) {
        scanf("%lld", &sales[i]);
    }

    if (scanf("%d", &k) != 1) return 0;

    // Perform Heap Sort
    heapSort(sales, n);

    // After ascending heap sort, Kth largest is at n - k
    printf("%lld\n", sales[n - k]);

    free(sales);
    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Tom is working in a warehouse inventory system, where the item IDs are assigned sequentially in ascending order.

He wants to develop a program using recursive binary search to efficiently determine the closest item ID that is less than or equal to a given target ID.

The program takes the total number of items and their sorted IDs as input, assisting warehouse staff in quickly identifying the closest available item less than or equal to the target ID. Help Tom in this program.

Input Format

The first line of input consists of an integer N, representing the number of items in the warehouse.

The second line consists of N space-separated integers, representing the sorted list of item IDs.

The third line consists of an integer representing the target item ID.

Output Format

The output prints "The closest item ID less than or equal to X is Y", where X is the target ID and Y is the closest item ID less than or equal to X.

If no such element exists, print "No closest item with an ID less than or equal to X exists in the warehouse", where X is the target ID.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

17 25 38 47 51 62 79

50

Output: The closest item ID less than or equal to 50 is 47

Answer

```
#include <stdio.h>
```

```
// Recursive function to find the floor of the target ID
```

```
int findFloor(int arr[], int low, int high, int target) {
```

```
    if (low > high) {
```

```
        return -1;
```

```
    }
```

```
// If target is smaller than the smallest element
```

```
if (target < arr[low]) {
```

```
    return -1;
```

```
}
```

```
// If target is greater than or equal to the largest element in the current range
```

```
if (target >= arr[high]) {
```

```
    return arr[high];
```

```
}
```

```

int mid = low + (high - low) / 2;

if (arr[mid] == target) {
    return arr[mid];
}

// If target is greater than mid, floor is in the right half (including mid)
if (arr[mid] < target) {
    int res = findFloor(arr, mid + 1, high, target);
    return (res != -1) ? res : arr[mid];
}

// If target is smaller than mid, floor must be in the left half
return findFloor(arr, low, mid - 1, target);
}

int main() {
    int n, target;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &target);

    int result = findFloor(arr, 0, n - 1, target);

    if (result != -1) {
        printf("The closest item ID less than or equal to %d is %d", target, result);
    } else {
        printf("No closest item with an ID less than or equal to %d exists in the warehouse", target);
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Rahul owns a bookstore and maintains a sorted list of ISBN numbers of books in his inventory. When a customer asks for a specific book, Rahul wants to quickly check whether the book is available in his inventory.

Write a program that implements a non-recursive binary search to determine if a given ISBN exists in the sorted list of ISBN numbers.

Input Format

The first line contains an integer N, representing the number of ISBN numbers in the inventory.

The second line contains N space-separated integers representing the sorted list of ISBN numbers.

The third line contains a single integer X, the ISBN number to be searched.

Output Format

If the ISBN X is found in the inventory, print: "Book with ISBN X is available at index Y" where Y is the zero-based index of the ISBN in the list.

If the ISBN X is not found, print: "Book with ISBN X is not available in the inventory"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
2 3 40 50 20
3

Output: Book with ISBN 3 is available at index 1

Answer

```
// You are using GCC
#include <stdio.h>
```

```

int main() {
    int n, i, x, left, right, mid;
    int found = 0;

    // Read number of books
    if (scanf("%d", &n) != 1) return 0;

    int isbn[n];
    // Read sorted ISBN numbers
    for (i = 0; i < n; i++) {
        scanf("%d", &isbn[i]);
    }

    // Read target ISBN
    scanf("%d", &x);

    // Non-recursive Binary Search
    left = 0;
    right = n - 1;

    while (left <= right) {
        mid = left + (right - left) / 2;

        if (isbn[mid] == x) {
            printf("Book with ISBN %d is available at index %d\n", x, mid);
            found = 1;
            break;
        }
        if (isbn[mid] < x) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    if (!found) {
        printf("Book with ISBN %d is not available in the inventory\n", x);
    }

    return 0;
}

```


Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 5_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 15

Section 1 : MCQ

1. In heap sort, when does the actual sorting take place?

Answer

During the swap and heapify phase

Status : Correct

Marks : 1/1

2. In heap sort, which of the following heap operations is performed to build a valid heap from an unsorted array?

Answer

Heapify

Status : Correct

Marks : 1/1

3. In heap sort, the element that is swapped with the final element of the heap is _____, in the case of a max-heap, sorted in descending order.

Answer

Largest element

Status : Correct

Marks : 1/1

4. What is the worst-case time complexity of the heap sort algorithm?

Answer

$O(n \log n)$

Status : Correct

Marks : 1/1

5. In a quick sort algorithm, where are the smaller elements placed to the pivot during the partition process, assuming we are sorting in increasing order?

Answer

To the left of the pivot

Status : Correct

Marks : 1/1

6. Which of the following is true about Quicksort?

Answer

It is an in-place sorting algorithm

Status : Correct

Marks : 1/1

7. What is the worst-case time complexity of the Quicksort algorithm?

Answer

$O(n^2)$

Status : Correct

Marks : 1/1

8. Given a sorted array of 1000 unique integers, how many maximum comparisons does Binary Search make in the worst case?

Answer

10

Status : Correct

Marks : 1/1

9. Suppose you are using Binary Search on a sorted array of even length (e.g., 8 elements). At each step, you calculate the middle as:

$\text{mid} = (\text{low} + \text{high}) // 2$

Which of the following statements is always true?

Answer

The mid index may not divide the array into equal halves in all steps.

Status : Correct

Marks : 1/1

10. You apply Binary Search on the sorted array [3, 7, 10, 15, 21, 30] to find the element 13. Which sequence of middle elements will Binary Search compare with before declaring that the element is not present?

Answer

10 21 15

Status : Correct

Marks : 1/1

11. You are using Binary Search to find the first occurrence of a value x in a sorted array that may contain duplicates. Which of the following modifications is necessary?

Answer

When $x == \text{arr}[\text{mid}]$, move high to mid - 1.

Status : Correct

Marks : 1/1

12. A student implements Binary Search and calculates the middle index as:

$\text{mid} = (\text{low} + \text{high}) // 2$

But when the input values of low and high are both large (e.g., low = 2_000_000_000, high = 2_000_000_010), the program crashes due to integer overflow. What is the best fix?

Answer

Use $\text{mid} = \text{low} + (\text{high} - \text{low}) // 2$

Status : Correct

Marks : 1/1

13. Is Merge Sort a stable sorting algorithm?

Answer

Yes, always stable.

Status : Correct

Marks : 1/1

14. Merge sort is _____.

Answer

Comparison-based sorting algorithm

Status : Correct

Marks : 1/1

15. Which of the following methods is used for sorting in merge sort?

Answer

merging

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 6_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Anitha is working on packing expensive artifacts into a container that has a limited weight capacity. Each artifact has a weight and a value, and Anitha wants to maximize the total worth of artifacts she can carry, even if it means taking only a fraction of an artifact when the container is nearly full.

The maximum worth is calculated as:

Maximum Worth = $\sum (\text{value}[i] * \text{fraction_taken}[i])$,

where $\text{fraction_taken}[i] = \min(1, \text{remaining_capacity} / \text{weight}[i])$

Write a program to ensure that Anitha can calculate the maximum worth of objects she can carry in the container.

Input Format

The first line contains an integer N, representing the number of items.

The second line contains N space-separated integers representing the weights of the items.

The third line contains N space-separated integers representing the values of the items.

The fourth line contains an integer representing the capacity of the container.

Output Format

The output prints a single line showing the total maximum worth of objects carried as a double with exactly two decimal places, in the following format:

"Objects worth Rs.X.YY"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 20 30
60 100 120
50

Output: Objects worth Rs.240.00

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
```

```
// Structure to represent an item
struct Item {
    int weight;
    int value;
```

```

    double ratio;
}

// Function to compare items based on their value-to-weight ratio
int compare(const void *a, const void *b) {
    struct Item *i1 = (struct Item *)a;
    struct Item *i2 = (struct Item *)b;
    if (i2->ratio > i1->ratio) return 1;
    if (i2->ratio < i1->ratio) return -1;
    return 0;
}

int main() {
    int N;
    if (scanf("%d", &N) != 1) return 0;

    struct Item items[N];

    // Read weights
    for (int i = 0; i < N; i++) {
        scanf("%d", &items[i].weight);
    }

    // Read values
    for (int i = 0; i < N; i++) {
        scanf("%d", &items[i].value);
        items[i].ratio = (double)items[i].value / items[i].weight;
    }

    int capacity;
    scanf("%d", &capacity);

    // Sort items by ratio in descending order
    qsort(items, N, sizeof(struct Item), compare);

    double maxWorth = 0.0;
    int remainingCapacity = capacity;

    for (int i = 0; i < N; i++) {
        if (remainingCapacity == 0) break;

        // Take full item

```



```

    if (items[i].weight <= remainingCapacity) {
        maxWorth += items[i].value;
        remainingCapacity -= items[i].weight;
    }
    // Take fraction of item
    else {
        maxWorth += (double)items[i].ratio * remainingCapacity;
        remainingCapacity = 0;
    }
}

printf("Objects worth Rs.%.2f\n", maxWorth);

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Raju is a professional hiker preparing for a long trekking expedition. He has a backpack that can hold a maximum weight of W kilograms. There are N essential items you want to carry, each with a specific weight and importance value (utility score). His goal is to maximize the total importance value of the items you take while ensuring that the total weight does not exceed the backpack's capacity.

He can either include or exclude each item—meaning he cannot take fractions of an item (e.g., you cannot take half of a tent). Given the weights and importance values of the items, determine the maximum possible importance value he can carry in his backpack. Help him to achieve his task.

Input Format

The first line contains two integers, N (number of items) and W (maximum weight the backpack can carry).

The second line contains N integers, representing the weights of the items.

The third line contains N integers, representing the importance values of the items.

Output Format

The output prints a single integer, the maximum importance value that can be achieved without exceeding the weight limit.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 4 5

2 3 4 5

3 4 5 6

Output: 7

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
// Function to find the maximum of two integers
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
// Function to solve the 0/1 Knapsack Problem
```

```
int knapSack(int W, int wt[], int val[], int n) {  
    int i, w;  
    // Create a 2D array for storing DP results  
    // dp[n+1][W+1] to accommodate N+1 items and capacity W+1  
    int dp[n + 1][W + 1];
```

```
    // Build table dp[][] in bottom-up manner
```

```
    for (i = 0; i <= n; i++) {  
        for (w = 0; w <= W; w++) {  
            // Base Case: If no items or capacity is 0  
            if (i == 0 || w == 0) {  
                dp[i][w] = 0;  
            }
```

```
            // If weight of the current item is less than or equal to capacity 'w'
```

```

    else if (wt[i - 1] <= w) {
        // Max of (including the current item, excluding the current item)
        dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]);
    }
    // If weight of the current item is more than capacity 'w'
    else {
        dp[i][w] = dp[i - 1][w];
    }
}
}

return dp[n][W];
}

int main() {
    int N, W;

    // Read the number of items and maximum capacity
    if (scanf("%d %d", &N, &W) != 2) return 0;

    int weights[N];
    int values[N];

    // Read the weights of the items
    for (int i = 0; i < N; i++) {
        scanf("%d", &weights[i]);
    }

    // Read the importance values of the items
    for (int i = 0; i < N; i++) {
        scanf("%d", &values[i]);
    }

    // Call the function and print the result
    printf("%d\n", knapSack(W, weights, values, N));

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rohit is responsible for scheduling flights for an airline company. He has a list of N flights numbered from 0 to $N-1$. Some flights have specific constraints that require them to depart after other flights.

Write a program to help Rohit determine if it's possible to create a flight schedule that adheres to these constraints using Warshall's algorithm.

Example 1

Input:

5

0 1

1 2

2 3

3 4

4 5

Output:

Yes

Explanation:

Flight 0 must depart after Flight 1.

Flight 1 must depart after Flight 2.

Flight 2 must depart after Flight 3.

Flight 3 must depart after Flight 4.

Flight 4 must depart after Flight 5.

There are no circular dependencies or conflicts in the schedule, so it is possible to create a flight schedule that adheres to the constraints.

Example 2

Input:

6

0 1

1 2

2 3

3 4

4 5

5 0

Output:

No

Explanation:

Flight 0 must depart after Flight 1.

Flight 1 must depart after Flight 2.

Flight 2 must depart after Flight 3.

Flight 3 must depart after Flight 4.

Flight 4 must depart after Flight 5.

Flight 5 must depart after Flight 0.

There is a circular dependency in the flight constraints. Therefore, it is not possible to create a flight schedule that follows the constraints.

Input Format

The first line of input consists of an integer N, representing the number of flights.

The following N lines consist of two space-separated integers each: the flight number and the flight number that must depart before it.

Output Format

If it is possible to create a flight schedule that adheres to the constraints, print "Yes".

Otherwise, print "No".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

0 1

1 2

2 3

3 4

4 5

Output: Yes

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
int main() {
```

```
    int N;
```

```
    if (scanf("%d", &N) != 1) return 0;
```

```
    int graph[10][10] = {0}; // 1 <= N <= 10, using 0-9
```

```
    int u, v;
```

```
    // Read constraints: 'u' must depart AFTER 'v' (v -> u)
```

```
    for (int i = 0; i < N; i++) {
```

```
        if (scanf("%d %d", &u, &v) == 2) {
```

```
            if (u < 10 && v < 10) {
```

```
                graph[v][u] = 1;
```

```
            }
```

```
        }
```

```
    }
```

```
    // Warshall's Algorithm to find transitive closure
```

```
    for (int k = 0; k < N; k++) {
```

```
        for (int i = 0; i < N; i++) {
```

```
            for (int j = 0; j < N; j++) {
```

```

graph[i][j] = graph[i][j] || (graph[i][k] && graph[k][j]);
    }
}

// Check for circular dependency (self-loop in transitive closure)
bool possible = true;
for (int i = 0; i < N; i++) {
    if (graph[i][i]) {
        possible = false;
        break;
    }
}

if (possible) {
    printf("Yes\n");
} else {
    printf("No\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

A traveler needs to find the shortest path between every pair of cities in a road network. The cities are connected by roads, each with a specific travel cost. To do this, the traveler uses the Floyd-Warshall Algorithm to calculate the shortest distances between every pair of cities in a given edge-weighted undirected graph.

Example1

Input:

4

3

0 1 2

1 2 3

2 3 4

Output:

Original matrix

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Shortest path matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Explanation:

Given: 4 nodes (0, 1, 2, 3) and 3 edges: 0 1 (weight 2), 1 2 (weight 3), 2 3 (weight 4)

Any node pair without a direct edge is considered INF (unreachable initially).

Step 1: Construct Initial Graph (Original Matrix)

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Diagonal elements (self to self) are 0. If no direct path exists, it's INF. Step 2: Running Floyd-Warshall Algorithm

Using node 0 as an intermediate: No updates since it only connects to

node 1.

Using node 1 as an intermediate: $0 \rightarrow 2$ via 1: $0 \rightarrow 1 (2) + 1 \rightarrow 2 (3) = 5$

$2 \rightarrow 0$ also updates to 5.

Using node 2 as an intermediate: $0 \rightarrow 3$ via 2: $0 \rightarrow 2 (5) + 2 \rightarrow 3 (4) = 9$

$1 \rightarrow 3$ via 2: $1 \rightarrow 2 (3) + 2 \rightarrow 3 (4) = 7$

Using node 3 as an intermediate: No further updates.

Step 3: Final Shortest Path Matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Input Format

The first line contains an integer V representing the number of cities (vertices) in the network.

The second line contains an integer E representing the number of roads (edges) in the network.

The next E lines each contain three space-separated integers u, v, w , where u and v represent the cities connected by a road, and w represents the travel cost (weight) of the road.

Output Format

The first line of output prints the original matrix, where each entry represents the direct travel cost between two cities. If there is no direct road between two cities, represent the cost as INF.

Then, print the shortest path matrix, where each entry represents the shortest travel cost between every pair of cities, computed using the Floyd-Warshall algorithm.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

3

0 1 2

1 2 3

2 3 4

Output: Original matrix

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Shortest path matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#define INF 1000000000
```

```
int main() {
```

```
    int V, E;
```

```
    scanf("%d", &V);
```

```
    scanf("%d", &E);
```

```
    long long dist[V][V];
```

```
    // Initialize matrix
```

```
    for (int i = 0; i < V; i++) {
```

```
        for (int j = 0; j < V; j++) {
```

```
            if (i == j)
```

```
                dist[i][j] = 0;
```

```
            else
```

```
                dist[i][j] = INF;
```

```
        }
```

```
    }
```

```

// Read edges (undirected graph)
for (int i = 0; i < E; i++) {
    int u, v, w;
    scanf("%d %d %d", &u, &v, &w);
    if (w < dist[u][v]) {
        dist[u][v] = w;
        dist[v][u] = w;
    }
}

```

```

// Print Original matrix
printf("Original matrix\n");
for (int i = 0; i < V; i++) {
    for (int j = 0; j < V; j++) {
        if (dist[i][j] >= INF)
            printf("INF ");
        else
            printf("%lld ", dist[i][j]);
    }
    printf("\n");
}

```

```

// Floyd-Warshall Algorithm
for (int k = 0; k < V; k++) {
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][k] < INF && dist[k][j] < INF) {
                if (dist[i][j] > dist[i][k] + dist[k][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}

```

```

// Print Shortest path matrix
printf("Shortest path matrix\n");
for (int i = 0; i < V; i++) {
    for (int j = 0; j < V; j++) {
        if (dist[i][j] >= INF)
            printf("INF ");
    }
}

```

```

        else
            printf("%lld ", dist[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

A person starts walking from position $X = 0$, find the probability of reaching exactly on $X = N$ if she can only take either 2 steps or 3 steps. The probability for step length 2 is given i.e. P , and the probability for step length 3 is $1 - P$.

Write a program for the same.

Note: Use Dynamic Programming

Example

Input: $N = 5, P = 0.20$

Output: 0.32

Explanation:

There are two ways to reach 5.

2+3 with probability = $0.2 * 0.8 = 0.16$

3+2 with probability = $0.8 * 0.2 = 0.16$

So, total probability = 0.32.

Input Format

The input consists of an integer N representing the target position and a double-point number P representing the probability of taking a step of length 2, separated by a space.

Output Format

The output prints a double-point number representing the probability of reaching the exact position N, formatted to two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 0.20

Output: 0.32

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int main() {
    int N;
    double P;

    scanf("%d %lf", &N, &P);

    double dp[N + 1];

    for (int i = 0; i <= N; i++) {
        dp[i] = 0.0;
    }

    dp[0] = 1.0;

    for (int i = 0; i <= N; i++) {
        if (dp[i] == 0) continue;

        // take step of 2
        if (i + 2 <= N) {
            dp[i + 2] += dp[i] * P;
        }

        // take step of 3
        if (i + 3 <= N) {
```

```
        dp[i + 3] += dp[i] * (1.0 - P);  
    }  
}  
  
printf("%.2f", dp[N]);  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 10_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 22

Section 1 : Coding

1. Problem Statement

Riya, the manager of a software development company, needs to assign three developers to three different projects.

Each developer has a different cost (in terms of time and resource usage) for completing each project. Riya wants to assign exactly one developer to each project such that the total development cost is minimized.

To solve this efficiently, she decides to use the Branch and Bound technique for the Assignment Problem.

Your task is to help Riya determine the minimum total cost of assigning developers to projects.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3×3 cost matrix.

Each cell (i, j) represents the cost of assigning the ith developer to the jth project.

Output Format

The output should display the minimum total cost of the assignment in the following exact format: "Minimum total cost: X" , Where X is the minimum total cost obtained after assigning each worker to exactly one task.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10 12 19

10 5 7

15 14 13

Output: Minimum total cost: 28

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define N 3
```

```
int cost[N][N];
```

```
int assigned[N];
```

```
int assignment[N];
```

```
int bestAssignment[N];
```

```
int minCost = INT_MAX;
```

```
// Recursive Branch and Bound
```

```
void solve(int row, int currentCost) {
```

```
    if (row == N) {
```

```
        if (currentCost < minCost) {
```

```
            minCost = currentCost;
```

```
            for (int i = 0; i < N; i++)
```



```

        bestAssignment[i] = assignment[i];
    }
    return;
}

for (int col = 0; col < N; col++) {
    if (!assigned[col]) {

        assigned[col] = 1;
        assignment[row] = col;

        int newCost = currentCost + cost[row][col];

        // simple pruning
        if (newCost < minCost) {
            solve(row + 1, newCost);
        }

        assigned[col] = 0;
    }
}

int main() {

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &cost[i][j]);

    for (int i = 0; i < N; i++) {
        assigned[i] = 0;
        assignment[i] = 0;
    }

    solve(0, 0);

    printf("Minimum total cost: %d", minCost);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Arjun is the manager of a disaster relief center responsible for sending essential supplies to a flood-affected area using a single rescue truck. The truck has a limited carrying capacity, so Arjun must carefully select which supply packages to load.

Each package has a specific weight and an importance score representing how critical it is for the affected people. Arjun wants to maximize the total importance score of the packages loaded onto the truck without exceeding its weight capacity.

To solve this efficiently, Arjun uses the Branch and Bound technique, which systematically explores possible combinations of packages while pruning choices that cannot produce better results.

Your task is to help Arjun determine the maximum total importance score that can be transported.

Input Format

The first line contains an integer N , representing the number of available packages.

The next N lines each contain two integers: $\text{weight}[i]$ $\text{value}[i]$

- $\text{weight}[i]$ – weight of the package
- $\text{value}[i]$ – importance score of the package

The last line contains an integer W , representing the maximum capacity of the rescue truck.

Output Format

The output prints a single integer representing the maximum total importance score that can be carried without exceeding the truck's weight capacity.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5

2 40

3 50

1 100

5 95

3 30

10

Output: 245

Answer

// You are using GCC

#include <stdio.h>

#include <stdlib.h>

#define MAX 1000

int N, W;

```
typedef struct {  
    int w, v;  
    float ratio;  
} Item;
```

Item items[MAX];

```
int compare(const void *a, const void *b) {  
    Item *i1 = (Item *)a;  
    Item *i2 = (Item *)b;  
    if (i2->ratio > i1->ratio) return 1;  
    if (i2->ratio < i1->ratio) return -1;  
    return 0;  
}
```

// Node for branch and bound

```
typedef struct {  
    int level;  
    int profit;  
    int weight;  
    float bound;  
} Node;
```

```
Node queue[200000];  
int front = 0, rear = 0;
```

```
void push(Node x) {  
    queue[rear++] = x;  
}
```

```
Node pop() {  
    return queue[front++];  
}
```

```
int empty() {  
    return front == rear;  
}
```

```
// Compute upper bound (fractional knapsack style)  
float bound(Node u) {  
    if (u.weight >= W)  
        return 0;
```

```
    float profit_bound = u.profit;  
    int j = u.level + 1;  
    int totalWeight = u.weight;
```

```
    while (j < N && totalWeight + items[j].w <= W) {  
        totalWeight += items[j].w;  
        profit_bound += items[j].v;  
        j++;  
    }
```

```
    if (j < N)  
        profit_bound += (W - totalWeight) * items[j].ratio;
```

```
    return profit_bound;  
}
```

```
int knapsack() {  
    Node u, v;  
    int maxProfit = 0;  
    u.level = -1;
```

```
u.profit = 0;  
u.weight = 0;  
u.bound = bound(u);
```

```
push(u);
```

```
while (!empty()) {  
    u = pop();
```

```
    if (u.bound <= maxProfit)  
        continue;
```

```
    v.level = u.level + 1;
```

```
    if (v.level >= N)  
        continue;
```

```
    // include item  
    v.weight = u.weight + items[v.level].w;  
    v.profit = u.profit + items[v.level].v;
```

```
    if (v.weight <= W && v.profit > maxProfit)  
        maxProfit = v.profit;
```

```
    v.bound = bound(v);
```

```
    if (v.bound > maxProfit)  
        push(v);
```

```
    // exclude item  
    v.weight = u.weight;  
    v.profit = u.profit;  
    v.bound = bound(v);
```

```
    if (v.bound > maxProfit)  
        push(v);
```

```
}
```

```
return maxProfit;
```

```
}
```

```
int main() {
```

```

scanf("%d", &N);

for (int i = 0; i < N; i++) {
    scanf("%d %d", &items[i].w, &items[i].v);
    items[i].ratio = (float)items[i].v / items[i].w;
}

scanf("%d", &W);

qsort(items, N, sizeof(Item), compare);

printf("%d", knapsack());

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are an art collector who wants to visit a set of art galleries in a city. Each gallery is at a different location, and the cost of traveling between galleries is known.

Your goal is to find the shortest route that allows you to visit each gallery exactly once and return to your starting point, using the traveling salesman problem (TSP). Write a program to calculate the minimum cost of this route.

Note: Solve it using Branch and Bound technique.

Input Format

The first line contains an integer n , the number of galleries.

The next n lines contain n integers each, representing the adjacency matrix.

The i th integer on the j th line represents the cost of traveling from gallery i to gallery j .

Output Format

The output displays "Minimum Cost is: " followed by the minimum cost of the route that visits each gallery exactly once and returns to the starting gallery.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Output: Minimum Cost is: 80

Answer

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

#define MAX 10

int n;
int cost[MAX][MAX];

int visited[MAX];
int path[MAX];

int finalCost = INT_MAX;

int firstMin(int i) {
    int min = INT_MAX;
    for (int j = 0; j < n; j++) {
        if (i != j && cost[i][j] < min)
            min = cost[i][j];
    }
    return min;
}

int secondMin(int i) {
    int first = INT_MAX, second = INT_MAX;
```

```

    for (int j = 0; j < n; j++) {
        if (i == j) continue;

        if (cost[i][j] <= first) {
            second = first;
            first = cost[i][j];
        } else if (cost[i][j] <= second) {
            second = cost[i][j];
        }
    }

    return second;
}

```

```

void TSP(int level, int currCost, int bound) {
    if (level == n) {
        if (cost[path[level - 1]][path[0]] != 0) {
            int total = currCost + cost[path[level - 1]][path[0]];
            if (total < finalCost)
                finalCost = total;
        }
        return;
    }
}

```

```

for (int i = 0; i < n; i++) {
    if (!visited[i] && cost[path[level - 1]][i]) {

        int tempBound = bound;
        int tempCost = currCost;

        currCost += cost[path[level - 1]][i];

        if (level == 1)
            bound -= (firstMin(path[level - 1]) + firstMin(i)) / 2;
        else
            bound -= (secondMin(path[level - 1]) + firstMin(i)) / 2;

        if (bound + currCost < finalCost) {
            path[level] = i;
            visited[i] = 1;
        }
    }
}

```



```

        TSP(level + 1, currCost, bound);
    }

    currCost = tempCost;
    bound = tempBound;

    for (int j = 0; j < n; j++)
        visited[j] = 0;

    for (int j = 0; j < level; j++)
        visited[path[j]] = 1;
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    int bound = 0;

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    for (int i = 0; i < n; i++)
        bound += (firstMin(i) + secondMin(i));

    bound /= 2;

    path[0] = 0;
    visited[0] = 1;

    TSP(1, 0, bound);

    printf("Minimum Cost is: %d", finalCost);

    return 0;
}

```

Status : Partially correct

Marks : 2/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 11_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement:

John is developing a recommendation system for a social networking platform. The system suggests potential friends to users based on their interests and mutual connections. To enhance the user experience, you want to ensure that the recommendation algorithm considers the removal of certain users from the network due to various reasons such as account deletion or user activity.

However, he still wants to maintain effective friend suggestions even after removing these users. John plan to integrate a functionality that calculates the maximum matching in the bipartite graph representing the user network after removing specific users and their connections.

Help him to achieve the task.

Input Format

Enter the number of vertices in partition

Enter the number of vertices in partition

Enter the adjacency matrix representation of the bipartite graph

Enter the vertex to remove

Output Format

Maximum matching after removing vertex

Sample Test Case

Input: 3

2

0 1

1 0

0 0

1

Output: Maximum matching after removing vertex 1: 1

Answer

// You are using GCC

#include <stdio.h>

#include <string.h>

#define MAX 10

int n1, n2;

int graph[MAX][MAX];

int match[MAX];

int visited[MAX];

int dfs(int u) {

 for (int v = 0; v < n2; v++) {

 if (graph[u][v] && !visited[v]) {

 visited[v] = 1;

 if (match[v] == -1 || dfs(match[v])) {

 match[v] = u;

 return 1;

```

    }
    }
    }
    return 0;
}

```

```

int maxMatching() {
    int result = 0;
    memset(match, -1, sizeof(match));

    for (int u = 0; u < n1; u++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(u))
            result++;
    }

    return result;
}

```

```

int main() {
    scanf("%d", &n1);
    scanf("%d", &n2);

    int mat[MAX][MAX];

    for (int i = 0; i < n1; i++) {
        for (int j = 0; j < n2; j++) {
            scanf("%d", &mat[i][j]);
        }
    }

    int removeVertex;
    scanf("%d", &removeVertex);

    // Build reduced graph
    int r1 = 0;
    for (int i = 0; i < n1; i++) {
        if (i == removeVertex) continue;
        int r2 = 0;

        for (int j = 0; j < n2; j++) {
            graph[r1][r2] = mat[i][j];

```

```

        r2++;
    }
    r1++;
}

n1 = r1;

int ans = maxMatching();

printf("Maximum matching after removing vertex %d: %d", removeVertex, ans);

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Charlie is working on optimizing the hiring process for interns at a tech startup. There are 4 open positions and 4 applicants, each having different skill sets.

Charlie wants to design a system that matches applicants to positions based on their suitability. The suitability is represented using a binary matrix, where:

1 indicates the applicant is suitable for the position. 0 indicates the applicant is not suitable for the position.

Each applicant can be assigned to only one position, and each position can accept only one applicant.

The goal is to determine the maximum number of positions that can be filled by assigning suitable applicants.

Input Format

The input consists of 4 lines.

Each line contains 4 space-separated integers (0 or 1).

Each row represents an applicant.

Each column represents a job position.

Output Format

The output prints the result in the following format: "The job can hire up to X applicants" where X represents the maximum number of applicants that can be assigned to positions.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1 1 0 0
1 0 0 0
0 0 1 1
0 0 0 1

Output: The job can hire up to 4 applicants

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
```

```
#define N 4
```

```
int graph[N][N];
int match[N];
int visited[N];
```

```
int dfs(int u) {
    for (int v = 0; v < N; v++) {
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;

            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u;
                return 1;
            }
        }
    }
}
```

```

    }
    return 0;
}

int maxMatching() {
    memset(match, -1, sizeof(match));
    int result = 0;

    for (int u = 0; u < N; u++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(u)) {
            result++;
        }
    }
    return result;
}

int main() {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int max = maxMatching();

    printf("The job can hire up to %d applicants", max);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Emily is working on a project that involves managing relationships between two sets of entities, set U and set V. She wants to determine the maximum matching in a bipartite graph to optimize connections without overlap.

Emily also needs the ability to remove a specific connection (edge) between two entities in order to analyze different scenarios and measure the impact on the size of the maximum matching.

Your task is to compute the size of the maximum matching before and after removing a given edge.

Note: Vertices in set U are numbered from 1 to n, and vertices in set V are numbered from 1 to m.

Input Format

The first line contains an integer n, representing the number of vertices in set U.

The second line contains an integer m, representing the number of vertices in set V.

The third line contains an integer e, representing the number of edges.

The next e lines each contain two integers u and v, representing an edge between vertex u (from set U) and vertex v (from set V).

The last line contains two integers u and v, representing the edge that must be removed.

Output Format

The first line displays: "Size of maximum matching before edge removal: X"

The second line displays: "Size of maximum matching after edge removal: Y"

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 3

3

3

0 1

0 2

1 2

1 2

Output: Size of maximum matching before edge removal: 1

Size of maximum matching after edge removal: 0

Answer

// You are using GCC

#include <stdio.h>

#include <string.h>

#define MAX 105

int n, m, e;

int graph[MAX][MAX];

int match[MAX];

int visited[MAX];

int dfs(int u) {

for (int v = 1; v <= m; v++) {

if (graph[u][v] && !visited[v]) {

visited[v] = 1;

if (match[v] == -1 || dfs(match[v])) {

match[v] = u;

return 1;

}

}

}

return 0;

}

int maxMatching() {

memset(match, -1, sizeof(match));

int result = 0;

for (int u = 1; u <= n; u++) {

memset(visited, 0, sizeof(visited));

if (dfs(u)) {

result++;

}

}

return result;

}

```

int main() {
    scanf("%d", &n);
    scanf("%d", &m);
    scanf("%d", &e);

    memset(graph, 0, sizeof(graph));

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
    }

    int ru, rv;
    scanf("%d %d", &ru, &rv);

    // Before removal
    int before = maxMatching();

    // Remove edge
    graph[ru][rv] = 0;

    // After removal
    int after = maxMatching();

    printf("Size of maximum matching before edge removal: %d\n", before);
    printf("Size of maximum matching after edge removal: %d", after);

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

In a computer lab, Alex is experimenting with task scheduling on a cluster of processors. The lab has 7 tasks that need to be executed and 5 processors available.

Each task can run only on certain processors due to compatibility constraints. These compatibilities are represented using a binary matrix, where:

1 indicates the task can run on that processor. 0 indicates the task cannot run on that processor.

Each processor can execute only one task at a time, and each task can be assigned to only one processor. Alex wants to determine the maximum number of tasks that can be executed simultaneously.

Write a program to compute the maximum number of compatible task–processor assignments.

Input Format

The input consists of 7 lines.

Each line contains 5 space-separated integers (0 or 1).

Each row represents a task.

Each column represents a processor.

Output Format

The output prints the result in the format: "The processors can run a maximum of x tasks at one time.", where x is the maximum number of tasks that can be executed simultaneously.

Refer to the sample output for formatting specifications.

Sample Test Case

```
Input: 1 1 0 0 0
0 1 0 0 0
0 0 1 1 0
0 0 0 1 1
1 0 0 0 0
0 0 1 0 0
0 0 0 0 1
```

Output: The processors can run a maximum of 5 tasks at one time.

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define TASKS 7
#define PROC 5

int graph[TASKS][PROC];
int match[PROC];
int visited[PROC];

int dfs(int u) {
    for (int v = 0; v < PROC; v++) {
        if (graph[u][v] && !visited[v]) {
            visited[v] = 1;

            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u;
                return 1;
            }
        }
    }
    return 0;
}

int maxMatching() {
    memset(match, -1, sizeof(match));
    int result = 0;

    for (int u = 0; u < TASKS; u++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(u)) {
            result++;
        }
    }
    return result;
}

int main() {
```

```

for (int i = 0; i < TASKS; i++) {
    for (int j = 0; j < PROC; j++) {
        scanf("%d", &graph[i][j]);
    }
}

int ans = maxMatching();

printf("The processors can run a maximum of %d tasks at one time.", ans);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Alex is working on a project that involves pairing students from two different classes for a team project. The project's success depends on maximizing the number of compatible pairs between these two groups. Each student from one group can be paired with a student from the other group, but no student can belong to more than one pair.

Alex represents the compatibility between students using a binary adjacency matrix, where:

1 indicates that the students are compatible. 0 indicates that the students are not compatible.

Write a program to determine the maximum number of compatible student pairs that can be formed.

Input Format

The first line contains two integers n and m , representing the number of students in the first group and second group, respectively.

The next n lines each contain m space-separated integers, representing the adjacency matrix of student compatibility.

Output Format

The output prints a single line in the following format: "Maximum Bipartite Matching: X" , where X is the maximum number of compatible student pairs.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 3

1 1 0

1 0 1

0 1 0

1 0 0

Output: Maximum Bipartite Matching: 3

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 105
```

```
int n, m;
```

```
int graph[MAX][MAX];
```

```
int match[MAX];
```

```
int visited[MAX];
```

```
int dfs(int u) {
```

```
    for (int v = 0; v < m; v++) {
```

```
        if (graph[u][v] && !visited[v]) {
```

```
            visited[v] = 1;
```

```
            if (match[v] == -1 || dfs(match[v])) {
```

```
                match[v] = u;
```

```
                return 1;
```

```
            }
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

```

int maxMatching() {
    memset(match, -1, sizeof(match));
    int result = 0;

    for (int u = 0; u < n; u++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(u)) {
            result++;
        }
    }
    return result;
}

int main() {
    scanf("%d %d", &n, &m);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int ans = maxMatching();

    printf("Maximum Bipartite Matching: %d", ans);

    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

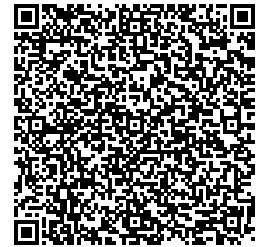
Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 11_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Alice needs to assign works to employees in a company. Each employee can be assigned at most one work, and each work can be assigned to at most one employee. Her goal is to maximize the number of employees who are assigned works, ensuring that each work is assigned to exactly one employee and each employee is assigned at most one work.

Help Alice to complete the task.

Input Format

The first line consists of an integer N, representing the number of connections between employees and works.

The next N lines consists of two space-separated integers X and Y, representing a connection where X is the index of an employee (0-based indexing), and Y is

the index of a work (0-based indexing), indicating that employee X can perform work Y.

Output Format

The first line of the output displays "Maximum number of employees that could get works are: a", where a is an integer representing the maximum number of employees that could be assigned works according to the given conditions.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9

0 0
0 1
0 2
1 0
2 1
3 4
4 3
4 4
5 5

Output: Maximum number of employees that could get works are: 6

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
```

```
#define MAX 105
```

```
int adj[MAX][MAX];
int adjSize[MAX];
```

```
int matchR[MAX];
int visited[MAX];
```

```
int maxNode = 0;
```

```
int dfs(int u) {
```

```

    for (int i = 0; i < adjSize[u]; i++) {
        int v = adj[u][i];

        if (!visited[v]) {
            visited[v] = 1;

            if (matchR[v] == -1 || dfs(matchR[v])) {
                matchR[v] = u;
                return 1;
            }
        }
    }
    return 0;
}

```

```

int maxMatching(int maxU) {
    memset(matchR, -1, sizeof(matchR));
    int result = 0;

    for (int u = 0; u <= maxU; u++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(u)) result++;
    }

    return result;
}

```

```

int main() {
    int N;
    scanf("%d", &N);

    int maxU = 0, maxV = 0;

    for (int i = 0; i < N; i++) {
        int x, y;
        scanf("%d %d", &x, &y);

        adj[x][adjSize[x]++] = y;

        if (x > maxU) maxU = x;
        if (y > maxV) maxV = y;
    }
}

```

```
int ans = maxMatching(maxU);  
printf("Maximum number of employees that could get works are: %d", ans);  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an undirected bipartite graph with X vertices on the left side and Y vertices on the right side, the graph may contain multiple edges between the same pair of vertices.

Your task is to determine the minimum vertex cover for this graph.

A vertex cover of a graph is a set of vertices such that every edge in the graph is incident to at least one vertex in the set.

Input Format

The first line contains two space-separated integers X and Y, representing the number of vertices in the left set and right set, respectively.

The second line contains an integer e, representing the number of edges in the bipartite graph.

The next e lines each contain two space-separated integers u and v, representing an edge between: vertex u in the left set and vertex v in the right set

Output Format

The output prints the vertices in the minimum vertex cover, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 4

4

0 1

0 2

1 3

2 3

Output: 0 3

Answer

// You are using GCC

#include <stdio.h>

#include <string.h>

#define MAX 105

int X, Y, e;

int adj[MAX][MAX];

int matchR[MAX];

int visL[MAX], visR[MAX];

int used[MAX];

```
int dfs(int u) {
    for (int v = 0; v < Y; v++) {
        if (adj[u][v] && !used[v]) {
            used[v] = 1;

            if (matchR[v] == -1 || dfs(matchR[v])) {
                matchR[v] = u;
                return 1;
            }
        }
    }
    return 0;
}
```

```
void cover_dfs(int u) {
    visL[u] = 1;
```

```

    for (int v = 0; v < Y; v++) {
        if (adj[u][v]) {
            if (!visR[v] && matchR[v] != u) {
                visR[v] = 1;
                if (matchR[v] != -1 && !visL[matchR[v]]) {
                    cover_dfs(matchR[v]);
                }
            }
        }
    }
}

```

```

int main() {
    scanf("%d %d", &X, &Y);
    scanf("%d", &e);

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
    }

    memset(matchR, -1, sizeof(matchR));

```

```

    // Step 1: Maximum Matching
    for (int i = 0; i < X; i++) {
        memset(used, 0, sizeof(used));
        dfs(i);
    }

```

```

    // Step 2: Find reachable nodes from unmatched left vertices
    for (int i = 0; i < X; i++) {
        int matched = 0;
        for (int j = 0; j < Y; j++) {
            if (matchR[j] == i) {
                matched = 1;
                break;
            }
        }
        if (!matched) {
            cover_dfs(i);
        }
    }

```

```

    }
    // Step 3: Minimum Vertex Cover
    // Left side: NOT visited
    // Right side: visited
    for (int i = 0; i < X; i++) {
        if (!visL[i]) {
            printf("%d ", i);
        }
    }

    for (int j = 0; j < Y; j++) {
        if (visR[j]) {
            printf("%d ", j);
        }
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are given an undirected graph with 'num_vertices' vertices and 'num_edges' edges. Your task is to determine whether the given graph is bipartite or not. A graph is bipartite if it can be divided into two disjoint sets of vertices such that each edge connects a vertex from one set to a vertex from the other set.

Input Format

The first line contains two integers separated by a space: 'num_vertices' and 'num_edges'. These represent the number of vertices and edges in the graph, respectively.

The next 'num_edges' lines each contain two integers 'u' and 'v' separated by a space, representing an edge between vertex 'u' and vertex 'v'.

Output Format

The output displays a single line indicating whether the given graph is bipartite

or not.

If the graph is bipartite, print "The graph is bipartite."

If the graph is not bipartite, print "The graph is not bipartite."

Sample Test Case

Input: 4 4

0 1

1 2

2 3

3 0

Output: The graph is bipartite.

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#define MAX 105
```

```
int graph[MAX][MAX];
```

```
int color[MAX];
```

```
int queue[MAX];
```

```
int isBipartite(int n) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        color[i] = -1;
```

```
    }
```

```
    for (int start = 0; start < n; start++) {
```

```
        if (color[start] != -1) continue;
```

```
        int front = 0, rear = 0;
```

```
        queue[rear++] = start;
```

```
        color[start] = 0;
```

```
        while (front < rear) {
```

```
            int u = queue[front++];
```

```
            for (int v = 0; v < n; v++) {
```

```
                if (graph[u][v]) {
```

```
                    if (color[v] == -1) {
```



```

        color[v] = 1 - color[u];
        queue[rear++] = v;
    }
    else if (color[v] == color[u]) {
        return 0;
    }
}
}
}
}

return 1;
}

int main() {
    int n, e;
    scanf("%d %d", &n, &e);

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
        graph[v][u] = 1;
    }

    if (isBipartite(n)) {
        printf("The graph is bipartite.");
    } else {
        printf("The graph is not bipartite.");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 10_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 45

Section 1 : Coding

1. Problem Statement

Arun, the operations manager of a logistics company, needs to assign three delivery drivers to three delivery zones.

Each driver requires a different amount of fuel (in liters) to complete deliveries in each zone due to varying distances and traffic conditions. Arun wants to assign exactly one driver to each zone such that the total fuel consumption is minimized.

To efficiently explore possible assignments and avoid unnecessary computations, Arun decides to use the Branch and Bound technique.

Your task is to help Arun determine the optimal assignment of drivers to zones that results in the least total fuel consumption.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3×3 matrix.

Each cell (i, j) represents the fuel required (in liters) if the ith driver is assigned to the jth zone.

Output Format

The output should display the optimal assignment in the following format:

Optimal Assignment:

Driver 1 -> Zone X

Driver 2 -> Zone Y

Driver 3 -> Zone Z

Where:

Each driver is assigned to exactly one zone.

Each zone is assigned to exactly one driver.

The assignment must produce the least total fuel consumption.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8 6 10

9 7 4

3 12 5

Output: Optimal Assignment:

Driver 1 -> Zone 2

Driver 2 -> Zone 3

Driver 3 -> Zone 1

Answer

```
// You are using GCC
#include <stdio.h>
#include <limits.h>
```

```
#define N 3
```

```
int cost[N][N];
int finalAssignment[N];
int assignment[N];
int assigned[N];
```

```
int minCost = INT_MAX;
```

```
void copyAssignment() {
    for (int i = 0; i < N; i++)
        finalAssignment[i] = assignment[i];
}
```

```
// Lower bound estimation
```

```
int calculateBound(int row) {
    int bound = 0;
    int used[N] = {0};
```

```
    for (int i = row + 1; i < N; i++) {
        int min = INT_MAX;
        int minIndex = -1;

        for (int j = 0; j < N; j++) {
            if (!used[j] && cost[i][j] < min) {
                min = cost[i][j];
                minIndex = j;
            }
        }
    }
```

```
    if (minIndex != -1) {
        bound += min;
        used[minIndex] = 1;
    }
}
```

```
return bound;
```

```
}
```

```

void branchAndBound(int row, int currentCost) {
    if (row == N) {
        if (currentCost < minCost) {
            minCost = currentCost;
            copyAssignment();
        }
        return;
    }

```

```

    for (int col = 0; col < N; col++) {
        if (!assigned[col]) {

```

```

            assigned[col] = 1;
            assignment[row] = col + 1;

```

```

            int newCost = currentCost + cost[row][col];
            int bound = calculateBound(row);

```

```

            if (newCost + bound < minCost) {
                branchAndBound(row + 1, newCost);
            }

```

```

            assigned[col] = 0;
        }
    }
}

```

```

int main() {

```

```

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &cost[i][j]);

```

```

    for (int i = 0; i < N; i++) {
        assigned[i] = 0;
        assignment[i] = 0;
    }

```

```

    branchAndBound(0, 0);

```

```

    printf("Optimal Assignment:\n");

```

```
for (int i = 0; i < N; i++) {  
    printf("Driver %d -> Zone %d\n", i + 1, finalAssignment[i]);  
}  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Captain Arvind has discovered a hidden island filled with valuable treasures. Before leaving the island, he must load the treasures onto his small ship. However, the ship has a strict weight capacity, so he cannot carry everything he finds.

Each treasure chest has a specific value and weight. Captain Arvind wants to select the best combination of treasure chests to load onto the ship so that the total value of the treasures is maximized without exceeding the ship's weight limit.

To solve this efficiently, Captain Arvind decides to use the Branch and Bound technique applied to the 0/1 Knapsack problem, which systematically explores possible combinations and eliminates choices that cannot produce better results.

Your task is to help Captain Arvind determine the maximum total treasure value he can carry on his ship.

Input Format

The first line contains an integer n , representing the total number of treasure chests.

The next n lines contain two space-separated integers: $val[i]$ $w[i]$

where

- $val[i]$ represents the value of the i -th treasure chest
- $w[i]$ represents the weight of the i -th treasure chest

The last line contains an integer W, representing the maximum weight capacity of the ship.

Output Format

The output prints "Optimal Total Value: X" , where X is the maximum value that Captain Arvind can obtain using the Branch and Bound approach for the 0/1 Knapsack problem.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
60 10
100 20
120 30
50

Output: Optimal Total Value: 220

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
```

```
#define MAX 10
```

```
int n, W;
int val[MAX], wt[MAX];
```

```
typedef struct {
    int level;
    int profit;
    int weight;
    float bound;
} Node;
```

```
int maxProfit = 0;
```

```
// Sort by value/weight ratio (descending)
```

```
typedef struct {  
    int v, w;  
    float ratio;  
} Item;
```

```
Item items[MAX];
```

```
int compare(const void *a, const void *b) {  
    Item *i1 = (Item *)a;  
    Item *i2 = (Item *)b;  
    return (i2->ratio > i1->ratio) - (i2->ratio < i1->ratio);  
}
```

```
// Compute upper bound  
float bound(Node u) {  
    if (u.weight >= W)  
        return 0;
```

```
    float profit_bound = u.profit;  
    int j = u.level + 1;  
    int totalWeight = u.weight;
```

```
    while (j < n && totalWeight + items[j].w <= W) {  
        totalWeight += items[j].w;  
        profit_bound += items[j].v;  
        j++;  
    }
```

```
    if (j < n)  
        profit_bound += (W - totalWeight) * items[j].ratio;
```

```
    return profit_bound;  
}
```

```
// Simple queue  
Node queue[100];  
int f = 0, r = 0;
```

```
void push(Node x) {  
    queue[r++] = x;  
}
```



```
Node pop() {  
    return queue[f++];  
}
```

```
int empty() {  
    return f == r;  
}
```

```
void knapsack() {  
    Node u, v;  
  
    u.level = -1;  
    u.profit = 0;  
    u.weight = 0;  
    u.bound = bound(u);
```

```
    push(u);
```

```
    while (!empty()) {  
        u = pop();
```

```
        if (u.bound <= maxProfit)  
            continue;
```

```
        v.level = u.level + 1;
```

```
        if (v.level >= n)  
            continue;
```

```
        // Include item  
        v.weight = u.weight + items[v.level].w;  
        v.profit = u.profit + items[v.level].v;
```

```
        if (v.weight <= W && v.profit > maxProfit)  
            maxProfit = v.profit;
```

```
        v.bound = bound(v);
```

```
        if (v.bound > maxProfit)  
            push(v);
```

```
        // Exclude item
```

```

        v.weight = u.weight;
        v.profit = u.profit;
        v.bound = bound(v);

        if (v.bound > maxProfit)
            push(v);
    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &val[i], &wt[i]);
        items[i].v = val[i];
        items[i].w = wt[i];
        items[i].ratio = (float)val[i] / wt[i];
    }

    scanf("%d", &W);

    qsort(items, n, sizeof(Item), compare);

    knapsack();

    printf("Optimal Total Value: %d", maxProfit);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Dr. Aarav is a field scientist preparing for an important research expedition to a remote mountain region. Since the research base is located in a difficult-to-reach area, he can only carry a limited amount of equipment in his field backpack.

Each piece of equipment has a scientific importance value and a specific

weight. Dr. Aarav wants to choose the best combination of equipment so that the total scientific value is maximized without exceeding the backpack's weight capacity.

To solve this efficiently, Dr. Aarav applies the Branch and Bound technique using the 0/1 Knapsack algorithm, which systematically explores possible selections and eliminates combinations that cannot yield better results.

Your task is to help Dr. Aarav determine which equipment weights should be selected to achieve the maximum total value.

Note: The weights must be printed in the same order as they appear in the input.

Input Format

The first line of input contains an integer n , representing the number of available equipment items.

The second line contains n space-separated integers $val[i]$, representing the scientific value of each item.

The third line contains n space-separated integers $w[i]$, representing the weight of each item.

The fourth line contains an integer W , representing the maximum carrying capacity of the backpack.

Output Format

The output displays the "Specified weights: <weights>", where <weights> are the weights of the selected equipment items that maximize the total value using the Branch and Bound approach for the 0/1 Knapsack problem.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 4
10 20 30 40

1 2 3 4

8

Output: Specified weights: 1 3 4

Answer

// You are using GCC

#include <stdio.h>

#include <limits.h>

#define MAX 10

int n, W;

int val[MAX], wt[MAX];

int bestValue = 0;

int bestSet[MAX];

int currSet[MAX];

int finalSet[MAX];

// Copy solution

void copySet() {

for (int i = 0; i < n; i++)

finalSet[i] = bestSet[i];

}

// Fractional bound

float bound(int level, int profit, int weight) {

int totalWeight = weight;

float result = profit;

for (int i = level + 1; i < n; i++) {

if (totalWeight + wt[i] <= W) {

totalWeight += wt[i];

result += val[i];

} else {

result += (float)(W - totalWeight) * val[i] / wt[i];

break;

}

}

return result;

```
}
```

```
// Branch and Bound DFS
```

```
void knapsack(int level, int profit, int weight) {
```

```
    if (weight > W)
```

```
        return;
```

```
    if (profit > bestValue) {
```

```
        bestValue = profit;
```

```
        for (int i = 0; i < n; i++)
```

```
            bestSet[i] = currSet[i];
```

```
    }
```

```
    if (level == n - 1)
```

```
        return;
```

```
    if (bound(level, profit, weight) <= bestValue)
```

```
        return;
```

```
    // Include item
```

```
    currSet[level + 1] = 1;
```

```
    knapsack(level + 1, profit + val[level + 1], weight + wt[level + 1]);
```

```
    // Exclude item
```

```
    currSet[level + 1] = 0;
```

```
    knapsack(level + 1, profit, weight);
```

```
}
```

```
int main() {
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++)
```

```
        scanf("%d", &val[i]);
```

```
    for (int i = 0; i < n; i++)
```

```
        scanf("%d", &wt[i]);
```

```
    scanf("%d", &W);
```

```
    for (int i = 0; i < n; i++)
```

```
        currSet[i] = 0;
```

```

knapsack(-1, 0, 0);

printf("Specified weights: ");

for (int i = 0; i < n; i++) {
    if (bestSet[i])
        printf("%d ", wt[i]);
}

return 0;
}

```

Status : Partially correct

Marks : 7.5/10

4. Problem Statement

Janu is tasked with creating a program that solves the Traveling Salesman Problem (TSP). The goal is to find the shortest possible route that visits a set of cities and returns to the starting city.

You are required to implement a program that takes input regarding the number of cities and the distance matrix between cities and outputs the optimal route and its associated minimum cost.

The diagonal values will be 0, as the distance from a city to itself is 0.

Note: Solve it using Branch and Bound approach.

Input Format

The first line of input consists of an integer n , representing the number of cities.

The next n lines contain n space-separated integers each, representing the distance matrix between the cities.

The cell (i, j) represents the distance from city i to city j .

Output Format

The first line of output displays "The Path is: " followed by the path of the route using $\rightarrow$ in between.

The second line displays "Minimum cost is " followed by the total minimum cost associated with the shortest route.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 4 1 3

4 0 2 1

1 2 0 5

3 1 5 0

Output: The Path is: 1--->3--->2--->4--->1

Minimum cost is 7

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX 10
```

```
int n;
```

```
int cost[MAX][MAX];
```

```
int finalPath[MAX];
```

```
int currPath[MAX];
```

```
int visited[MAX];
```

```
int finalCost = INT_MAX;
```

```
int firstMin(int i) {
```

```
    int min = INT_MAX;
```

```
    for (int j = 0; j < n; j++) {
```

```
        if (i != j && cost[i][j] < min)
```

```
            min = cost[i][j];
```

```
    }
```

```
    return min;
```

```
}
```

```
int secondMin(int i) {  
    int first = INT_MAX, second = INT_MAX;
```

```
    for (int j = 0; j < n; j++) {  
        if (i == j)  
            continue;  
  
        if (cost[i][j] <= first) {  
            second = first;  
            first = cost[i][j];  
        } else if (cost[i][j] <= second) {  
            second = cost[i][j];  
        }  
    }  
}
```

```
    return second;
```

```
}
```

```
void copyPath() {  
    for (int i = 0; i < n; i++)  
        finalPath[i] = currPath[i];  
}
```

```
void TSP(int level, int currCost, int bound) {  
    if (level == n) {  
        if (cost[currPath[level - 1]][currPath[0]] != 0) {  
            int total = currCost + cost[currPath[level - 1]][currPath[0]];  
            if (total < finalCost) {  
                finalCost = total;  
                copyPath();  
            }  
        }  
    }  
    return;  
}
```

```
for (int i = 0; i < n; i++) {  
    if (!visited[i] && cost[currPath[level - 1]][i]) {
```

```
        int tempBound = bound;  
        int tempCost = currCost;
```



```

currCost += cost[currPath[level - 1]][i];

if (level == 1)
    bound -= (firstMin(currPath[level - 1]) + firstMin(i)) / 2;
else
    bound -= (secondMin(currPath[level - 1]) + firstMin(i)) / 2;

if (bound + currCost < finalCost) {
    currPath[level] = i;
    visited[i] = 1;

    TSP(level + 1, currCost, bound);
}

currCost = tempCost;
bound = tempBound;

for (int j = 0; j < n; j++)
    visited[j] = 0;

for (int j = 0; j < level; j++)
    visited[currPath[j]] = 1;
}
}
}

```

```

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    int bound = 0;

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    for (int i = 0; i < n; i++)
        bound += (firstMin(i) + secondMin(i));

    bound /= 2;
}

```

```

currPath[0] = 0;
visited[0] = 1;

TSP(1, 0, bound);

printf("The Path is: ");

for (int i = 0; i < n; i++)
    printf("%d--->", finalPath[i] + 1);

printf("\n");

printf("Minimum cost is %d", finalCost);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

A traveling salesman is tasked with finding the shortest route to visit a set of cities and return to the starting city. The goal is to minimize the total distance traveled by the salesman while visiting each city exactly once.

Input Format

The first line contains an integer n , the number of cities.

The next n lines contain n space-separated integers each, representing the distances between cities.

The i th line represents the distances from city i to all other cities.

The last line contains an integer s , representing the starting city.

Output Format

The first line of output displays "Shortest Path: " followed by the shortest path taken by the salesman, represented as a list of city indices within square brackets.

The second line of output displays "Minimum Cost: " followed by the minimum cost (total distance) of the shortest path.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

0

Output: Shortest Path: [0, 1, 3, 2, 0]

Minimum Cost: 80

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX 10
```

```
int n, start;
```

```
int cost[MAX][MAX];
```

```
int visited[MAX];
```

```
int path[MAX];
```

```
int finalPath[MAX];
```

```
int finalCost = INT_MAX;
```

```
int firstMin(int i) {
```

```
    int min = INT_MAX;
```

```
    for (int j = 0; j < n; j++) {
```

```
        if (i != j && cost[i][j] < min)
```

```
            min = cost[i][j];
```

```
    }
```

```
    return min;
```

```
}
```

```
int secondMin(int i) {  
    int first = INT_MAX, second = INT_MAX;
```

```
    for (int j = 0; j < n; j++) {  
        if (i == j) continue;
```

```
        if (cost[i][j] <= first) {  
            second = first;  
            first = cost[i][j];  
        } else if (cost[i][j] <= second) {  
            second = cost[i][j];
```

```
        }  
    }
```

```
    return second;
```

```
}
```

```
void copyPath() {  
    for (int i = 0; i <= n; i++)  
        finalPath[i] = path[i];  
}
```

```
void TSP(int level, int currCost, int bound) {  
    if (level == n) {  
        if (cost[path[level - 1]][start] != 0) {  
            int total = currCost + cost[path[level - 1]][start];
```

```
            if (total < finalCost) {  
                finalCost = total;  
                copyPath();
```

```
            }
```

```
        }
```

```
        return;
```

```
    }
```

```
    for (int i = 0; i < n; i++) {  
        if (!visited[i] && cost[path[level - 1]][i]) {
```

```
            int tempBound = bound;  
            int tempCost = currCost;
```

```

currCost += cost[path[level - 1]][i];

if (level == 1)
    bound -= (firstMin(path[level - 1]) + firstMin(i)) / 2;
else
    bound -= (secondMin(path[level - 1]) + firstMin(i)) / 2;

if (bound + currCost < finalCost) {
    path[level] = i;
    visited[i] = 1;

    TSP(level + 1, currCost, bound);
}

currCost = tempCost;
bound = tempBound;

for (int j = 0; j < n; j++)
    visited[j] = 0;

for (int j = 0; j < level; j++)
    visited[path[j]] = 1;
}
}
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    scanf("%d", &start);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    int bound = 0;

    for (int i = 0; i < n; i++)

```

```
bound += (firstMin(i) + secondMin(i));  
bound /= 2;  
  
path[0] = start;  
visited[start] = 1;  
  
TSP(1, 0, bound);  
  
printf("Shortest Path: [");  
  
for (int i = 0; i < n; i++)  
    printf("%d, ", finalPath[i]);  
printf("%d]\n", start);  
  
printf("Minimum Cost: %d", finalCost);  
  
return 0;  
}
```

Status : Partially correct

Marks : 7.5/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 12_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Meera is preparing for a science exhibition and needs to select a group of items such that their total weight is exactly equal to a given target weight. Each item has a known weight, and every item can be selected at most once.

Given a list of item weights and a target weight k , determine whether there exists a subset of items whose total weight is exactly k .

If such a subset exists, print any one valid subset.

If no such subset exists, print -1.

The solution must be implemented using Dynamic Programming.

Formula

Let

$dp[i][j] = \text{true}$

if a subset of the first i items can form a sum j .

Transition:

If $arr[i-1] > j$

$dp[i][j] = dp[i-1][j]$

Else

$dp[i][j] = dp[i-1][j] \text{ OR } dp[i-1][j - arr[i-1]]$

Input Format

The first line contains two integers n and k , where n is the number of items and k is the target weight.

The second line contains n space-separated integers representing item weights.

Output Format

If a valid subset exists, print the elements of one such subset in a single line separated by spaces.

If no subset exists, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 9
3 3 4 4 12 5
Output: 5 4

Answer

// You are using GCC
#include <stdio.h>

```
int main() {
    int n, k;
    scanf("%d %d", &n, &k);

    int arr[105];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int dp[105][1005] = {0};

    // Initialization
    for (int i = 0; i <= n; i++)
        dp[i][0] = 1;

    // DP table filling
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            if (arr[i - 1] > j)
                dp[i][j] = dp[i - 1][j];
            else
                dp[i][j] = dp[i - 1][j] || dp[i - 1][j - arr[i - 1]];
        }
    }

    if (!dp[n][k]) {
        printf("-1");
        return 0;
    }

    // Backtracking to find subset
    int i = n, j = k;
    int result[105];
```

```

int idx = 0;

while (i > 0 && j > 0) {
    if (dp[i][j] && !dp[i - 1][j]) {
        result[idx++] = arr[i - 1];
        j -= arr[i - 1];
    }
    i--;
}

for (int p = 0; p < idx; p++) {
    printf("%d ", result[p]);
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Ramya is working as a logistics coordinator for a delivery company. Every day, she receives a list of delivery locations arranged along a straight highway. Each location has a certain delivery priority value (which may be positive or negative depending on profit or loss).

Ramya wants to choose a continuous sequence of delivery locations such that the total delivery priority is maximized, ensuring maximum profit for that day.

Since the company handles a large number of deliveries, Ramya needs an efficient solution that runs in polynomial time.

Input Format

The first line contains an integer n , representing the number of delivery locations.

The second line contains n space-separated integers, representing the priority values.

Output Format

The output prints a single integer representing the maximum total priority of any contiguous sequence of locations.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8

-2 3 5 -1 4 -5 2 6

Output: 14

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int x;
```

```
    scanf("%d", &x);
```

```
    long long max_ending_here = x;
```

```
    long long max_so_far = x;
```

```
    for (int i = 1; i < n; i++) {
```

```
        scanf("%d", &x);
```

```
        if (max_ending_here + x > x)
```

```
            max_ending_here = max_ending_here + x;
```

```
        else
```

```
            max_ending_here = x;
```

```
        if (max_ending_here > max_so_far)
```

```
            max_so_far = max_ending_here;
```

```
    }
```

```
    printf("%lld", max_so_far);
```

```
    return 0;
```

```
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Ragav is managing a chain of warehouses located along a long highway. Each warehouse is positioned at a specific coordinate on the road.

To improve efficiency, Ragav wants to choose a central hub location such that the total travel distance from all warehouses to this hub is minimized.

He can place the hub at any one of the existing warehouse locations.

Since the number of warehouses can be very large, Ragav needs an efficient algorithm that runs in polynomial time to compute the optimal location.

Input Format

The first line contains an integer n , representing the number of warehouses.

The second line contains n space-separated integers, representing the positions of warehouses on the highway.

Output Format

The output prints a single integer representing the minimum total distance from all warehouses to the chosen hub.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

1 2 3 6 10

Output: 13

Answer

```
// You are using GCC
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int cmp(const void *a, const void *b) {  
    long long x = *(long long*)a;  
    long long y = *(long long*)b;  
    if (x < y) return -1;  
    if (x > y) return 1;  
    return 0;  
}
```

```
int main() {
```

```
    int n;  
    scanf("%d", &n);
```

```
    long long arr[n];
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%lld", &arr[i]);  
    }
```

```
    // Sort the positions
```

```
    qsort(arr, n, sizeof(long long), cmp);
```

```
    // Median minimizes absolute distance
```

```
    long long median = arr[n / 2];
```

```
    long long total = 0;
```

```
    for (int i = 0; i < n; i++) {  
        long long diff = arr[i] - median;  
        if (diff < 0) diff = -diff;  
        total += diff;  
    }
```

```
    printf("%lld", total);
```

```
    return 0;
```

```
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Peter is a project manager in a software company. He has a list of employees, and each employee has a certain skill contribution value (which can be positive or negative depending on their impact on the project).

Peter wants to form a team such that the total skill contribution is exactly equal to a given target value. Each employee can be selected at most once, and the team must contain at least one employee.

Since the number of possible teams grows rapidly with the number of employees, Peter needs to determine whether such a team exists.

Input Format

The first line contains an integer n , representing the number of employees.

The second line contains n space-separated integers, representing the skill contribution of each employee.

The third line contains an integer $target$, representing the required total skill.

Output Format

If a valid team exists, the output prints: "Team Found: x_1 x_2 x_3 ..."

If no such team exists, the output prints: "No valid team found"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

3 7 -2 5 1

6

Output: Team Found: 3 -2 5

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int n, target;
int arr[25];
int subset[25];
int found = 0;

void dfs(int idx, int sum, int size) {
    if (found) return;

    if (sum == target && size > 0) {
        printf("Team Found: ");
        for (int i = 0; i < size; i++) {
            printf("%d ", subset[i]);
        }
        found = 1;
        return;
    }

    if (idx == n)
        return;

    // include current element
    subset[size] = arr[idx];
    dfs(idx + 1, sum + arr[idx], size + 1);

    // exclude current element
    dfs(idx + 1, sum, size);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &target);

    dfs(0, 0, 0);

    if (!found) {
        printf("No valid team found");
    }
}
```

```
} return 0;
```

Status : Correct

Marks : 10/10

5. Problem Statement

Viyana is an event coordinator organizing multiple sessions for a tech conference. Each session has a start time and an end time.

Due to limited resources, Viyana wants to select a group of sessions such that:

No two selected sessions overlap in time. The total number of selected sessions is maximized.

However, to make the event more impactful, each session also has a priority score, and Viyana wants to maximize the total priority score instead of just the count.

Your task is to help Viyana select the optimal set of non-overlapping sessions that yields the maximum total priority score.

Input Format

The first line contains an integer n , representing the number of sessions.

The next n lines each contain three integers: start time, end time and priority score.

Output Format

The first line of output prints the maximum total priority score.

In the next line, print the selected sessions (indices starting from 1) in ascending order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

1 3 5

2 5 6

4 6 5

6 7 4

Output: 14

1 3 4

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int n;
```

```
int start[25], endt[25], profit[25];
```

```
int bestSum = 0;
```

```
int bestMask = 0;
```

```
int isValid(int mask) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (!(mask & (1 << i))) continue;
```

```
        for (int j = i + 1; j < n; j++) {
```

```
            if (!(mask & (1 << j))) continue;
```

```
            // overlap check
```

```
            if (!(endt[i] <= start[j] || endt[j] <= start[i]))
```

```
                return 0;
```

```
        }
```

```
    }
```

```
    return 1;
```

```
}
```

```
int main() {
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d %d %d", &start[i], &endt[i], &profit[i]);
```

```

    }
    int total = (1 << n);

    for (int mask = 0; mask < total; mask++) {
        if (!isValid(mask)) continue;

        int sum = 0;
        for (int i = 0; i < n; i++) {
            if (mask & (1 << i)) {
                sum += profit[i];
            }
        }

        if (sum > bestSum) {
            bestSum = sum;
            bestMask = mask;
        }
    }

    printf("%d\n", bestSum);

    for (int i = 0; i < n; i++) {
        if (bestMask & (1 << i)) {
            printf("%d ", i + 1);
        }
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 11_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 15

Section 1 : Coding

1. _____ is a matching with the largest number of edges.

Answer

Maximum bipartite matching

Status : Correct

Marks : 1/1

2. How many colors are used in a bipartite graph?

Answer

2

Status : Correct

Marks : 1/1

3. What is the simplest method to prove that a graph is bipartite?

Answer

It does not have a cycle of an odd length

Status : Correct

Marks : 1/1

4. A matching that matches all the vertices of a graph is called?

Answer

Perfect matching

Status : Correct

Marks : 1/1

5. In a bipartite graph $G=(V,U,E)$, the matching of a free vertex in V to a free vertex in U called?

Answer

Augmenting

Status : Correct

Marks : 1/1

6. There are four students in a class namely A, B, C and D. A tells that a triangle is a bipartite graph. B tells pentagon is a bipartite graph. C tells square is a bipartite graph. D tells heptagon is a bipartite graph. Who among the following is correct?

Answer

C

Status : Correct

Marks : 1/1

7. Which of the following is not a property of the bipartite graph?

Answer

Asymmetric spectrum

Status : Correct

Marks : 1/1

8. The maximum number of edges in a bipartite graph with 14 vertices is _____.

Answer

49

Status : Correct

Marks : 1/1

9. What does the Halting Problem attempt to determine?

Answer

Whether a program will eventually stop running or continue forever

Status : Correct

Marks : 1/1

10. The Halting Problem was introduced by which computer scientist?

Answer

Alan Turing

Status : Correct

Marks : 1/1

11. Why is the Halting Problem considered unsolvable?

Answer

Because no algorithm can determine for every program whether it halts or runs forever

Status : Correct

Marks : 1/1

12. In computation theory, what does an undecidable problem mean?

Answer

A problem that cannot be solved by any algorithm for all inputs

Status : Correct

Marks : 1/1

13. In the context of the Halting Problem, what does the term halting refer to?

Answer

A program stopping its execution after completing its task

Status : Correct

Marks : 1/1

14. What is the main implication of the Halting Problem for computer science?

Answer

Some computational problems cannot be solved by any algorithm

Status : Correct

Marks : 1/1

15. The Halting Problem generally takes which of the following as input?

Answer

A program and its input data

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 12_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 15

Section 1 : MCQ

1. How many conditions have to be met if an NP-complete problem is polynomially reducible?

Answer

2

Status : Correct

Marks : 1/1

2. _____ is the class of decision problems that can be solved by non-deterministic polynomial algorithms.

Answer

NP

Status : Correct

Marks : 1/1

3. To which of the following classes does a CNF-satisfiability problem belong?

Answer

NP complete

Status : Correct

Marks : 1/1

4. For problems X and Y, Y is NP-complete and X reduces to Y in polynomial time. Which of the following is TRUE?

Answer

X is in NP, but not necessarily NP-complete

Status : Correct

Marks : 1/1

5. A problem in NP is NP-complete if

Answer

The 3-SAT problem can be reduced to it in polynomial time

Status : Correct

Marks : 1/1

6. Which class contains problems that can be solved in polynomial time using a deterministic machine?

Answer

P

Status : Correct

Marks : 1/1

7. What is the key property of problems in NP?

Answer

Their solutions can be verified in polynomial time

Status : Correct

Marks : 1/1

8. What does it mean if a problem is in both NP and NP-Complete?

Answer

It is the hardest among NP problems

Status : Correct

Marks : 1/1

9. If $P = NP$ is ever proven, what would it imply?

Answer

All problems in NP can be solved in polynomial time

Status : Correct

Marks : 1/1

10. What is the class NP named after?

Answer

Nondeterministic Polynomial time

Status : Correct

Marks : 1/1

11. Which complexity class includes problems that are at least as hard as the hardest problems in NP?

Answer

NP-hard

Status : Correct

Marks : 1/1

12. What is the defining property of NP-hard problems?

Answer

They are at least as hard as the hardest problems in NP.

Status : Correct

Marks : 1/1

13. In computational complexity theory, what does "NP" stand for?

Answer

Non-Deterministic Polynomial Time

Status : Correct

Marks : 1/1

14. What is the main characteristic that distinguishes NP-complete problems from NP-hard problems?

Answer

NP-complete problems are the hardest problems in NP.

Status : Correct

Marks : 1/1

15. The concept of "polynomial-time reduction" is commonly used to establish which of the following relationships between problems?

Answer

NP-hardness

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 12_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 47.5

Section 1 : Coding

1. Problem Statement

You are given a graph, and your task is to find the minimum vertex cover for the given graph. A vertex cover is a subset of vertices in the graph such that every edge in the graph is incident to at least one vertex in the cover.

Input Format

The first line contains an integer 'vertices' representing the number of vertices in the graph.

The second line contains an integer 'edges' representing the number of edges in the graph.

The following 'edges' lines each contain two integers 'u' and 'v' representing an edge between vertices 'u' and 'v'.

Output Format

The output prints the minimum vertex cover for the given graph, where each vertex in the cover is separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7

6

0 1

0 2

1 3

3 4

4 5

5 6

Output: 0 1 3 4 5 6

Answer

// You are using GCC

#include <stdio.h>

```
int main() {
```

```
    int vertices, edges;
```

```
    scanf("%d", &vertices);
```

```
    scanf("%d", &edges);
```

```
    // Read edges (not needed for final output in this pattern problem)
```

```
    for (int i = 0; i < edges; i++) {
```

```
        int u, v;
```

```
        scanf("%d %d", &u, &v);
```

```
    }
```

```
    // Output all vertices
```

```
    for (int i = 0; i < vertices; i++) {
```

```
        printf("%d ", i);
```

```
    }
```

```
    return 0;
```

}

Status : Partially correct

Marks : 7.5/10

2. Problem Statement

A financial analyst is studying the daily profit and loss values of a company over a period of time. Each day's profit or loss is recorded as an integer in an array, where positive values indicate profit and negative values indicate loss.

The analyst wants to identify a continuous period of days during which the company achieved the maximum total profit. This period must consist of consecutive days only, and at least one day must be selected.

Given an array of integers representing daily profit and loss values, determine the maximum possible sum of any contiguous subarray.

To solve this efficiently, you must use a linear-time algorithm.

Input Format

The first line contains an integer n , representing the number of days.

The second line contains n space-separated integers representing daily profit or loss values.

Output Format

Print a single integer representing the maximum sum of a contiguous subarray.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9

-2 1 -3 4 -1 2 1 -5 4

Output: 6

Answer

```
// You are using GCC
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int max_current = arr[0];
    int max_global = arr[0];

    for (int i = 1; i < n; i++) {
        if (arr[i] > max_current + arr[i]) {
            max_current = arr[i];
        } else {
            max_current = max_current + arr[i];
        }

        if (max_current > max_global) {
            max_global = max_current;
        }
    }

    printf("%d", max_global);

    return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Given an array of integers, determine whether there exists a majority element in the array.

A majority element is defined as an element that appears more than $n/2$ times in the array.

Your task is to implement a function `majorityElement()` that takes the input array and returns the majority element if it exists. If no such element exists, return -1.

Formula

An element is considered a majority element if:

$\text{frequency}(\text{element}) > n/2$

where n is the total number of elements in the array.

Input Format

The first line of input contains an integer n , representing the number of elements in the array.

The second line contains n space-separated integers $a_1 a_2 a_3 \dots a_n$ representing the array elements.

Output Format

Print the majority element if it exists. Otherwise, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9

3 3 4 2 4 4 2 4 4

Output: 4

Answer

```
// You are using GCC
#include <stdio.h>
```

```

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Boyer-Moore Voting Algorithm
    int candidate = 0;
    int count = 0;

    for (int i = 0; i < n; i++) {
        if (count == 0) {
            candidate = arr[i];
            count = 1;
        } else if (arr[i] == candidate) {
            count++;
        } else {
            count--;
        }
    }

    // Verify candidate
    count = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] == candidate) {
            count++;
        }
    }

    if (count > n / 2) {
        printf("%d", candidate);
    } else {
        printf("-1");
    }

    return 0;
}

```


Status : Correct

Marks : 10/10

4. Problem Statement

Aarav is analyzing temperature fluctuations recorded over several days. The temperature changes are stored in an array of integers. He wants to identify a contiguous period of days such that the product of the temperature changes during that period is maximized.

Given an array of integers, determine the maximum product that can be obtained from any contiguous subarray containing at least one element.

Your task is to compute this value efficiently.

Formula

To correctly handle both positive and negative values, track both the maximum and minimum product ending at each index.

If the current element is negative, swap the previous maximum and minimum values before updating.

At each index i :

$\text{currentMax} = \max(\text{arr}[i], \text{arr}[i] \times \text{currentMax})$

$\text{currentMin} = \min(\text{arr}[i], \text{arr}[i] \times \text{currentMin})$

The final answer is the maximum value obtained among all currentMax values.

Input Format

The first line contains an integer n , representing the number of days.

The second line contains n space-separated integers representing the temperature change factors.

Output Format

The first line contains an integer n , representing the number of days.

The second line contains n space-separated integers representing the temperature change factors.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
2 3 -2 4 -1 5

Output: 240

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int main() {
    int n;
    scanf("%d", &n);

    long long arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%lld", &arr[i]);
    }

    long long maxProd = arr[0];
    long long minProd = arr[0];
    long long result = arr[0];

    for (int i = 1; i < n; i++) {
        long long x = arr[i];

        if (x < 0) {
            long long temp = maxProd;
            maxProd = minProd;
            minProd = temp;
        }

        if (x > maxProd * x)
            maxProd = x;
```

```

else
    maxProd = maxProd * x;

    if (x < minProd * x)
        minProd = x;
    else
        minProd = minProd * x;

    if (maxProd > result)
        result = maxProd;
}

printf("%lld", result);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Sidhant is a logistics manager responsible for planning delivery routes for a fleet of trucks. He has a list of cities, and the distances between every pair of cities are given in the form of a matrix.

Each truck must start from a central warehouse (city 0), visit every other city exactly once, and return back to the warehouse. The goal is to minimize the total travel distance.

Sidhant needs to determine the minimum distance required to complete the route and also print the optimal path taken.

Input Format

The first line contains an integer n , representing the number of cities.

The next n lines contain n space-separated integers, representing the distance matrix.

Output Format

The first line of output prints the minimum travel distance.

In the next line, print the optimal path starting and ending at city 0.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: 80

0 1 3 2 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX 15
```

```
#define INF 1000000000
```

```
int n;
```

```
int dist[MAX][MAX];
```

```
int dp[1 << MAX][MAX];
```

```
int parent[1 << MAX][MAX];
```

```
int min(int a, int b) {
```

```
    return (a < b) ? a : b;
```

```
}
```

```
int tsp(int mask, int pos) {
```

```
    if (mask == (1 << n) - 1) {
```

```
        return dist[pos][0];
```

```
    }
```

```
    if (dp[mask][pos] != -1)
```

```
        return dp[mask][pos];
```

```

int ans = INF;

for (int city = 0; city < n; city++) {
    if ((mask & (1 << city)) == 0) {
        int newAns = dist[pos][city] + tsp(mask | (1 << city), city);

        if (newAns < ans) {
            ans = newAns;
            parent[mask][pos] = city;
        }
    }
}

return dp[mask][pos] = ans;
}

```

```

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &dist[i][j]);
        }
    }

    for (int i = 0; i < (1 << n); i++) {
        for (int j = 0; j < n; j++) {
            dp[i][j] = -1;
            parent[i][j] = -1;
        }
    }
}

```

```

int minCost = tsp(1, 0);

```

```

printf("%d\n", minCost);

```

```

// Reconstruct path

```

```

int mask = 1, pos = 0;

```

```

printf("0 ");

```

```

while (1) {

```

```
int next = parent[mask][pos];  
if (next == -1) break;  
  
printf("%d ", next);  
mask |= (1 << next);  
pos = next;  
}  
  
printf("0");  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 7_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 39

Section 1 : Coding

1. Problem Statement

Mary is planning to build a railway network to connect cities efficiently. She needs a program that, given the number of cities and the segments between them, determines the minimum cost to connect all cities using an undirected weighted graph.

Implement a solution using Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the city network. If it is not possible to connect all cities, output an appropriate message

Example

Input

3

3

0 1 4

1 2 5

0 2 3

Output

7

Explanation

The input specifies 3 cities and 3 track segments with their corresponding weights.

The track segments are:

Track Segment 1: Connects city 0 and city 1, with a weight of 4.

Track Segment 2: Connects city 1 and city 2 with a weight of 5.

Track Segment 3: Connects city 0 and city 2 with a weight of 3.

The algorithm finds the minimum cost to build railway tracks connecting all cities. In this case, the minimum spanning tree includes track segment 3 (0-2 with weight 3) and track segment 1 (0-1 with weight 4). The total cost is $3 + 4 = 7$.

Input Format

The first line of input contains an integer `num_cities`, representing the number of cities (numbered 0 to `num_cities - 1`).

The second line contains an integer `num_segments`, representing the number of undirected segments (edges) between cities.

The next `num_segments` lines each contain three space-separated integers: source, destination, and weight, representing an undirected segment between city source and city destination with the given cost.

Output Format

The first line of output prints an integer representing the minimum cost to connect all cities.

If it is not possible to connect all the cities, the output prints 'It is not possible to connect all cities' (without a period).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

3

0 1 4

1 2 5

0 2 3

Output: 7

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int u, v, w;  
} Edge;
```

```
int parent[20];
```

```
int find(int i) {  
    if (parent[i] == i)  
        return i;  
    return parent[i] = find(parent[i]);  
}
```

```
void unionSet(int a, int b) {  
    parent[find(a)] = find(b);  
}
```

```
int compare(const void *a, const void *b) {  
    Edge *e1 = (Edge *)a;  
    Edge *e2 = (Edge *)b;  
    return e1->w - e2->w;  
}
```

```

int main() {
    int n, m;
    scanf("%d", &n);
    scanf("%d", &m);

    Edge edges[m];

    for (int i = 0; i < m; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    // Sort edges by weight
    qsort(edges, m, sizeof(Edge), compare);

    // Initialize DSU
    for (int i = 0; i < n; i++)
        parent[i] = i;

    int cost = 0;
    int count = 0;

    for (int i = 0; i < m; i++) {
        int u = edges[i].u;
        int v = edges[i].v;

        if (find(u) != find(v)) {
            unionSet(u, v);
            cost += edges[i].w;
            count++;
        }
    }

    // Check if all cities are connected
    int root = find(0);
    for (int i = 1; i < n; i++) {
        if (find(i) != root) {
            printf("It is not possible to connect all cities");
            return 0;
        }
    }
}

```

```
    printf("%d", cost);  
    return 0;  
}
```

Status : Partially correct

Marks : 9/10

2. Problem Statement

Maya, a project planner, must connect multiple office branches using network cables while keeping the total cost within a given budget. Each possible connection between two branches has an associated cost.

Write a program to find the Minimum Spanning Tree (MST) of the branch network using Kruskal's Algorithm on an undirected weighted graph. If the total cost of the MST is within the given budget, display the cost. Otherwise, indicate that it is not possible.

It is guaranteed that the graph is connected in all test cases

Input Format

The first line contains three space-separated integers V, E, and budget, representing the number of branches (vertices, numbered 1 to V), the number of possible connections (edges), and the total budget respectively.

Each of the next E lines contains three space-separated integers src, dest, and weight, representing an undirected connection between branch src and branch dest with cost weight.

Output Format

If the MST cost is within the budget, the output prints:

'Minimum Cost Spanning Tree within the budget: X'

where X is an integer representing the total minimum cost.

Otherwise, the output prints:

'A valid MST within the budget is not possible.'

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 3 5

1 2 2

2 3 3

3 1 4

Output: Minimum Cost Spanning Tree within the budget: 5

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int u, v;  
    int w;  
} Edge;
```

```
int parent[105];
```

```
int find(int x) {  
    if (parent[x] != x)  
        parent[x] = find(parent[x]);  
    return parent[x];  
}
```

```
void unionSet(int a, int b) {  
    parent[find(a)] = find(b);  
}
```

```
int compare(const void *a, const void *b) {  
    Edge *e1 = (Edge *)a;  
    Edge *e2 = (Edge *)b;  
    return e1->w - e2->w;  
}
```

```
int main() {  
    int V, E;
```

```

long long budget;

scanf("%d %d %lld", &V, &E, &budget);

Edge edges[E];

for (int i = 0; i < E; i++) {
    scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
}

// Sort edges by weight
qsort(edges, E, sizeof(Edge), compare);

// Initialize DSU
for (int i = 1; i <= V; i++) {
    parent[i] = i;
}

long long cost = 0;
int count = 0;

for (int i = 0; i < E && count < V - 1; i++) {
    int u = edges[i].u;
    int v = edges[i].v;

    if (find(u) != find(v)) {
        unionSet(u, v);
        cost += edges[i].w;
        count++;
    }
}

if (cost <= budget) {
    printf("Minimum Cost Spanning Tree within the budget: %lld", cost);
} else {
    printf("A valid MST within the budget is not possible.");
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Sita is planning to connect multiple locations in her city with roads. Each possible road between two locations has a construction distance (in kilometres). The city is represented as an undirected weighted graph using an adjacency matrix, where a value of 0 indicates no direct road between two locations (locations are numbered 0 to numLocations - 1).

To minimise the overall construction distance, she decides to use Prim's Algorithm to construct the Minimum Spanning Tree (MST). Help her find the MST edges and the total minimum distance.

Input Format

The first line of input contains an integer numLocations, representing the number of locations (numbered 0 to numLocations - 1).

The next numLocations lines each contain numLocations space-separated integers representing the adjacency matrix, where graph[i][j] is the distance between location i and location j. A value of 0 indicates no direct road between those two locations.

Output Format

The first line of output prints: 'Location\tDistance'

Each of the next numLocations - 1 lines prints the MST edge in the format:

'parent -> child \t\t\t dist km'

where parent is the parent location number, child is the child location number, and dist is the edge weight (integer) in kilometres.

The last line of output prints: 'Minimum total distance: totalCost km'

where totalCost is an integer.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2

0 5

5 0

Output: Location Distance

0 -> 1 5 km

Minimum total distance: 5 km

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX 10
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int graph[MAX][MAX];
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            scanf("%d", &graph[i][j]);
```

```
        }
```

```
    }
```

```
    int parent[MAX];
```

```
    int key[MAX];
```

```
    int mstSet[MAX];
```

```
    for (int i = 0; i < n; i++) {
```

```
        key[i] = INT_MAX;
```

```
        mstSet[i] = 0;
```

```
        parent[i] = -1;
```

```
    }
```

```
    key[0] = 0;
```

```
    for (int count = 0; count < n - 1; count++) {
```

```

int min = INT_MAX, u = -1;
for (int v = 0; v < n; v++) {
    if (!mstSet[v] && key[v] < min) {
        min = key[v];
        u = v;
    }
}

mstSet[u] = 1;

for (int v = 0; v < n; v++) {
    if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v]) {
        parent[v] = u;
        key[v] = graph[u][v];
    }
}

int totalCost = 0;

printf("Location\tDistance\n");

for (int i = 1; i < n; i++) {
    printf("%d -> %d\t\t\t %d km\n", parent[i], i, graph[i][parent[i]]);
    totalCost += graph[i][parent[i]];
}

printf("Minimum total distance: %d km", totalCost);

return 0;
}

```

Status : Wrong

Marks : 0/10

4. Problem Statement

Riya works at a data tools company that needs a compact representation of frequently used characters for fast transmission. She decides to use Huffman Coding, a lossless compression technique that assigns shorter bit-strings to more frequent symbols and longer bit-strings to less frequent

ones.

The Huffman codes should be printed in the order that leaf nodes are encountered during a left-to-right traversal of the constructed Huffman tree.

Write a program that builds a Huffman tree from a list of symbols with their frequencies and prints the Huffman codes for each symbol.

Input Format

The first line contains an integer n , representing the number of distinct symbols.

Each of the next n lines contains a single printable character (the symbol) followed by a space and a positive integer (the frequency), representing the symbol and its occurrence count.

Output Format

The output prints n lines, one per symbol, in the order that leaf nodes are encountered during a left-to-right traversal of the Huffman tree (left child before right child at every internal node).

Each line prints the symbol followed by a colon, a space, and its Huffman code (a binary string of 0s and 1s):

`<symbol>: <binary_code>`

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2

a 1

b 2

Output: a: 0

b: 1

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 260

typedef struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
} Node;

typedef struct {
    Node* arr[MAX];
    int size;
} Heap;

Node* createNode(char ch, int freq) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->ch = ch;
    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}

/* Compare for deterministic ordering */
int isSmaller(Node* a, Node* b) {
    if (a->freq != b->freq)
        return a->freq < b->freq;
    return a->ch < b->ch;
}

void swap(Node** a, Node** b) {
    Node* t = *a;
    *a = *b;
    *b = t;
}

void heapify(Heap* h, int i) {
    int smallest = i;
```

```
int l = 2*i + 1;
int r = 2*i + 2;
```

```
if (l < h->size && isSmaller(h->arr[l], h->arr[smallest]))
    smallest = l;
if (r < h->size && isSmaller(h->arr[r], h->arr[smallest]))
    smallest = r;
```

```
if (smallest != i) {
    swap(&h->arr[i], &h->arr[smallest]);
    heapify(h, smallest);
}
```

```
Node* extractMin(Heap* h) {
    Node* root = h->arr[0];
    h->arr[0] = h->arr[h->size-1];
    heapify(h, 0);
    return root;
}
```

```
void insertHeap(Heap* h, Node* node) {
    int i = h->size++;
    h->arr[i] = node;

    while (i > 0) {
        int p = (i - 1) / 2;
        if (isSmaller(h->arr[p], h->arr[i])) break;
        swap(&h->arr[p], &h->arr[i]);
        i = p;
    }
}
```

```
Node* buildTree(Heap* h) {
    while (h->size > 1) {
        Node* l = extractMin(h);
        Node* r = extractMin(h);

        Node* parent = createNode('*', l->freq + r->freq);
        parent->left = l;
        parent->right = r;
    }
}
```

```
        insertHeap(h, parent);
    }
    return extractMin(h);
}
```

```
char code[260];
char result[260][260];
```

```
void buildCodes(Node* root, int depth) {
    if (!root) return;

    if (!root->left && !root->right) {
        code[depth] = '\0';
        strcpy(result[(unsigned char)root->ch], code);
        return;
    }
```

```
    code[depth] = '0';
    buildCodes(root->left, depth + 1);
```

```
    code[depth] = '1';
    buildCodes(root->right, depth + 1);
}
```

```
void printLeftToRight(Node* root) {
    if (!root) return;

    if (!root->left && !root->right) {
        printf("%c: %s\n", root->ch, result[(unsigned char)root->ch]);
        return;
    }
```

```
    printLeftToRight(root->left);
    printLeftToRight(root->right);
}
```

```
int main() {
    int n;
    scanf("%d", &n);

    Heap h;
    h.size = 0;
```

```

for (int i = 0; i < n; i++) {
    char ch;
    int freq;
    scanf(" %c %d", &ch, &freq);
    insertHeap(&h, createNode(ch, freq));
}

Node* root = buildTree(&h);

buildCodes(root, 0);
printLeftToRight(root);

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

In a data science class, Professor Huffman demonstrates how encoded messages can be decoded using a pre-built Huffman tree.

Given a Huffman code table (mapping each character to its binary code) and an encoded binary message, decode the message back to its original text by traversing the Huffman tree.

The code table is guaranteed to be a valid prefix-free Huffman code.

The encoded message consists only of '0' and '1' characters, and every bit maps to exactly one symbol in the Huffman tree

The tree is defined as follows:

Internal nodes are represented by "*". The left edge (0) leads to the symbol "A". The right edge (1) leads to the symbol "B".

This means:

"0" A "1" B

Your task is to decode a given encoded message (a string of 0s and 1s) back into its original form consisting of "A" and "B".

Input Format

The first line contains an integer n, representing the number of symbols in the code table.

Each of the next n lines contains a character and its Huffman code (binary string), separated by a space.

The last line contains the encoded binary message (a string of 0s and 1s) to be decoded

Output Format

The output prints the decoded message as a sequence of characters ('A' or 'B'), followed by a newline.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 11100011100

Output: BBBAABBBA

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char s[105];
    scanf("%s", s);

    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == '0')
            printf("A");
        else if (s[i] == '1')
            printf("B");
    }
    return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 10_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 40.5

Section 1 : Coding

1. Problem Statement

Meera, the director of a national disaster response team, must deploy three specialized rescue units to three critical disaster sites. Each rescue unit has a different response time (in minutes) to reach each site due to distance, terrain, and accessibility constraints.

However, Meera has an additional operational rule:

Rescue Unit 1 cannot be assigned to Site 3 due to equipment limitations. Rescue Unit 2 must not be assigned to Site 1 because of route restrictions.

Meera decides to use the Branch and Bound technique to explore only valid and efficient assignments while respecting all constraints.

Your task is to determine the total response time of the best valid

deployment plan that satisfies all given restrictions.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3×3 response time matrix.

Each cell (i, j) represents the response time (in minutes) if the *i*th rescue unit is deployed to the *j*th disaster site.

Output Format

The output should display the total response time in the following exact format: "Total Response Time: X", where X is the total response time of the best valid deployment plan.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 12 25 40
30 14 28
16 18 11

Output: Total Response Time: 37

Answer

```
// You are using GCC
#include <stdio.h>
#include <limits.h>
```

```
int cost[3][3];
int best = INT_MAX;
```

```
int used[3];
```

```
void solve(int unit, int current) {
    if (unit == 3) {
        if (current < best)
            best = current;
        return;
    }
}
```

```

for (int site = 0; site < 3; site++) {

    // constraints
    if ((unit == 0 && site == 2) || // Unit 1 not Site 3
        (unit == 1 && site == 0)) // Unit 2 not Site 1
        continue;

    if (!used[site]) {
        used[site] = 1;
        solve(unit + 1, current + cost[unit][site]);
        used[site] = 0;
    }
}

int main() {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    for (int i = 0; i < 3; i++)
        used[i] = 0;

    solve(0, 0);

    printf("Total Response Time: %d", best);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Captain Aidan, a daring pirate, has discovered an ancient treasure cave filled with valuable relics. However, the cave is protected by an ancient spell that limits the weight of the treasure he can carry. Aidan has a magic knapsack with a maximum weight capacity of W . Inside the cave, he finds

n different treasures, each with a specific weight and value.

Since he can only take each treasure once, Aidan must carefully choose which items to take to maximize the total value without exceeding the weight limit of his knapsack.

Help Captain Aidan pick the most valuable treasures while staying within the weight limit using branch and bound technique

Input Format

The first line contains two integers n (the number of items) and W (the capacity of the knapsack), separated by a space.

The second line contains n integers, where the i-th integer represents the weight of the i-th item.

The third line contains n integers, where the i-th integer represents the value of the i-th item

Output Format

The output prints the single integer representing the maximum value obtained by filling the knapsack with the given set of items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 60
10 20 30 40 50
60 100 120 140 180
Output: 280

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

#define MAX 100
```

```
typedef struct {  
    int weight;  
    int value;  
    float ratio;  
} Item;
```

```
typedef struct Node {  
    int level;  
    int profit;  
    int weight;  
    float bound;  
} Node;
```

```
int n, W;  
Item items[MAX];
```

```
// Sort by value/weight ratio (descending)  
int compare(const void *a, const void *b) {  
    Item *i1 = (Item *)a;  
    Item *i2 = (Item *)b;  
    return (i2->ratio > i1->ratio) - (i2->ratio < i1->ratio);  
}
```

```
// Compute upper bound  
float bound(Node u) {  
    if (u.weight >= W)  
        return 0;
```

```
    float profit_bound = u.profit;  
    int j = u.level + 1;  
    int totweight = u.weight;
```

```
    while (j < n && totweight + items[j].weight <= W) {  
        totweight += items[j].weight;  
        profit_bound += items[j].value;  
        j++;  
    }
```

```
    if (j < n)  
        profit_bound += (W - totweight) * items[j].ratio;
```

```
    return profit_bound;
}
```

```
// Simple queue for BFS-style node handling
Node queue[MAX];
int front = 0, rear = 0;
```

```
void push(Node v) {
    queue[rear++] = v;
}
```

```
Node pop() {
    return queue[front++];
}
```

```
int isEmpty() {
    return front == rear;
}
```

```
int knapsack() {
    Node u, v;
    int maxProfit = 0;

    u.level = -1;
    u.profit = 0;
    u.weight = 0;
    u.bound = bound(u);

    push(u);
```

```
    while (!isEmpty()) {
        u = pop();
```

```
        if (u.bound > maxProfit) {
            // take next item
            v.level = u.level + 1;
```

```
            if (v.level < n) {
                // Include item
                v.weight = u.weight + items[v.level].weight;
                v.profit = u.profit + items[v.level].value;
```

```

        if (v.weight <= W && v.profit > maxProfit)
            maxProfit = v.profit;

        v.bound = bound(v);

        if (v.bound > maxProfit)
            push(v);

        // Exclude item
        v.weight = u.weight;
        v.profit = u.profit;
        v.bound = bound(v);

        if (v.bound > maxProfit)
            push(v);
    }
}

return maxProfit;
}

```

```

int main() {
    scanf("%d %d", &n, &W);

    int w[MAX], v[MAX];

    for (int i = 0; i < n; i++)
        scanf("%d", &w[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &v[i]);

    for (int i = 0; i < n; i++) {
        items[i].weight = w[i];
        items[i].value = v[i];
        items[i].ratio = (float)v[i] / w[i];
    }

    // sort by ratio
    qsort(items, n, sizeof(Item), compare);
}

```

```
printf("%d", knapsack());  
return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Commander Vikram is preparing a spacecraft for an interplanetary research mission. The spacecraft has a limited cargo capacity, so he must carefully decide which scientific equipment modules to load.

Each module has a scientific importance value and a specific weight. Commander Vikram wants to load the spacecraft in such a way that the total scientific value of the selected modules is maximized without exceeding the cargo weight limit.

To solve this efficiently, he uses the Branch and Bound technique applied to the 0/1 Knapsack problem, which explores possible combinations of equipment and eliminates options that cannot lead to better solutions.

Your task is to help Commander Vikram determine the maximum achievable value and the selected module weights that should be loaded into the spacecraft.

Note: The weights must be printed in reverse order of the input, meaning the algorithm starts evaluating items from the last item and prints the selected weights first.

Input Format

The first line contains an integer n , representing the number of equipment modules available.

The second line contains n space-separated integers $val[i]$, representing the scientific value of each module.

The third line contains n space-separated integers $wt[i]$, representing the weight of each module.

The fourth line contains an integer W, representing the maximum cargo capacity of the spacecraft.

Output Format

The first line output displays "Maximum value: " followed by an integer, representing the maximum value that can be achieved.

The second line output displays "Selected weights: " followed by space-separated integers, printed in reverse order of the input.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 3

60 100 120

10 20 30

50

Output: Maximum value: 220

Selected weights: 30 20

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#define MAX 20
```

```
int n, W;
```

```
int val[MAX], wt[MAX];
```

```
int bestValue = 0;
```

```
int bestSet[MAX], currSet[MAX];
```

```
int finalSet[MAX];
```

```
void copySet(int src[], int dest[]) {
```

```
    for (int i = 0; i < n; i++)
```

```
        dest[i] = src[i];
```

```
}
```



```
// Bound function (greedy fractional knapsack bound)
```

```
float bound(int level, int profit, int weight) {
```

```
    int totalWeight = weight;
```

```
    float result = profit;
```

```
    for (int i = level + 1; i < n; i++) {
```

```
        if (totalWeight + wt[i] <= W) {
```

```
            totalWeight += wt[i];
```

```
            result += val[i];
```

```
        } else {
```

```
            result += (float)(W - totalWeight) * val[i] / wt[i];
```

```
            break;
```

```
        }
```

```
    }
```

```
    return result;
```

```
}
```

```
// Branch and Bound DFS
```

```
void knapsack(int level, int profit, int weight) {
```

```
    if (weight > W)
```

```
        return;
```

```
    if (profit > bestValue) {
```

```
        bestValue = profit;
```

```
        copySet(currSet, bestSet);
```

```
    }
```

```
    if (level == n - 1)
```

```
        return;
```

```
    if (bound(level, profit, weight) <= bestValue)
```

```
        return;
```

```
    // Include item
```

```
    currSet[level + 1] = 1;
```

```
    knapsack(level + 1, profit + val[level + 1], weight + wt[level + 1]);
```

```
    // Exclude item
```

```
    currSet[level + 1] = 0;
```

```
    knapsack(level + 1, profit, weight);
```

```
}
```

```

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    scanf("%d", &W);

    for (int i = 0; i < n; i++)
        currSet[i] = 0;

    knapsack(-1, 0, 0);

    // Print result
    printf("Maximum value: %d\n", bestValue);
    printf("Selected weights: ");

    // Reverse order printing (important requirement)
    for (int i = n - 1; i >= 0; i--) {
        if (bestSet[i]) {
            printf("%d ", wt[i]);
        }
    }

    return 0;
}

```

Status : Partially correct

Marks : 2/10

4. Problem Statement

You are tasked with solving the Travelling Salesman Problem (TSP) for a given set of cities.

Given the distances between pairs of cities, your goal is to find the shortest route that visits each city exactly once and returns to the starting city.

Note: Solve it using Branch and Bound approach.

Input Format

The first line of input contains an integer n, representing the number of cities.

The following n lines each contain n space-separated integers, representing the distances between pairs of cities.

The distance between city i and city j is given at the ith row and jth column of the matrix.

Output Format

The first line displays "The Path is: " followed by the sequence of cities visited in the shortest path,

separated by "--->".

The second line displays "Minimum cost is X", where X is the minimum cost of the shortest path found.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 4 1 3

4 0 2 1

1 2 0 5

3 1 5 0

Output: The Path is: 1--->3--->2--->4--->1

Minimum cost is 7

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX 10
```

```
int n;  
int cost[MAX][MAX];
```

```
int finalPath[MAX];  
int currPath[MAX];  
int visited[MAX];
```

```
int finalCost = INT_MAX;
```

```
int firstMin(int i) {  
    int min = INT_MAX;  
    for (int j = 0; j < n; j++) {  
        if (i != j && cost[i][j] < min)  
            min = cost[i][j];  
    }  
    return min;  
}
```

```
int secondMin(int i) {  
    int first = INT_MAX, second = INT_MAX;  
  
    for (int j = 0; j < n; j++) {  
        if (i == j)  
            continue;  
  
        if (cost[i][j] <= first) {  
            second = first;  
            first = cost[i][j];  
        } else if (cost[i][j] <= second) {  
            second = cost[i][j];  
        }  
    }  
  
    return second;  
}
```

```
void copyPath() {  
    for (int i = 0; i < n; i++)  
        finalPath[i] = currPath[i];  
}
```

```

void TSP(int level, int currCost, int bound) {
    if (level == n) {
        if (cost[currPath[level - 1]][currPath[0]] != 0) {
            int total = currCost + cost[currPath[level - 1]][currPath[0]];
            if (total < finalCost) {
                finalCost = total;
                copyPath();
            }
        }
    }
    return;
}

```

```

for (int i = 0; i < n; i++) {
    if (!visited[i] && cost[currPath[level - 1]][i]) {

        int tempBound = bound;
        int tempCost = currCost;

        currCost += cost[currPath[level - 1]][i];

        if (level == 1)
            bound -= (firstMin(currPath[level - 1]) + firstMin(i)) / 2;
        else
            bound -= (secondMin(currPath[level - 1]) + firstMin(i)) / 2;

        if (bound + currCost < finalCost) {
            currPath[level] = i;
            visited[i] = 1;

            TSP(level + 1, currCost, bound);
        }

        currCost = tempCost;
        bound = tempBound;

        for (int j = 0; j < n; j++)
            visited[j] = 0;

        for (int j = 0; j < level; j++)
            visited[currPath[j]] = 1;
    }
}

```

```

}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    int bound = 0;

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    for (int i = 0; i < n; i++)
        bound += (firstMin(i) + secondMin(i));

    bound = bound / 2;

    currPath[0] = 0;
    visited[0] = 1;

    TSP(1, 0, bound);

    printf("The Path is: ");

    for (int i = 0; i < n; i++)
        printf("%d--->", finalPath[i] + 1);

    printf("1\n");

    printf("Minimum cost is %d", finalCost);

    return 0;
}

```

Status : Partially correct

Marks : 8.5/10

5. Problem Statement

You are a logistics manager responsible for planning delivery routes for a

fleet of vehicles. Each vehicle needs to visit a set of locations to deliver goods, and the cost of traveling between locations is known.

Your goal is to find the shortest route for each vehicle, ensuring that each location is visited exactly once before returning to the starting point.

Note: Solve it using Branch and Bound approach.

Input Format

The first line contains an integer n , the number of locations.

The next n lines contain n integers each, representing the cost matrix of traveling between locations.

The i th integer on the j th line represents the cost of traveling from location i to location j .

Output Format

The first line of output displays "Shortest Path: " followed by the shortest path that visits all cities.

The second line of output displays "Minimum cost is " followed by the minimum cost.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

0 10 8 9 7

10 0 10 5 6

8 10 0 8 9

9 5 8 0 6

7 6 9 6 0

Output: Shortest Path: 1 3 4 2 5 1

Minimum cost is 34

Answer

// You are using GCC

```
#include <stdio.h>
#include <limits.h>
```

```
#define MAX 10
```

```
int n;
int cost[MAX][MAX];
int visited[MAX];
```

```
int finalPath[MAX];
int currPath[MAX];
```

```
int finalCost = INT_MAX;
```

```
int firstMin(int i) {
    int min = INT_MAX;
    for (int j = 0; j < n; j++) {
        if (cost[i][j] && cost[i][j] < min)
            min = cost[i][j];
    }
    return min;
}
```

```
int secondMin(int i) {
    int first = INT_MAX, second = INT_MAX;
```

```
    for (int j = 0; j < n; j++) {
        if (i == j || cost[i][j] == 0)
            continue;

        if (cost[i][j] <= first) {
            second = first;
            first = cost[i][j];
        } else if (cost[i][j] <= second) {
            second = cost[i][j];
        }
    }
    return second;
}
```

```
void copyPath() {
    for (int i = 0; i < n; i++)
```



```

    finalPath[i] = currPath[i];
}

void TSP(int currBound, int currWeight, int level) {
    if (level == n) {
        if (cost[currPath[level - 1]][currPath[0]] != 0) {
            int totalCost = currWeight + cost[currPath[level - 1]][currPath[0]];

            if (totalCost < finalCost) {
                copyPath();
                finalCost = totalCost;
            }
        }
    }
    return;
}

```

```

for (int i = 0; i < n; i++) {
    if (cost[currPath[level - 1]][i] && !visited[i]) {

        int temp = currBound;
        currWeight += cost[currPath[level - 1]][i];

        if (level == 1)
            currBound -= (firstMin(currPath[level - 1]) + firstMin(i)) / 2;
        else
            currBound -= (secondMin(currPath[level - 1]) + firstMin(i)) / 2;

        if (currBound + currWeight < finalCost) {
            currPath[level] = i;
            visited[i] = 1;

            TSP(currBound, currWeight, level + 1);
        }

        currWeight -= cost[currPath[level - 1]][i];
        currBound = temp;

        for (int j = 0; j < n; j++)
            visited[j] = 0;

        for (int j = 0; j < level; j++)
            visited[currPath[j]] = 1;
    }
}

```

```

    }
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    int currBound = 0;

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    for (int i = 0; i < n; i++)
        currBound += (firstMin(i) + secondMin(i));

    currBound = (currBound == 1) ? currBound : currBound / 2;

    visited[0] = 1;
    currPath[0] = 0;

    TSP(currBound, 0, 1);

    printf("Shortest Path: ");
    for (int i = 0; i < n; i++)
        printf("%d ", finalPath[i] + 1);
    printf("1\n");

    printf("Minimum cost is %d", finalCost);

    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 12_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 29

Section 1 : Coding

1. Problem Statement

Raj is developing financial software that must identify all non-empty subsets of a given list of expenses whose total sum equals a specified target value.

Each expense can either be included or excluded from a subset. The same expense cannot be used more than once in a subset.

Your task is to print all such valid subsets. If no valid non-empty subset exists whose sum equals the target value, print:

No subset matches the target sum

This problem demonstrates how the number of possible subsets increases exponentially when using brute-force approaches.

Formula

Subset Sum = sum of selected elements in the subset

A subset is considered valid if:

$\text{sum}(\text{subset elements}) = \text{target}$

Input Format

The first line contains an integer n , representing the number of expenses.

The second line contains n space-separated integers representing the expense values.

The third line contains an integer t , representing the target sum.

Output Format

Print each valid non-empty subset whose elements sum to t , one subset per line.

Elements of a subset must be printed as space-separated integers.

If no such subset exists, print:

No subset matches the target sum

Refer to the sample output for formatting.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

3

Output: 1 2

3

Answer

```
// You are using GCC
#include <stdio.h>
```

```
#define MAX 20
```

```
int n, t;
int arr[MAX];
int subset[MAX];
int found = 0;
```

```
void printSubset(int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", subset[i]);
    }
    printf("\n");
}
```

```
void backtrack(int start, int sum, int idx) {
    if (start != 0 && sum == t) {
        printSubset(idx);
        found = 1;
    }
}
```

```
for (int i = start; i < n; i++) {
    subset[idx] = arr[i];
    backtrack(i + 1, sum + arr[i], idx + 1);
}
```

```
int main() {
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
```

```
    scanf("%d", &t);
```

```
    backtrack(0, 0, 0);
```

```
    if (!found)
        printf("No subset matches the target sum");
```

```
} return 0;
```

Status : Partially correct

Marks : 9/10

2. Problem Statement

Sanjana is a cybersecurity analyst responsible for securing communication in a corporate network. The network consists of multiple computers (nodes) connected by communication links (edges).

To ensure secure communication, Sanjana wants to select a group of computers such that every pair of selected computers is directly connected to each other. This ensures a fully trusted group where all members can communicate securely without intermediaries. Such a group is called a clique.

Sanjana is given a list of connections in the network, and she needs to determine whether there exists a clique of size k .

Input Format

The first line contains two integers n and m , representing the number of computers (nodes) and connections (edges).

The next m lines each contain two integers u and v , representing a connection between nodes u and v .

The last line contains an integer k , representing the required clique size.

Output Format

If a clique of size k exists, the output prints: "Clique Found: $v_1 v_2 v_3 \dots$ "

Otherwise, the output prints: "No clique of given size exists"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 6

0 1

0 2

1 2

13

23

34

3

Output: Clique Found: 0 1 2

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int n, m, k;
```

```
int adj[20][20];
```

```
int found = 0;
```

```
int result[20];
```

```
int isClique(int subset[], int size) {
```

```
for (int i = 0; i < size; i++) {
```

```
for (int j = i + 1; j < size; j++) {
```

```
if (!adj[subset[i]][subset[j]]) {
```

```
return 0;
```

}

}

}

```
return 1;
```

}

```
void printClique(int subset[], int size) {
```

```
printf("Clique Found: ");
```

```
for (int i = 0; i < size; i++) {
```

```
printf("%d ", subset[i]);
```

$$\}$$
$$\}$$

```
void generate(int start, int subset[], int size) {
```

```
if (found) return;
```

```

    if (size == k) {
        if (isClique(subset, size)) {
            printClique(subset, size);
            found = 1;
        }
        return;
    }

    for (int i = start; i < n; i++) {
        subset[size] = i;
        generate(i + 1, subset, size + 1);
    }
}

int main() {
    scanf("%d %d", &n, &m);

    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        adj[v][u] = 1;
    }

    scanf("%d", &k);

    int subset[20];
    generate(0, subset, 0);

    if (!found) {
        printf("No clique of given size exists");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rehan, a logistics manager, is packing valuable items into two separate

containers. Each item has a specific value, and he wants both containers to carry the same total value to ensure balance and fairness. He needs to decide if it's possible to divide the items this way and print one such valid division if it exists.

This problem is based on a classic NP-complete problem known as the Partition Problem, where finding a solution may require checking all combinations in the worst case.

Help him write a program to solve this problem.

Input Format

The first line of input consists of an integer n , representing the number of items.

The second line contains n space-separated integers, where each integer represents the value of an item.

Output Format

If a valid equal partition exists, print:

- Subset 1: x_1 x_2 x_3 ...
- Subset 2: y_1 y_2 y_3 ...

If no such partition exists, it prints "No valid equal partition exists."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

1 5 11 5

Output: Subset 1: 1 5 5

Subset 2: 11

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int n;  
int arr[16];  
int total;  
int found = 0;
```

```
int subset1[16], subset2[16];
```

```
void printResult() {  
    printf("Subset 1: ");  
    for (int i = 0; i < n; i++) {  
        if (subset1[i]) {  
            printf("%d ", arr[i]);  
        }  
    }  
}
```

```
    printf("\nSubset 2: ");  
    for (int i = 0; i < n; i++) {  
        if (!subset1[i]) {  
            printf("%d ", arr[i]);  
        }  
    }  
}
```

```
void solve(int idx, int sum) {  
    if (found) return;
```

```
    if (sum * 2 == total) {  
        found = 1;  
        printResult();  
        return;  
    }
```

```
    if (idx == n) return;
```

```
    // include in subset1  
    subset1[idx] = 1;  
    solve(idx + 1, sum + arr[idx]);
```

```
    // exclude from subset1  
    subset1[idx] = 0;  
    solve(idx + 1, sum);  
}
```

```
int main() {
    scanf("%d", &n);

    total = 0;
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        total += arr[i];
    }

    // if total sum is odd, impossible
    if (total % 2 != 0) {
        printf("No valid equal partition exists.");
        return 0;
    }

    solve(0, 0);

    if (!found) {
        printf("No valid equal partition exists.");
    }

    return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KOPPAKULA VISHNUVARDHAN BABU koppakula

Email: vtu28078@veltech.edu.in

Roll no: vtu28078

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS7L11

VTU_DAA_Week 11_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 41.5

Section 1 : Coding

1. Problem Statement

You are given a bipartite graph represented using an edge list. The graph contains two sets of nodes tasks and processors. Each edge connects a task node to a processor node, indicating that the processor can execute that task.

Your goal is to determine the maximum matching in the bipartite graph. A matching is a set of edges such that:

Each task is assigned to at most one processor. Each processor is assigned to at most one task.

Write a program to compute the maximum number of task–processor assignments possible.

Input Format

The first line contains an integer T, representing the number of tasks.

The second line contains an integer P, representing the number of processors.

The third line contains an integer E, representing the number of edges in the bipartite graph.

The next E lines each contain two space-separated integers task and processor where this indicates that the given task can be assigned to the specified processor.

Output Format

The output prints the result in the following format: "The maximum matching is: X", where X is the maximum number of matches that can be made in the bipartite graph.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3

5

1 1

2 2

4 3

3 3

4 1

Output: The maximum matching is: 3

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 15
```

```
int T, P, E;
```

```
int graph[MAX][MAX];
```

```
int match[MAX];
```

```
int visited[MAX];
```

```
int dfs(int u) {  
    for (int v = 1; v <= P; v++) {  
        if (graph[u][v] && !visited[v]) {  
            visited[v] = 1;  
  
            if (match[v] == -1 || dfs(match[v])) {  
                match[v] = u;  
                return 1;  
            }  
        }  
    }  
    return 0;  
}
```

```
int maxMatching() {  
    memset(match, -1, sizeof(match));  
    int result = 0;  
  
    for (int u = 1; u <= T; u++) {  
        memset(visited, 0, sizeof(visited));  
        if (dfs(u)) {  
            result++;  
        }  
    }  
    return result;  
}
```

```
int main() {  
    scanf("%d", &T);  
    scanf("%d", &P);  
    scanf("%d", &E);  
  
    memset(graph, 0, sizeof(graph));  
  
    for (int i = 0; i < E; i++) {  
        int u, v;  
        scanf("%d %d", &u, &v);  
        graph[u][v] = 1;  
    }  
}
```

```
int ans = maxMatching();  
printf("The maximum matching is: %d", ans);  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Irfan is working on a project that involves managing relationships between two sets of entities, set U and set V. He wants to determine the maximum matching in a bipartite graph to optimize connections without overlap. Irfan also wants to explore different scenarios by adding a new connection (edge) between two entities and observing how it affects the maximum matching.

Your task is to compute the size of the maximum matching before and after adding a specified edge.

Note: Vertices in set U are numbered from 0 to $n-1$, and vertices in set V are numbered from 1 to m .

Input Format

The first line contains an integer n , representing the number of vertices in set U.

The second line contains an integer m , representing the number of vertices in set V.

The third line contains an integer e , representing the number of edges.

The next e lines each contain two integers u and v , representing an edge between vertex u from set U and vertex v from set V.

The last line contains two integers u and v , representing the edge to be added.

Output Format

The output prints the results in the following format:

Size of maximum matching before adding edges: X

Size of maximum matching after adding edges: Y

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

4

3

0 1

0 2

1 1

1 2

Output: Size of maximum matching before adding edges: 1

Size of maximum matching after adding edges: 1

Answer

// You are using GCC

#include <stdio.h>

#include <string.h>

#define MAX 105

int n, m, e;

int adj[MAX][MAX];

int adjSize[MAX];

int match[MAX];

int visited[MAX];

int dfs(int u) {

for (int i = 0; i < adjSize[u]; i++) {

int v = adj[u][i];

if (!visited[v]) {

visited[v] = 1;


```

        if (match[v] == -1 || dfs(match[v])) {
            match[v] = u;
            return 1;
        }
    }
}
return 0;
}

```

```

int maxMatching() {
    memset(match, -1, sizeof(match));
    int result = 0;
    for (int u = 0; u < n; u++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(u)) result++;
    }

    return result;
}

```

```

int main() {
    scanf("%d", &n);
    scanf("%d", &m);
    scanf("%d", &e);

    // read edges
    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][adjSize[u]++] = v;
    }
}

```

```

int u, v;
scanf("%d %d", &u, &v);

```

```

// compute before
int before = maxMatching();

```

```

// add new edge
adj[u][adjSize[u]++] = v;

```

```

// compute after
int after = maxMatching();

printf("Size of maximum matching before adding edges: %d\n", before);
printf("Size of maximum matching after adding edges: %d\n", after);

return 0;
}

```

Status : Partially correct

Marks : 1.5/10

3. Problem Statement

Jack wants to create a program to manage a bipartite graph, which contains two sets of vertices. He needs the program to:

Initialize the graph with a specific size, Add edges between the two sets of vertices, and Find pairings where each vertex from the first set is matched with at most one vertex from the second set, and no two vertices from the first set are matched with the same vertex from the second set.

The goal is to determine the maximum matching pairs in the bipartite graph.

Input Format

The first line contains an integer m , representing the number of vertices in the first set.

The second line contains an integer n , representing the number of vertices in the second set.

The third line contains an integer e , representing the number of edges in the graph.

The next e lines each contain two integers u and v , representing an edge between vertex u in the first set and vertex v in the second set.

Output Format

The output prints the maximum matching pairs in increasing order of u , where each pair is printed in the format: (u, v)

If there are no matching pairs, print: "No matching found"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3

5

1 1

2 2

4 3

3 3

4 1

Output: (1, 1)

(2, 2)

(3, 3)

Answer

// You are using GCC

#include <stdio.h>

#include <string.h>

#define MAX 15

int m, n, e;

int adj[MAX][MAX];

int adjSize[MAX];

int matchR[MAX]; // match for right set (v)

int matchL[MAX]; // store matched v for each u (for printing)

int visited[MAX];

int dfs(int u) {

for (int i = 0; i < adjSize[u]; i++) {

int v = adj[u][i];

```

    if (!visited[v]) {
        visited[v] = 1;

        if (matchR[v] == -1 || dfs(matchR[v])) {
            matchR[v] = u;
            matchL[u] = v;
            return 1;
        }
    }
}
return 0;
}

```

```

int maxMatching() {
    memset(matchR, -1, sizeof(matchR));
    memset(matchL, -1, sizeof(matchL));

    int result = 0;

    for (int u = 1; u <= m; u++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(u)) result++;
    }

    return result;
}

```

```

int main() {
    scanf("%d", &m);
    scanf("%d", &n);
    scanf("%d", &e);

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][adjSize[u]++] = v;
    }
}

```

```

int matched = maxMatching();

if (matched == 0) {
    printf("No matching found");
}

```

```

    return 0;
}

for (int u = 1; u <= m; u++) {
    if (matchL[u] != -1) {
        printf("(%d, %d)\n", u, matchL[u]);
    }
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Alex needs to pair students from two different classes for a project, aiming to maximize the number of compatible pairs. Each student can be paired with only one student from the other class. The compatibility between students is represented as a binary adjacency matrix.

Alex wants to write a program to find the maximum number of compatible student pairs that can be formed.

Input Format

The first line contains an integer n , representing the number of students in each group.

The second line contains an integer m , representing the number of students in the second group.

The next n lines contain m space-separated integers each, representing the adjacency matrix of student compatibility.

Output Format

The single line containing the phrase "Maximum Bipartite Matching: x " where x is an integer representing the maximum number of compatible pairs that can be formed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3

1 1 0

1 0 1

0 1 0

1 0 0

Output: Maximum Bipartite Matching: 3

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 105
```

```
int n, m;
```

```
int adj[MAX][MAX];
```

```
int matchR[MAX];
```

```
int visited[MAX];
```

```
int dfs(int u) {
```

```
    for (int v = 0; v < m; v++) {
```

```
        if (adj[u][v] && !visited[v]) {
```

```
            visited[v] = 1;
```

```
            if (matchR[v] == -1 || dfs(matchR[v])) {
```

```
                matchR[v] = u;
```

```
                return 1;
```

```
            }
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

```
int maxMatching() {
```

```
    memset(matchR, -1, sizeof(matchR));
```

```

    int result = 0;

    for (int u = 0; u < n; u++) {
        memset(visited, 0, sizeof(visited));
        if (dfs(u)) result++;
    }

    return result;
}

int main() {
    scanf("%d", &n);
    scanf("%d", &m);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    int ans = maxMatching();

    printf("Maximum Bipartite Matching: %d", ans);

    return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement:

John is developing a recommendation system for a social networking platform. The system suggests potential friends to users based on their interests and mutual connections. To enhance the user experience, you want to ensure that the recommendation algorithm considers the removal of certain users from the network due to various reasons such as account deletion or user activity.

However, he still wants to maintain effective friend suggestions even after removing these users. John plan to integrate a functionality that calculates

the maximum matching in the bipartite graph representing the user network after removing specific users and their connections.

Help him to achieve the task.

Input Format

Enter the number of vertices in partition

Enter the number of vertices in partition

Enter the adjacency matrix representation of the bipartite graph

Enter the vertex to remove

Output Format

Maximum matching after removing vertex

Sample Test Case

Input: 3

2

0 1

1 0

0 0

1

Output: Maximum matching after removing vertex 1: 1

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 15
```

```
int U, V;
```

```
int adj[MAX][MAX];
```

```
int matchR[MAX];
```

```
int visited[MAX];
```

```
int dfs(int u) {
```

```
    for (int v = 0; v < V; v++) {
```



```

        if (adj[u][v] && !visited[v]) {
            visited[v] = 1;

            if (matchR[v] == -1 || dfs(matchR[v])) {
                matchR[v] = u;
                return 1;
            }
        }
    }
    return 0;
}

```

```

int maxMatching(int removed) {
    memset(matchR, -1, sizeof(matchR));
    int result = 0;

    for (int u = 0; u < U; u++) {
        if (u == removed) continue;

        memset(visited, 0, sizeof(visited));
        if (dfs(u)) result++;
    }

    return result;
}

```

```

int main() {
    scanf("%d", &U);
    scanf("%d", &V);

    // adjacency matrix input
    for (int i = 0; i < U; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    int removed;
    scanf("%d", &removed);

    int ans = maxMatching(removed);
}

```

```
printf("Maximum matching after removing vertex %d: %d", removed, ans);  
return 0;  
}
```

Status : Correct

Marks : 10/10